
Software Requirements Specification

for

I-Fest Database Management System

Prepared by :

Diya Shah (202512025)
Mahi Kansara (202512078)

Group Number :

18

Dhirubhai Ambani University

17th November , 2025

Table of Contents

1.Introduction.....	4 - 12
1.1 Purpose.....	4
1.2 Document Conventions	10
1.3 Intended Audience and Reading Suggestions.....	10
1.4 Product Scope.....	11
1.5 References.....	12
2. Requirements Gathering.....	15 - 28
2.1 Interview.....	15
2.2 Questionnaire	18
2.3 Background Reading.....	24
2.4 Prototype Approach	25
2.5 Fact Finding Chart	26
2.6 Combined Requirements	28
3. Overall Description	29 - 39
3.1 Product Functions	29
3.2 Privileges	30
3.3 User Classes and Characteristics	31
3.4 Operating Environment	38
3.5 Assumptions and Dependencies	38
4. External Interface Requirements.....	40
3.1 User Interfaces	40
3.2 Hardware Interfaces	40
3.3 Software Interfaces	40
3.4 Communication Interfaces.....	40
5. Other Non-Functional Requirements.....	41
5.1 Software Quality Attributes.....	41

5.2 Business Constraints.....	41
6. Extended Noun and Verb Analysis	42 - 52
7. Database Design	53 - 74
7.1 ER Diagram	53
7.2 Relational Model	54
7.3 Normalization	55
7.4 Functional Dependencies	58
7.5 Functional Dependency Diagram.....	58
7.6 Final Normalized Tables.....	71
8. Schema	76 - 91
9. Complex SQL Queries	92 - 130
10. Database Connectivity (Using JDBC).....	131 - 133
11. CRUD Operations Code	134 - 138
11.1 Create.....	135
11.2 Read	136
11.3 Update	137
11.4 Delete.....	138
12. Frontend Screenshots	139 - 148
13. Conclusion.....	149

1. Introduction

1.1 Purpose

The i-Fest is an annual event organized by DAU (Dhirubhai Ambani University) that includes multiple technical and non-technical events such as workshops, hackathons, quizzes, cultural programs, exhibitions, and celebrity performances. Each year, the management of participant registrations, scheduling of events, certificate generation, prize distributions, and feedback collection is performed manually or using basic tools such as Google Forms and Excel spreadsheets.

This approach introduces several inefficiencies that significantly affect the smooth functioning of the event. The current manual practices are prone to data duplication, entry errors, lack of real-time updates, poor data integrity, and slow processing times, which hinder the overall experience for both participants and organizers.

1. KEY PROBLEMS IN CURRENT SYSTEM

1.1 Duplicate Registrations

Currently, participant registrations are handled via Google Forms. There are no built-in mechanisms to prevent participants from registering multiple times using the same email or phone number.

Example Scenario:

During the last i-Fest, some participants accidentally submitted multiple registration forms using the same email address. This resulted in multiple entries in Excel sheets:

Name	Email	Event
Diya Shah	diya.shah@gmail.com	Hackathon 1
Diya Shah	diya.shah@gmail.com	Hackathon 1

Organizers had to manually check and delete duplicate entries, consuming valuable time and increasing the chance of human error.

1.2 Incorrect Certificates :

Certificates are generated manually by extracting participant data from Excel sheets and copying them into certificate templates.

This manual process is error-prone.

Example Scenario:

A participant, "Neha Shah", who participated in "Coding Challenge", was issued a certificate with her name misspelled as "Naha Shah". The event title was also wrongly written as "Code Challenge" instead of "Coding Challenge". These mistakes caused dissatisfaction among participants and damaged the event's reputation.

1.3 Lack of Real-Time Schedule Updates :

Event schedules are communicated to participants via static PDFs or word-of-mouth announcements. If any schedule changes occur (e.g., venue change, time change), the participants are not notified in real-time.

Example Scenario:

On the day of the event, the venue for the "Quiz Competition" was changed from Room A to Room B. The volunteers were tasked with informing participants, resulting in miscommunication. Many participants missed the event or arrived late, creating chaos.

1.4 Manual Feedback Collection

Feedback collection is conducted by distributing paper-based forms after events. Organizers need to collect, digitize, and analyze this feedback manually, leading to delays in understanding participant opinions.

Example Scenario:

At the end of i-Fest, 500 feedback forms were manually collected, scanned, and entered into a system for analysis. This delayed actionable insights by several days.

1.5 Inadequate Security & Identification :

No unique identifiers (such as ID cards, QR codes, or Fest IDs) are assigned to participants or volunteers. This leads to difficulties in verifying the identity of entrants during on-site checks.

Example Scenario:

During peak entry times, volunteers had to manually cross-verify participant names with printed Google Sheets. Unauthorized persons often sneaked in due to lack of proper ID verification, creating security vulnerabilities

2. Proposed Solution : Centralized I-Fest Database System

2.1 Unique Identifiers

Each participant is assigned a unique Fest ID and QR code upon successful registration. This prevents duplication and helps in efficient tracking of entries, event participation, and certificate issuance.

Example Workflow:

- During registration: System checks for existing email/phone combinations.
- Fest ID is auto-generated (e.g., FEST2025-0001).
- QR code is linked to the Fest ID.

2.2 Automated Certificate Generation

Certificates are dynamically generated from database data after results are finalized, ensuring accurate participant names, event names, and results.

Example Flow:

Judges enter scores in the system → System calculates results → Certificate is auto-generated → Participant downloads the Certificates.

2.3 Real-Time Schedule Updates

Event schedules are maintained in the database with capability for instant updates. Participants and volunteers receive automated SMS/email notifications when there are changes.

Example Flow:

Organizer updates schedule in system → API triggers SMS/Email notifications to all registered participants → Updated schedule appears in participant dashboard in real-time.

2.4 Digital Feedback Collection

Participants can submit feedback online through their dashboard. The system stores, analyzes, and generates reports on feedback in real-time.

Example Scenario:

After the event, a participant logs in, selects the event attended, rates it from 1 to 5, and adds comments. The system aggregates data to generate insights.

2.5 Secure Role-Based Access

The system defines multiple user roles: Admin, Student, Judge, Sponsor, Faculty/Mentor, Celebrity, External Team, Audience. Each role has limited access to ensure security.

Role	Access Privileges
Admin	Full CRUD on events, participants, results, certificates
Judge	Enter scores, view assigned events
Sponsor	View event reports, prize allocations
Participant	Register, view schedule/results/certificates
Visitor	View schedules only

2.6 Team and Individual Registrations

Supports both individual and team registrations. A team leader is identified, and member details are linked, helping judges evaluate teams systematically.

Example Scenario:

A team of 3 external members registers under a Hackathon event. The system assigns a Team ID and links all members under that ID.

2.7 Event Capacity and Deadline Enforcement

The system prevents overbooking and ensures event registration deadlines are followed.

Example Scenario :

An event has a capacity of 100 participants. When 100 registrations are reached, the system automatically disables new registrations for that event.

2.8 Archiving of Past Events

All past i-Fest data (participants, schedules, certificates, results) is stored in the database for future reference.

Example Scenario:

An organizer wants to review last year's technical event participation trends. The system retrieves the past data efficiently for analysis.

1.2 Document Conventions

Fonts and Styles

- Headings: Bold, 18pt, Times New Roman
- Subheadings: Bold, 16pt, Times New Roman.
- Body Text: Normal text, 14pt, Calibri.

1.3 Intended Audience and Reading Suggestions

Intended Audience :

- **Organizing Committee:** To manage registrations, avoid duplicates, generate accurate certificates, and collect feedback.
- **Department Faculty:** To oversee technical and research events with reliable participant data.
- **DAU Students and External Teams:** Participants for tech and non-tech events such as hackathons, escape room, coding races, and cultural programs. To register smoothly and receive error free e-certificates.
- **Judges and Mentors:** To evaluate projects, and similar more tech events.
- **Sponsors:** To monitor event outreach, sponsored prizes, and visibility at celebtech interactions.
- **General Audience and Celebrities:** For attending flagship events like standup comedies, concerts and to benefit from a well-managed and improved event experience.

Reading Suggestions :

- New readers can start with Introduction to understand the purpose of the project.
- Organizers and management should focus on the system features and functionalities and planning events accordingly.
- Developers should carefully read the technical requirements and database details.

1.4 Product Scope

- Manage technical events like workshops, hackathons, Robo-Clash, Quizzes, i.Papyrus, etc.
- Manage non-technical like Treasure Hunt, i.Maze, Chess64 , etc.
- Track registrations of individuals and teams with mechanisms to prevent duplicate entries.
- Store details of judges, sponsors, celebrities, and volunteers.
- Support scheduling for three days including parallel events and celebrity sessions.
- Allow submission and review of technical papers.
- Record event results, generate error-free e-certificates, and track prizes.
- Maintain links of all the past year's i-Fest events.
- Ensure a unique and effective database design that improves accuracy, efficiency, and scalability.
- Participants sometimes register multiple times using the same email or phone number, resulting in multiple entries for the same person in the Excel sheets. This causes confusion during check-ins and results in multiple certificates being issued for the same person. Solution in Proposed Database System: The system will enforce unique constraints on key fields such as email, phone number and a generated fest_id. During registration, the database checks for existing entries and prevents duplicates by rejecting any record that violates these constraints.
- Manual certificate generation leads to errors like misspelled participant names or wrong event names printed on certificates. Solution in Proposed Database System: Certificates will be generated dynamically using verified data directly from the database. Each certificate will pull the correct participant name, event name, result, and date from the structured tables.

1.5 References

<https://aitfest.thunderdevelops.in/>

- User Registration
- Storing detailed information about multiple events (hackathons, workshops, quizzes).
- Different workshops and individual registrations for each using google forms so the database might need reference of these events and will basically store a link to the form URL.
- Storing details of core team members and organizing clubs (like GDSC, Coding Club), and linking them to the events they manage.
- A timeline of the whole overall event along with dates, timings and detailed information along with the directions for each event on campus.
- Storing and displaying contact details for main organizers and specific event points of contact.

<https://www.techexpogujarat.com/>

- User Registration that means users sign up for updates or register for events.
- Different events including workshops, seminars etc. taking place and their information.
- Each event has a detailed map for making it easier for users to reach the destination.
- Different sponsors with different categories are displayed to users.
- A proper detailed timeline for each event
- Different speakers appear for seminars and their schedules.
- A section where multiple visuals from events are posted that means the photos captured.
- Exhibitor/Company Profiles that are collaborating with the university for different events.

<http://ifest.daiict.ac.in/archive/ifest22/>

- Events section with detailed information about the event and registration status (that means open or closed).
- Contact details about the core committee and team members.
- Details of each member and their allotted events.
- Sponsor's Details recent as well as past for references and their detailed description.
- A section where all the information for social media and connect details are provided.
- All the upcoming events updates.
- Reference links to the past year events of the same fest.

<https://www.knowafest.com/explore/events>

- The site lists multiple events from colleges across India requires a database of events with categories.
- Students browse/search by city, state, department, event type (workshops, fests, symposiums).
- Each event has a name, host college, description, type, department, date, registration link.
- Some events are shown as special or highlighted, so the database should have a field (like a tag or mark) to show their priority/importance.
- Each event is linked to a specific college or department.
- Requires storage of resource persons, schedules, and topics.
- Some events list collaborating organisations.

<https://techfest.org/>

- Students register online for workshops, competitions, and exhibitions.
- Different event types (workshops, competitions, lectures, exhibitions, campaigns) need to be stored with name, date, time, and details.
- Each has its own registration, rules, and seats need separate tables.
- Store details of companies, startups, and guest speakers.
- Store schedules, performers, and audience info.

- Keep data of sponsors/partners (companies, NGOs, UNESCO, UNICEF, etc.).

<https://www.hetc.ac.in/TECHetc2K25/events>

- Students sign up (online or offline) to join coding contests, robotics, quizzes, etc.
- Each event (like coding, quizzes) needs details stored name, date, time, rules, and description.
- Some events may have fees or team limits databases should capture registration links, fees, and seat capacity.
- Store prize info (certificates, kits, snacks) linked to each event.
- A full schedule with dates, times, and venues for each activity.
- Details of faculty, student committees, or clubs managing events.
- Store event-specific rules and team structures (e.g., 2–4 members per team).
- Media & Highlights: Store event photos, videos, or reports after the fest for display or archive.

2 Documenting the Requirement Collection

2.1 Interview Plans

Interview - 1 :

System : I-Fest Database System

Participants : Virag Koradiya (I-Fest Committee Member)

Date : 03-09-2025

Duration : 15 minutes

Purpose of Interview :

Preliminary meeting to identify problems and requirements regarding security at the i-Fest.

1. Registration of the participants takes place using the google forms which then stored in excel sheets so it leads to multiple same entries.
2. Duplication of Entries taking place during registration.
3. Lack of time management due to more than one event being held at the same time.
4. Errors while generating the e-certificates.
5. Chaos is caused during the on venue verification of the participants and visitors.
6. Resolving the payment issues that might be caught during the registration process.
7. No unique identifier (such as Fest ID/ QR Code) is currently assigned to participants ,making it difficult to link registrations, payments, certificates and event entries.
8. Lack of centralized database design that is no role based access (admin/volunteer/participant) or integration with other modules like accommodation, scheduling , or certificate generation.

Interview – 2:

System : I-Fest Database System

Participants : Purav Shah (Outside participant in Hackathon)

Date : 01-09-2025

Duration : 10 minutes

Purpose of Interview :

Preliminary meeting to identify problems and requirements regarding security at the i-Fest.

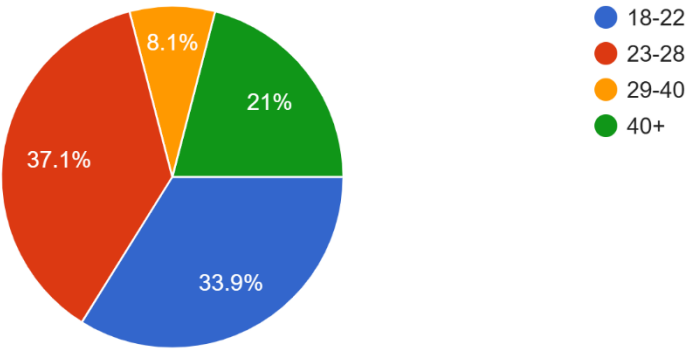
1. Overrush at the main gate during peak time.
2. Very few people to do verification. So lack of volunteers.
3. No digital check-in system that means everything was done manually, causing delays.
4. Lack of accomodations due to the lack of availability of rooms in hostel.
5. During the other i-Fest event, overlapping of events took place.
6. Delays found in all the events which resulted in clashes.
7. No proper complaint or feedback mechanism during fests.
8. Lack of security as no identification measures were taken. That means there were high chances anyone could sneak into the campus at anytime.
9. No ID cards or passes issued for participants and volunteers, leading to confusion at entry points.
10. No live updates or announcements so people miss events.

The key solutions to manage the problems identified in the interviews are :

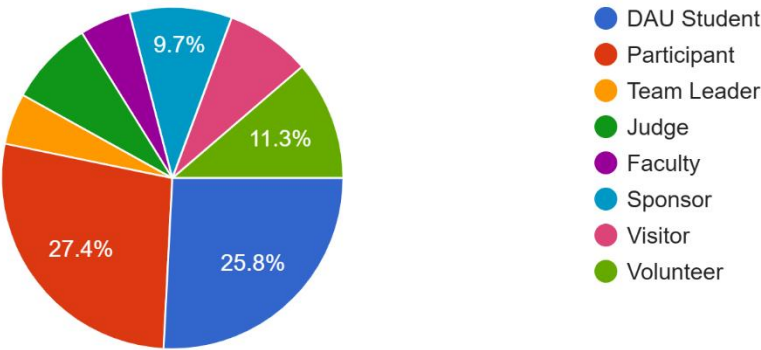
- **Problem: Duplicate Entries & Data Errors**
 - **Solution:** Enforce data integrity with unique identifiers.
 - **Action:** In your Participants table, set ParticipantID as a **Primary Key** and add a **UNIQUE constraint** to the Email field to prevent duplicate submissions. A separate Registrations table with foreign keys to Participants and Events will link data without duplication.
- **Problem: On-Venue Chaos & Lack of Passes**
 - **Solution:** Implement a digital check-in system with a unique QR code.
 - **Action:** Add a field for QRCodeData to the Participants table. Create a new CheckInLogs table with ParticipantID (Foreign Key) to digitally record every entry, replacing the manual process.
- **Problem: Lack of Role-Based Access**
 - **Solution:** Secure the system with role-based access control (RBAC).
 - **Action:** Create a separate Roles table (RoleID, RoleName). The Users table will then have a RoleID foreign key that links to the Roles table, ensuring users can only access the data and functions appropriate for their role (e.g., organizer, volunteer, participant).
- **Problem: No Centralized Feedback Mechanism**
 - **Solution:** Create a dedicated table to store and analyze all feedback.
 - **Action:** Add a new Feedback table with fields such as FeedbackID, ParticipantID, EventID, Rating, and Comments to provide a structured way to capture and analyze all feedback.

2.2 Questionnaire Responses :

Age
62 responses

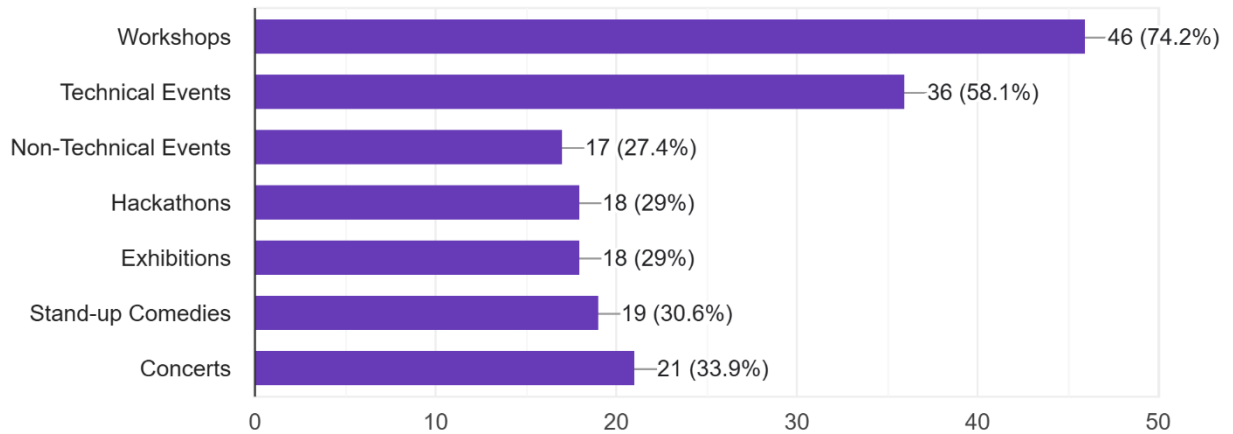


Role
62 responses



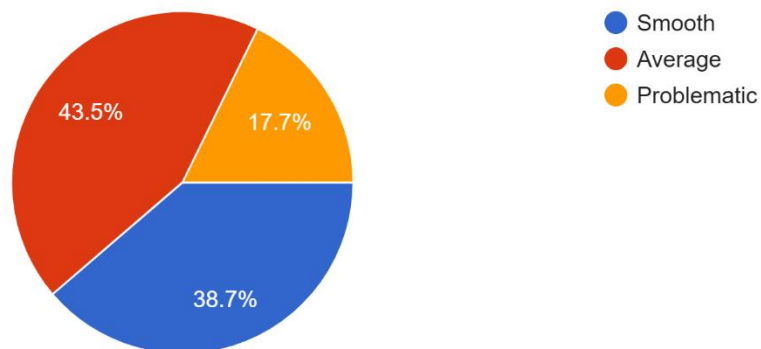
What all events you participated in ?

62 responses



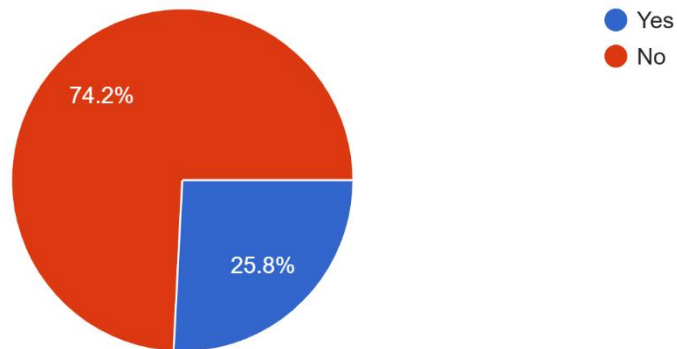
How was your registration experience?

62 responses



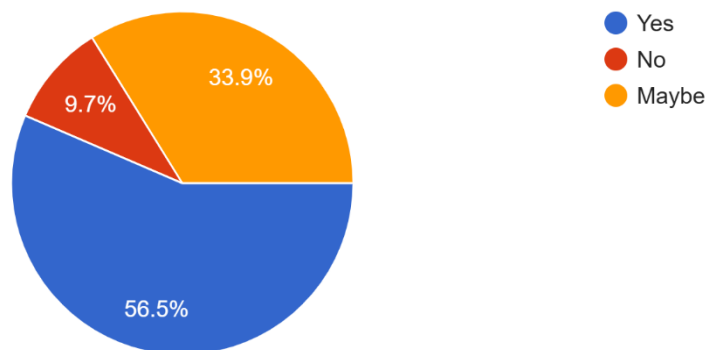
Did you face duplicate entry or login issues?

62 responses



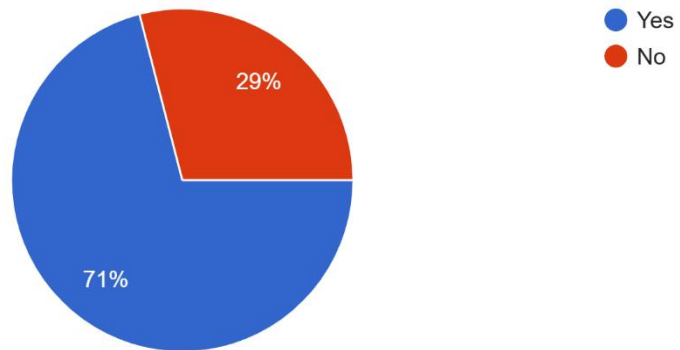
Would a personalized event calendar or real-time notifications help?

62 responses



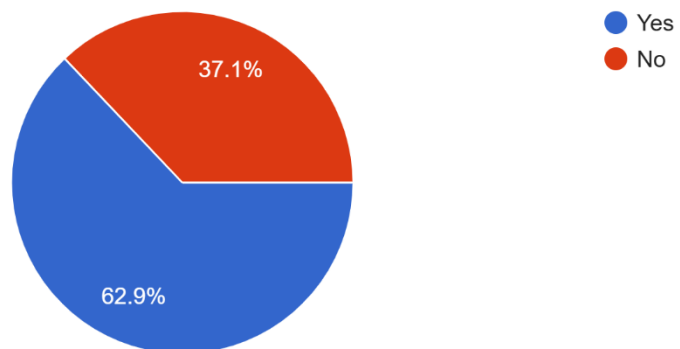
Were results announced on time?

62 responses



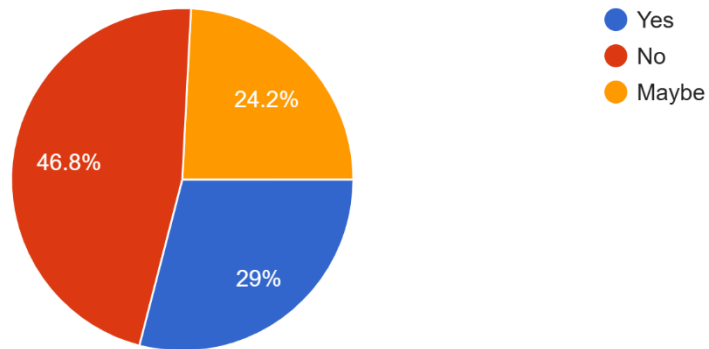
Were your certificates accurate and easy to access?

62 responses



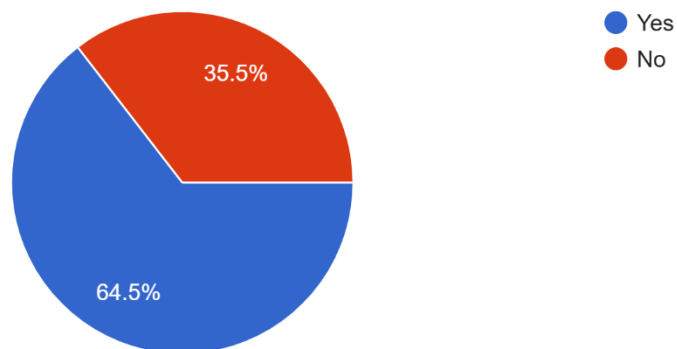
Did you miss important announcements due to lack of notifications?

62 responses



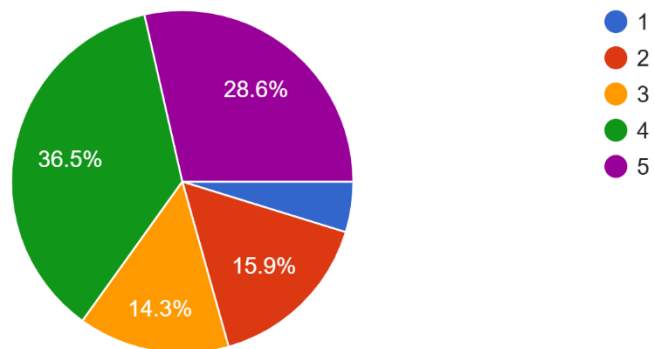
Was there a clear way to give feedback or complaints?

62 responses



Rate your overall i-Fest experience

63 responses



One key suggestion for improvement

63 responses

Add a mobile app calendar.

Better signage at event locations.

Improve schedule notifications beforehand.

Make registration smoother with fewer steps.

Ensure certificates are downloadable easily.

Nothing

Good

2.3 Background Reading :

1. Registration & IDs

- **Observation:** Duplicate registrations were common; students often registered multiple times.
- **Solution:** Implement a **Unique Fest ID / QR Code** generated at registration, stored in the database, preventing duplicate entries.

2. Event Management & Scheduling

- **Observation:** Overlapping and clashing events created confusion for both participants and organizers.
- **Solution:** Introduce a **centralized scheduling module** that automatically checks for conflicts before approving event slots, and provides participants with a **personalized event calendar**.

3. Team Registration

- **Observation:** Team leaders struggled to link members; members got misplaced or unregistered.
- **Solution:** Design a **team registration system** that links a team leader with member profiles (via their Fest IDs). The leader can manage and edit the team structure.

4. Result Processing & Certificates

- **Observation:** Results took a long time to process, and certificates were often delayed.
- **Solution:** Provide a **judge scoring interface** where multiple judges can enter scores directly. Results are auto-calculated and **certificates auto-generated** with participant details and results.

5. Communication (Notifications)

- **Observation:** Participants missed last-minute schedule changes and announcements.
- **Solution:** Add a **real-time notification system** (via email/SMS/app push notifications) for updates on schedules, results, or announcements.

6. Feedback & Complaints

- **Observation:** No structured system to handle feedback or complaints.
- **Solution:** Include a **feedback/complaint submission module** linked to the Admin dashboard for review and action.

Observation

- Volunteers spent significant time verifying participants during fest check-in, without any automated ID system.
- Certificate lists were compiled by sorting and filtering sheets, which occasionally included the wrong participants due to misspellings or duplicate records.

2.4 Prototype Approach

- No automatic duplicate prevention in Google Forms; prototype enforces unique IDs and contacts.
- Teams registered separately before; prototype manages teams as a whole with linked members.
- No event capacity or deadline enforcement earlier; prototype blocks over-registration.
- Data errors common as no field validation; prototype validates inputs in real time.
- Slow, manual venue check-ins without unique IDs; prototype uses QR code/Fest ID scanning.
- Certificates prepared manually; prototype auto-generates error-free certificates.
- No automatic notifications for schedule changes; prototype sends email/SMS alerts.
- No role-based security before; prototype restricts data access by user roles.

2.5 Fact Finding Chart

Method Used	Purpose	Data Collected	Remarks
Observation	To study how registration currently happens	Registration done via Google Forms; responses go into Google Sheets. Organizers manually check entries for duplication and correctness.	DBMS must import/store responses from Google Sheets into event-specific tables.
Interview (Organizer)	To understand what data is needed from form responses	Organizers want: participant info (name, email, phone), event choice, team details, payment status, accommodation request.	DBMS must normalize form data into structured tables (Participants, Events, Teams, Payments, Accommodation).
Interview (Participant)	To check if participants face issues in form-based registration	Participants get no confirmation of hostel allotment or real-time schedule updates from the form.	DBMS should generate confirmation records and allow live updates linked with form data.
Questionnaire	To collect wider requirements from students/mentors	Users suggested need for: certificate generation, ID/pass system, prize tracking, event result announcements.	DBMS must integrate extra features not supported by Google Forms (certificates, ID generation, announcements).

Document Review (Past Fests & References)	To analyze existing fest registration systems and how they manage external form inputs	Found other fests use Google Forms + custom DBMS. Forms store basic data, DBMS handles scheduling, results, certificates, sponsor data, and archives.	i-Fest DBMS should be designed to link Google Form data with internal DB system for extended functionalities.
Prototyping	To get early feedback on system interface and workflow from stakeholders	Created mockups of the registration dashboard, event schedule management, and certificate generation modules. Collected stakeholder comments.	Helped refine interface design and workflow to be more user-friendly and aligned with real-world usage before full system development.
Broader Stakeholder Input	To capture comprehensive requirements from additional stakeholder groups	Sponsors want visibility tracking, prize allocation features; Judges/Mentors want evaluation workflows and plagiarism checks; Volunteers need efficient check-ins.	Inputs led to designing modules such as sponsor management, role-based access control, digital check-ins, feedback system, and detailed result management.

2.6 Combined Requirements for i-Fest Database Management System

Requirement	Frequency (Occurrences)	Notes / Source
User Registration System (via Google Forms, stored in DBMS)	★★★★ (4)	Mentioned in Purpose, Product Scope, References, Interviews
Unique User ID (prevent duplicate registrations)	★★★ (3)	Assumptions, Interview (organizers), Observation
Event Management (Create, Edit, Delete events, categories: Tech/Non-Tech/Celebrity)	★★★★ (4)	Product Functions, Admin Privileges, Scope
Team Registration Handling (link leader + members)	★★ (2)	Questionnaire, Event Scope
Scheduling System (avoid event clashes, track timelines)	★★★ (3)	Purpose, Product Scope, Observation
Result Management (Judges entering scores, generating results)	★★★ (3)	Product Functions, Judge Privileges, Interviews
Automated Certificate Generation (with participant name, event, result)	★★★ (3)	Product Functions, Assumptions, Questionnaire
Prize Tracking (linked with results/certificates)	★★ (2)	Product Scope, Functions
Sponsor Management (store sponsors, link with events, prize info)	★★ (2)	Product Scope, References
Dashboard for Different Roles (Admin, Student, Judge, Sponsor, Audience)	★★★ (3)	Privileges, User Classes, Scope
Security & Access Control (Role-based privileges, ID cards/passes, login system)	★★★ (3)	Interviews, Privileges, Assumptions
Complaint/Feedback System (for participants)	★★ (2)	Observation, Interviews
Archive Past Events Data (results, certificates, media, reports)	★★ (2)	Product Scope, References
Notifications/Announcements (real-time updates)	★★ (2)	Interviews, External Interface (API/SMS/Email)
Paper Submission & Review (for research events)	★★ (2)	Product Scope, Faculty/Mentor Privileges
Scalability (handle large number of registrations/events)	★★ (2)	Business Constraints, Assumptions
Performance Reports (participation stats, sponsor visibility, judge evaluations)	★★ (2)	Sponsors Privilege, Product Functions

3. Description

3.1 Product Functions

Participant Management Functions

- Register new participants with unique identification
- Validate participant data to prevent duplicate entries
- Update existing participant information
- Generate unique participant IDs for tracking
- Handle group/team registrations with team leader designation
- Link individual participants to their respective teams
- Generate unique team IDs and manage team membership
- Store team-specific information (team name, category, member count)
- Validate team size constraints based on event requirements

Event Registration Functions

- Link participants to specific events/activities
- Track registration status and timestamps
- Manage event capacity and waitlist functionality
- Store event-specific participant data
- Handle bulk registration operations

Data Integrity Functions

- Enforce referential integrity between tables
- Implement data validation rules and constraints
- Maintain audit trails for all data changes
- Perform regular data consistency checks
- Handle concurrent access and locking mechanisms

Certificate Management Functions

- Generate accurate participant lists for certificate printing
- Validate participant eligibility for certificates
- Track certificate generation and distribution status
- Store certificate metadata and templates
- Link certificates to specific events and participants
- Maintain certificate numbering sequence for uniqueness
- Store certificate issue dates and print status

Feedback System Functions

- Capture structured feedback from participants and visitors
- Store feedback with proper categorization and ratings
- Link feedback to specific events and time periods
- Support multiple feedback formats (ratings, comments, surveys)
- Enable feedback analysis and reporting queries

3.2 Privileges

User Class	Privileges
Admin	Full CRUD access on all tables (Events, Registrations, Judges, Sponsors, Results, Papers, Certificates, Prizes, Archive Wall). Can create new schemas, update schedules, manage relationships between entities, and back up/restore data.
DAU Student	Insert (registration, paper submissions), Update (personal/team info), Read (events, schedules, results, certificates), No Delete privilege.
External Team	Insert (team registration, project submissions), Update (team details), Read (results, event info, certificates), No Delete privilege.
Judge	Insert (scores, evaluations), Update (remarks), Read (event details, team info, assigned papers), No Delete privilege.
Faculty/Mentor	Read (papers, team submissions), Update (review status, plagiarism checks), Insert (remarks/feedback), limited Delete (can reject submissions before acceptance).
Sponsor	Read (event info, reports, brand exposure data), Insert (sponsored prize info), Update (sponsorship details), No Delete privilege.
Audience/Visitor	Strictly Read-only (event schedules, results, archive wall). No write privileges.

3.3 User Classes and Characteristics

User Name	Role
Admin	Manages the entire system. Creates, edits, and deletes events. Assigns judges, approves sponsors, monitors registrations, schedules events, and updates the archive wall.
Student (Internal)	Registers for events, joins teams, submits technical papers, views results, downloads certificates, and claims prizes.
External Team	Registers via team leader, joins hackathons/competitions, accesses results, and downloads certificates.
Judge	Evaluates projects, hackathons, reverse hackathons, and papers. Inputs scores, provides remarks, and finalizes winners.
Sponsor	Provides support (financial/material), monitors sponsored events, tracks prize allocations, and receives engagement reports.
Faculty/Mentor	Oversees technical papers and Reality Lab projects, validates plagiarism checks, mentors students, and ensures academic quality.
Audience/Visitor	General participants attending shows, exhibitions, and non-competitive events. Can view schedules, updates, and archives (read-only access).

Participants

Attribute	Type	Constraints
participant_id	INTEGER	PRIMARY KEY
fest_id	VARCHAR	UNIQUE (QR/ID for tracking)
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
phone	VARCHAR	UNIQUE, NOT NULL
gender	ENUM('Male', 'Female', 'Other')	-
role	ENUM('Student', 'External', 'Visitor')	-
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
qr_code_img	BLOB	NOT NULL

- Stores unique records of all individuals.
- fest_id, email, and phone ensure no duplicate participants.
- Used as the base table to link with registrations, certificates, results, accommodation, and feedback.

Events

Attribute	Type	Constraints
event_id	INTEGER	PRIMARY KEY
event_name	VARCHAR	NOT NULL
category	ENUM('Technical', 'Non-Technical', 'Cultural', 'Celebrity')	-
date	DATE	NOT NULL
time	TIME	NOT NULL
venue	VARCHAR	-
capacity	INTEGER	-

- Stores details of each fest event (technical, non-technical, cultural, celebrity).
- event_name + date + time should ideally be unique to avoid scheduling clashes.
- Works as a parent entity for Teams, Registrations, Results, Certificates, and Feedback.

Teams

Attribute	Type	Constraints
team_id	INTEGER	PRIMARY KEY
team_name	VARCHAR	NOT NULL
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
leader_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)

- team_name + event_id should be unique (no duplicate team names in the same event).
- leader_id ensures every team has one designated leader.
- Connected to Registrations and Results for tracking team performance

Registrations

Attribute	Type	Constraints
registration_id	INTEGER	PRIMARY KEY
participant_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
team_id	INTEGER	FOREIGN KEY REFERENCES Teams(team_id) NULLABLE
registered_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

- team_id nullable → if NULL, participant registered as individual.
- UNIQUE(participant_id, event_id) ensures no participant registers twice for the same event.
- Prevents participants from joining two different teams in the same event.

Judges

Attribute	Type	Constraints
judge_id	INTEGER	PRIMARY KEY
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
phone	VARCHAR	UNIQUE, NOT NULL
expertise	VARCHAR	-

- linked to event evaluation and results.
- email and phone ensure no duplicate judge records.

Sponsors

Attribute	Type	Constraints
sponsor_id	INTEGER	PRIMARY KEY
name	VARCHAR	NOT NULL
contact_person	VARCHAR	-
email	VARCHAR	UNIQUE, NOT NULL
phone	VARCHAR	UNIQUE, NOT NULL
sponsorship_type	VARCHAR	-

Certificates

Attribute	Type	Constraints
certificate_id	INTEGER	PRIMARY KEY
participant_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
issue_date	DATE	-
certificate_number	VARCHAR	UNIQUE

- Auto-generates certificates based on validated participant-event pairs.
- certificate_number ensures each certificate is unique.
- UNIQUE(participant_id, event_id) prevents issuing multiple certificates for the same event.

Feedback

Attribute	Type	Constraints
feedback_id	INTEGER	PRIMARY KEY
participant_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
rating	INTEGER	CHECK (rating BETWEEN 1 AND 5)
comments	TEXT	-
submitted_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

- Linked to both participant and event.

Results

Attribute	Type	Constraints
result_id	INTEGER	PRIMARY KEY
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
participant_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id) NULLABLE
team_id	INTEGER	FOREIGN KEY REFERENCES Teams(team_id) NULLABLE
position	ENUM('Winner', 'Runner-Up', 'Finalist')	-
prize_details	VARCHAR	-

- Works for both individuals and teams (only one of participant_id or team_id required).

Archive

Attribute	Type	Constraints
archive_id	INTEGER	PRIMARY KEY
Year	YEAR	NOT NULL, must be a valid year ($\geq$ first fest year)
event_links	TEXT	-
Highlights	TEXT	-

Equipment

Attribute	Type	Constraints
equip_id	INTEGER	PRIMARY KEY
name	VARCHAR	NOT NULL
category	VARCHAR	sound, lights, projectors, etc.
total_quantity	INTEGER	NOT NULL
available_quantity	INTEGER	NOT NULL
status	ENUM('available', 'in-use', 'under-maintenance')	DEFAULT 'available'
description	TEXT	-
last_maintenance_date	DATE	-

Resource Allocation

Attribute	Type	Constraints
allocation_id	INTEGER	PRIMARY KEY
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)
equip_id	INTEGER	FOREIGN KEY REFERENCES Equipment(equip_id)
user_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)
quantity_reserved	INTEGER	NOT NULL
start_time	DATETIME	NOT NULL
end_time	DATETIME	NOT NULL
return_status	ENUM('returned', 'not-returned', 'damaged', 'missing')	DEFAULT 'not-returned'
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
UNIQUE	-	event_id, equip_id, start_time, end_time (Prevent double-booking)

Equipment Audit Log

Attribute	Type	Constraints
log_id	INTEGER	PRIMARY KEY
allocation_id	INTEGER	FOREIGN KEY REFERENCES Resource_Allocation(allocation_id)
equip_id	INTEGER	FOREIGN KEY REFERENCES Equipment(equip_id)
event_id	INTEGER	FOREIGN KEY REFERENCES Events(event_id)

user_id	INTEGER	FOREIGN KEY REFERENCES Participants(participant_id)
action_type	ENUM('reserved', 'returned', 'damaged', 'missing')	-
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP
notes	TEXT	-

3.4 Operating Environment

i. Hardware, Software, or Connectivity Requirements :

Hardware:

- Server/PC with minimum 4 GB RAM, 500 GB storage, and i3 or higher processor.
- End-user devices (laptop/desktop/mobile) with basic browsing capability.

Software:

- Operating System: Windows/Linux/macOS (for server and clients).
- Database: PostgreSQL/MySQL.
- Backend: PHP/Python/Java.
- Frontend: HTML,CSS,JS.

Connectivity:

- Stable internet or LAN connection.
- Minimum bandwidth of 1 Mbps for smooth usage.

3.5 Assumptions and Dependencies

Registration via Google Forms

- All participants will enter correct and complete details (name, email, phone, team info) when filling Google Forms.
- Each participant/team will use a unique email ID for registration to avoid duplicate entries.
- Data exported from Google Forms (Google Sheets) will be accurate and imported into the DBMS without errors.

Event Registration & Scheduling

- Event schedules, rules, and seat capacities will be correctly entered into the database by organizers.
- Registration deadlines will be strictly followed, and late entries will not be included in the system.
- Each event record in the DBMS will correctly link to its registered participants and teams.

Certificates & Results

- Certificates will be auto-generated only for verified participants who completed registration.
- Names and details from registration will be directly used for certificate generation (system assumes no spelling mistakes in names).
- Judges/organizers will submit event results on time and without errors.
- Certificates and prizes will be generated based on the results stored in the database.

System Reliability

- The database will not contain duplicate participant or event records.
- Organizers will verify registration and result data before certificate distribution.
- Past i-Fest event data will be available for upload into the DBMS archive for referenc

4. External Interface Requirements

User Interface:

- Web-based responsive interface accessible via browser.
- Simple dashboard for Admin, Event Coordinators, Students, and Judges.

Hardware Interface:

- Standard keyboard, mouse, or touchscreen for input.
- Printer support (for generating certificates, results, etc.).

Software Interface:

- Database server connection (PostgreSQL/MySQL).
- Email/SMS gateway for sending event updates, OTPs, and notifications.

Hardware/Software/Third-Party APIs & External Sources

APIs/Services Used:

- Email API (e.g., Gmail SMTP, SendGrid) for notifications.
- SMS Gateway API (optional) for mobile alerts.

External Sources:

- Standard libraries/frameworks (e.g., Bootstrap for UI, PHP/JavaScript libraries).
- Open-source authentication libraries (for login & security).

5. Other Nonfunctional Requirements

5.1 Software Quality Attributes

- **Usability** – The system will be easy to use for students, organizers, judges, and sponsors. The interface will be clear and simple so that even first-time users can use it without difficulty.
- **Reliability** – The system will work properly during the fest without crashing. Data will always be safe and will not get lost.
- **Performance** – The system will be fast. Even if many students register at the same time, it will not become slow.
- **Scalability** – The system will be able to handle more participants, more events, and bigger fests in the future without major changes.
- **Security** – The system will be safe. Only authorized people (admins, judges, etc.) will have access to sensitive data. Passwords and personal details will be protected.
- **Correctness & Robustness** – The system will check that data entered is correct (like valid email and no duplicate registrations) and will not fail if wrong data is entered.
- **Reusability** – The main modules like Registration, Certificate Generation, and Feedback will be reusable for future i-Fests or similar college events.

5.2 Business Constraints

- System must be operational before fest week.
- Scalability to handle multiple registrations across 3 days.
- cost-efficient technologies to be used to stay within student fest budget.
- Event schedules must avoid clashes.
- Data security for student records, sponsor agreements, and celebrity contracts.
- Certificates must be auto-generated.
- All payments must be recorded digitally.

6. Extended Noun & Verb Analysis

The table below lists an extended collection of Nouns and Verbs extracted from the problem description of our i-Fest Management System.

Nouns	Verbs
i-Fest	manage
participant	eliminate
data	ensure
error	prevent

entry	capture
system	analyze

experience	improve
organizer	store
certificate	generate
volunteer	communicate
registration form	submit
spreadsheet	cross-check
participant ID	assign
QR code	link

database	validate
feedback	submit
notification	send
role	define
team	support
judge	evaluate
result	calculate
report	generate
sponsor	monitor
archive	store

API	trigger
-----	---------

registration	track
scheduling	organize
paper	review
event	record
prize	track

volunteer	assign
user	authenticate
profile	update
login	validate
logout	terminate
session	manage
email	notify
SMS	alert
committee	approve
faculty	oversee
mentor	guide
celebrity	appear

audience	attend
competition	participate
seminar	conduct
workshop	host

technical event	schedule
-----------------	----------

non-technical event	organize
cultural event	promote
exhibition	display
technical paper	submit
team leader	coordinate
participant group	register
evaluation form	fill
plagiarism check	perform
feedback form	collect
media	upload
sponsor agreement	store
contract	manage
guest speaker	invite

document	archive
task	assign
rule	define
capacity	limit

seat	reserve
deadline	enforce
input validation	perform

error log	maintain
user interface	design
backend	develop
frontend	build
API key	generate
authentication	verify
authorization	implement
data consistency	maintain
concurrent access	control
audit trail	record
database schema	define

data normalization	apply
data import	perform
data export	enable
server	configure
bandwidth	ensure
connection	establish
hardware	support
software	install
third-party API	integrate
SMS gateway	connect

email server	configure
password	encrypt
user session	track
role management	implement
access control	enforce
guest access	restrict
system log	generate
performance report	generate

database backup	schedule
data recovery	implement
mobile device	support
responsive design	apply
certificate template	store
notification template	customize
event log	track
plagiarism report	generate
technical resource	allocate
session timing	schedule
event venue	assign
data field	validate
form submission	handle
confirmation email	send
feedback analysis	perform
participant record	update
score entry	record

result summary	generate
----------------	----------

attendance list	maintain
event calendar	update
system alert	trigger
activity log	record
system update	deploy
error message	display
user guide	provide
system documentation	maintain

Nouns, Attributes, and Relationships

Step 1: Criteria for Truncating Initial Noun List

The initial noun list was reduced using the following criteria:

- a. Duplicates (D): Removed duplicate nouns that represent the same entity.
- b. Irrelevant (I): Removed entities not part of the intended system.
- c. General (G): Removed broad nouns that cannot be useful entities.
- d. Attributes (At): Retained some words as attributes of entities rather than entities themselves.
- e. Associations (As): Removed words that represent relationships rather than entities.

Step 2: Candidate Entity Set

Candidate Entity Set	Candidate Attribute Set	Candidate Relationship Set
Participant	Name, Email, Phone, Fest_ID, QR_Code	Registers for Event, Submits Feedback, Receives Certificate
Event	Event_ID, Name, Date, Time, Venue, Category, Capacity	Has Registrations, Generates Results, Linked with Certificates
Team	Team_ID, Team_Name, Leader_ID, Member_Count	Belongs to Event, Formed by Participants
Registration	Registration_ID, Timestamp, Status	Links Participant with Event/Team
Judge	Judge_ID, Name, Expertise	Evaluates Event, Assigns Results
Sponsor	Sponsor_ID, Name, Sponsorship_Type	Sponsors Event, Allocates Prize
Certificate	Certificate_ID, Certificate_Number, Issue_Date	Awarded to Participant/Event
Feedback	Feedback_ID, Rating, Comments, Date	Given by Participant for Event

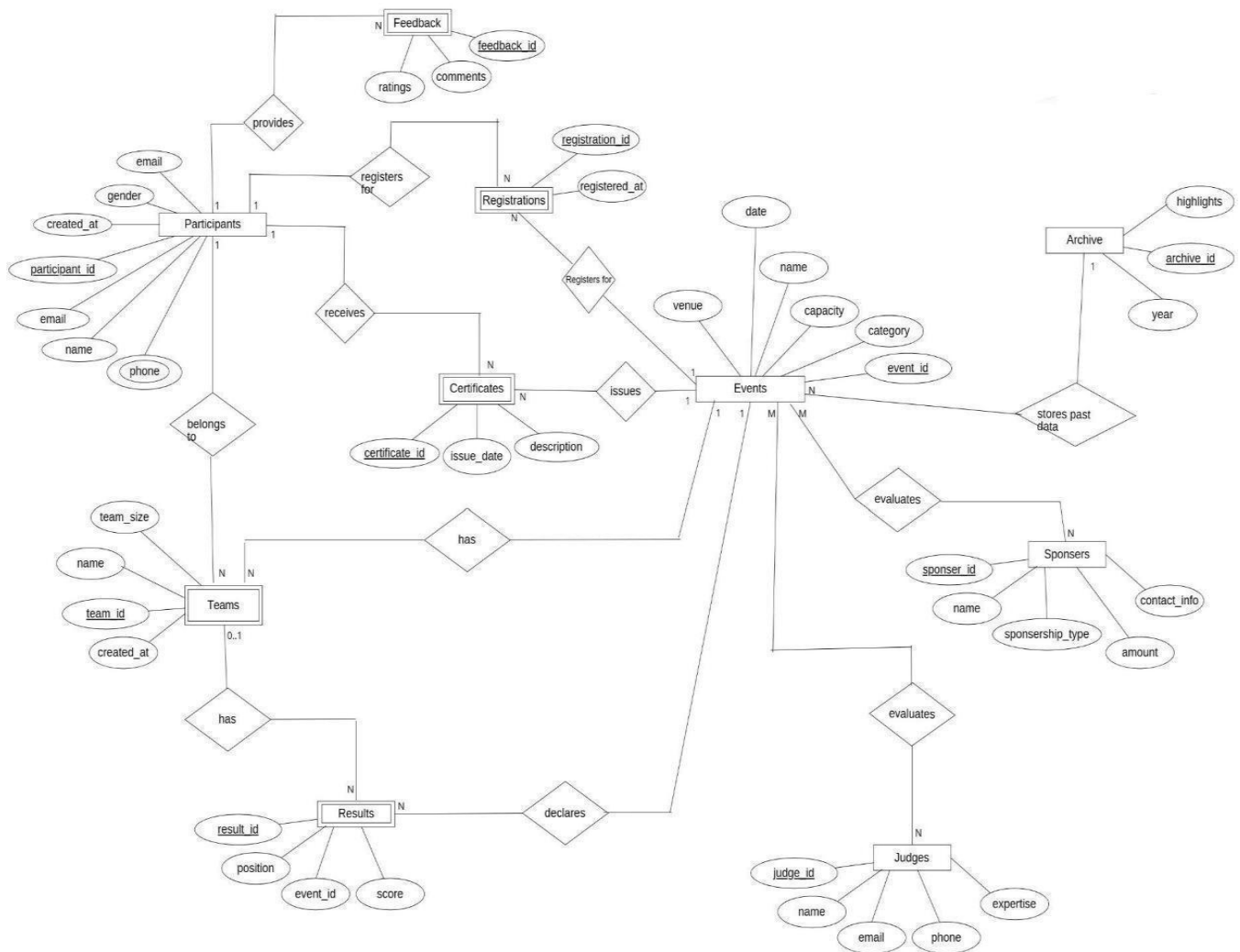
Result	Result_ID, Position, Prize_Details	Generated from Judge evaluations for Event
Archive	Year, Event_Links, Highlights	Stores past Fest Data

Table 3. Rejected Noun & Verbs list

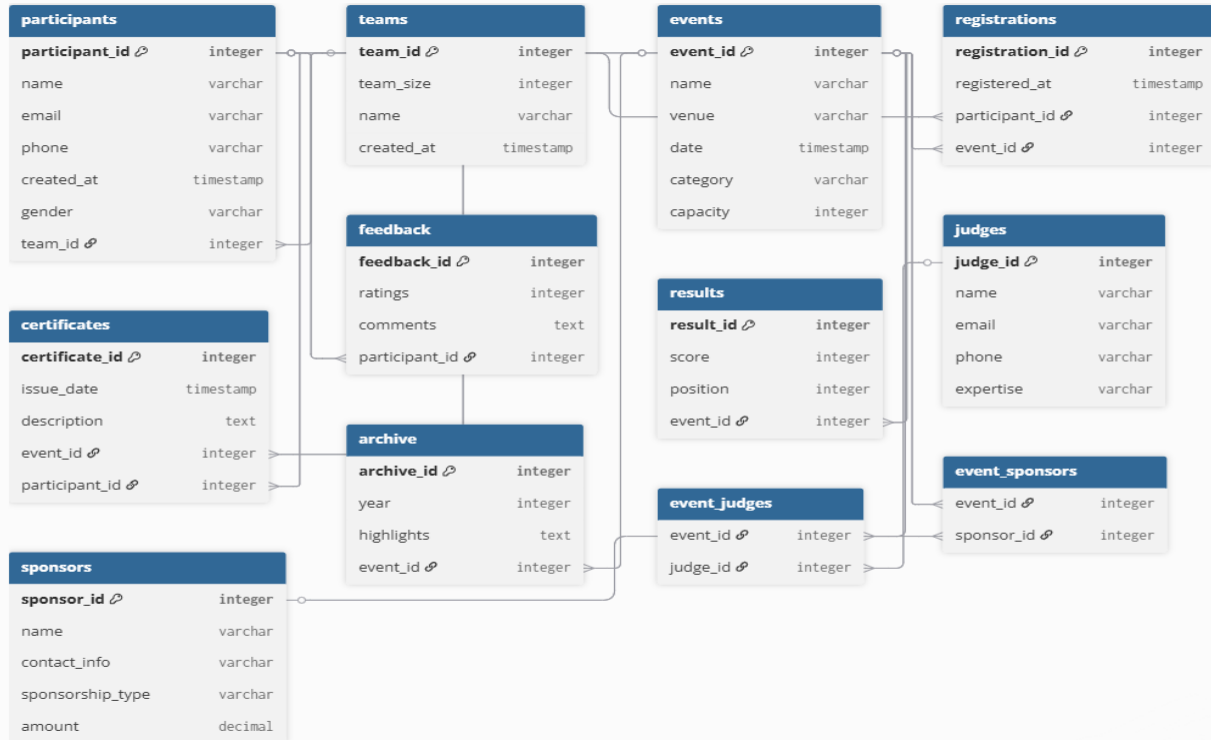
Noun	Reject Reason
Data	General term, used as attribute not entity
Error	Irrelevant to entity set
Experience	General, not a concrete entity
Spreadsheet	Tool, not part of core system entities
Paper	Medium, irrelevant as system entity
Contract	Considered as attribute under Sponsor Agreement
Hardware	Infrastructure, not a direct system entity
Software	General term, not specific to system entities
Media	General, not concrete enough
Committee	Role/association, not entity
Audience	Irrelevant, not stored in system database
Celebrity	Irrelevant, only appears for event hosting
Mentor	Association, not stored as entity
Faculty	Association, not stored as entity
Session	Represents scheduling, already covered under Event
Task	Represents activity, not entity
Rule	Represents system constraints, not entity

7. Database Design

7.1 Entity-Relationship Diagram



7.2 Relational Model



7.3 Normalization to 2NF (Expanded with Dependencies Anomalies)

Stage 1: Unnormalized Tables (UNF)

These are raw forms containing repeating groups and non-atomic attributes.

Participants :

(participant_id,name,email,phone,gender,created_at,team_id,feedback_id,feedback_comments,feedback_ratings)

Teams :

Teams(team_id,name,team_size,created_at,member_list[member1,member2,member3,...])

Events :

Events(event_id,name,date,venue,capacity,category,judges_list[{judge_name1,expertise},{judge_name2,expertise},...],results[{team_id,position,score},...],certificates[{participant_id,issue_date,description},...])

Registrations :

Registrations(registration_id,registered_at,participant_id,event_id)

Archive :

Archive(archive_id,year,highlights,event_list[{event_id,name,date},...])

Anomaly Analysis :

Table	Redundancy	Insertion	Deletion	Update
Participants	Feedback info repeated	Cannot add participant without feedback	Deleting feedback removes participant	Updating feedback requires multiple updates
Teams	Member list repeated	Cannot add member without full team info	Deleting member removes team	Changing member affects multiple records
Events	Judges/results repeated	Cannot add event without sublists	Deleting result removes event	Updating judge info in many rows
Archive	Event info repeated	Cannot add year without events	Deleting event removes archive entry	Changing event affects multiple rows

First Normal Form (1NF)

Removed repeating groups, ensured atomicity, introduced foreign keys.

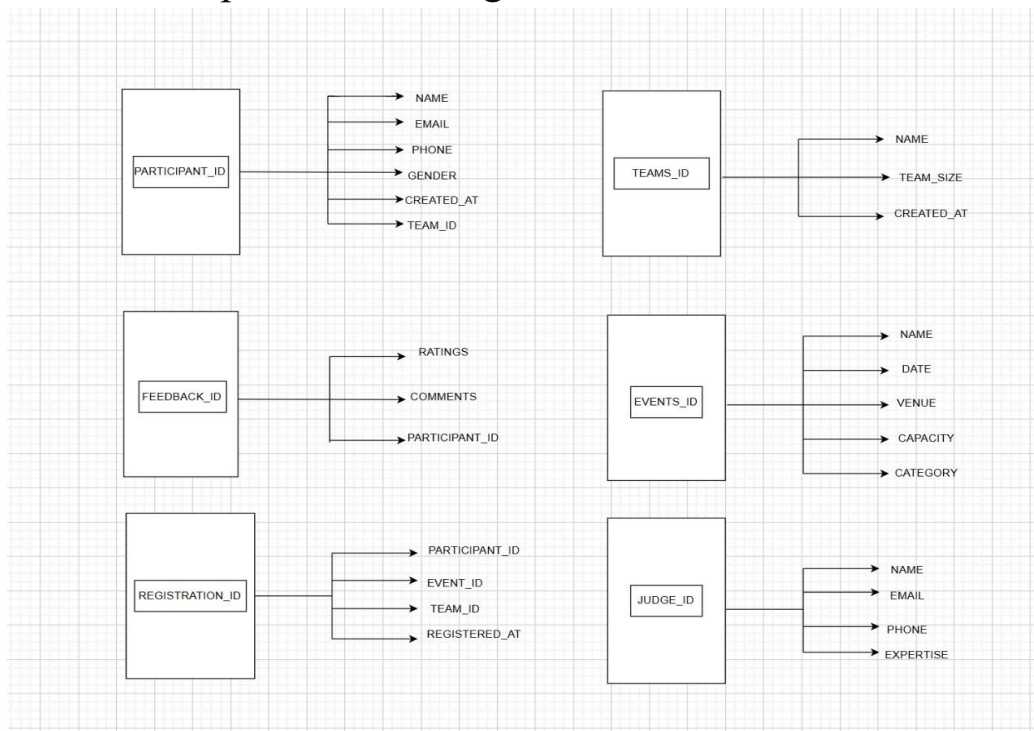
Table	Attributes	Primary Key
Participants	participant_id, name, email, phone, gender, created_at, team_id(FK)	participant_id
Feedback	feedback_id, ratings, comments, participant_id(FK)	feedback_id
Teams	team_id, name, team_size, created_at	team_id
Events	event_id, name, date, venue, capacity, category	event_id
Registrations	registration_id, registered_at, participant_id(FK), team_id(FK), event_id(FK)	registration_id
Judges	judge_id, name, email, phone, expertise	judge_id
Event_Judges	event_id(FK), judge_id(FK)	(event_id, judge_id)
Sponsors	sponsor_id, name, contact_info, sponsorship_type, amount	sponsor_id
Event_Sponsors	event_id(FK), sponsor_id(FK)	(event_id, sponsor_id)
Certificates	certificate_id, issue_date, description, event_id(FK), participant_id(FK)	certificate_id
Results	result_id, position, score, event_id(FK), team_id(FK)	result_id

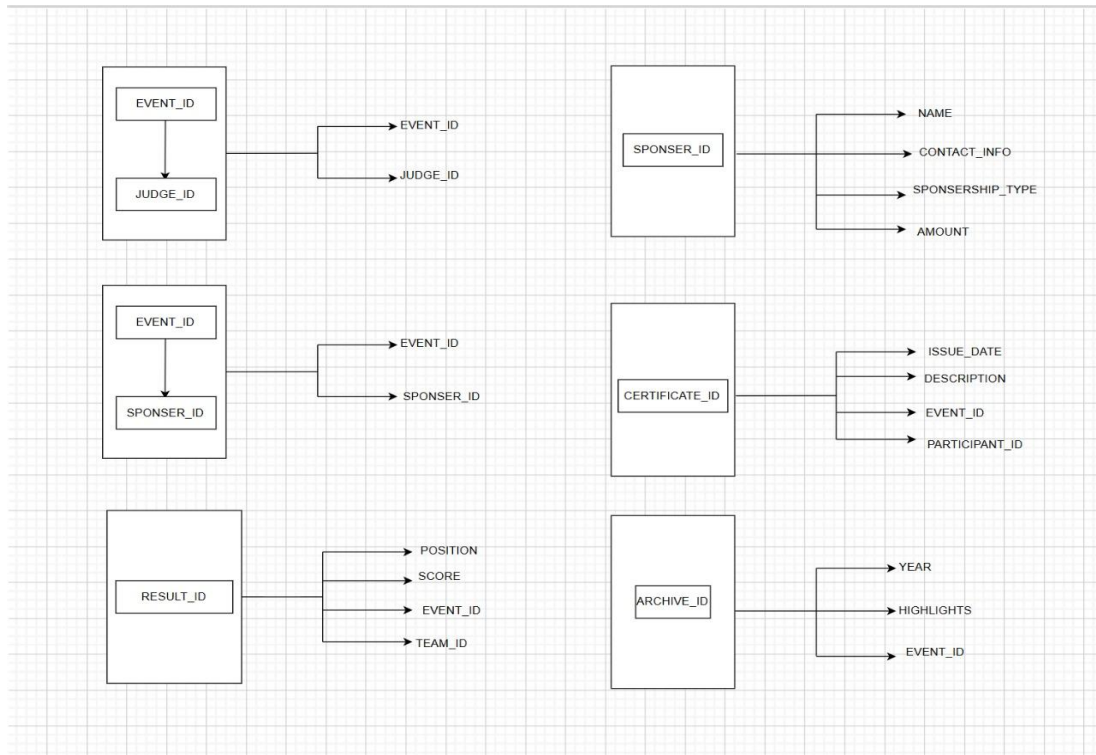
Archive	archive_id, year, highlights, event_id(FK)	archive_id
---------	---	------------

7.4 Functional Dependencies :

- Participants: participant_id → name, email, phone, gender, created_at, team_id
- Feedback: feedback_id → ratings, comments, participant_id
- Teams: team_id → name, team_size, created_at
- Events: event_id → name, date, venue, capacity, category
- Registrations: registration_id → participant_id, event_id, team_id, registered_at
- Judges: judge_id → name, email, phone, expertise
- Event_Judges: (event_id, judge_id) → {event_id, judge_id}
- Sponsors: sponsor_id → name, contact_info, sponsorship_type, amount
- Event_Sponsors: (event_id, sponsor_id) → {event_id, sponsor_id}
- Certificates: certificate_id → issue_date, description, event_id, participant_id
- Results: result_id → position, score, event_id, team_id
- Archive: archive_id → year, highlights, event_id

7.5 Functional Dependencies Diagram :





Anomaly Analysis (1NF)

Table	Redundancy	Insertion	Deletion	Update
Participants	Team_id repeated	Cannot add without team	Deleting team deletes participants	Team ID update affects many
Feedback	Participant repeated	Cannot add until participant exists	Deleting participant deletes feedback	Rating change affects many
Events	Venue/date repeated	Cannot add event without info	Deleting judge remove s event link	Venue update affects all rows

Registrations	Repetition Of participant ent pairs	Cannot add without event	Deleting Participant removes all registrations	Event ID update affects many
---------------	---	--------------------------------	---	---------------------------------

Results	Score repeated	Cannot store result without team	Deleting team remove s results	Score updates require multiple edits
---------	----------------	---	---	---

Second Normal Form (2NF)

Removed partial dependencies. Ensured non-key attributes depend on the full key.

Logic for 2NF

All non-key attributes depend entirely on their primary keys. Composite key tables (Event_Judges, Event_Sponsors) have no partial dependencies.

Relation	Attributes	Primary Key
Participants	participant_id, name, email, phone, gender, created_at, team_id(FK)	participant_id
Feedback	feedback_id, ratings, comments, participant_id(FK)	feedback_id
Teams	team_id, name, team_size, created_at	team_id
Events	event_id, name, date, venue, capacity, category	event_id
Registrations	registration_id, registered_at, participant_id(FK), team_id(FK), event_id(FK)	registration_id
Judges	judge_id, name, email, phone, expertise	judge_id

Event_Judges	event_id(FK), judge_id(FK)	(event_id, judge_id)
Sponsors	sponsor_id, name, contact_info, sponsorship_type, amount	sponsor_id
Event_Sponsors	event_id(FK), sponsor_id(FK)	(event_id, sponsor_id)
Certificates	certificate_id, issue_date, description, event_id(FK), participant_id(FK)	certificate_id
Results	result_id, position, score, event_id(FK), team_id(FK)	result_id
Archive	archive_id, year, highlights, event_id(FK)	archive_id

- All tables are in First Normal Form (1NF) — no repeating groups and atomic attributes.
- Partial dependencies removed - every non-key attribute depends on the entire primary key.
- Composite key tables (Event_Judges, Event_Sponsors) verified - no partial dependencies remain.
- Separated entities like Judges and Sponsors to remove dependency on part of composite keys.
- Each table now represents a single logical concept (Participants, Teams, Events, etc.).
- Data redundancy reduced - same information is not repeated across multiple rows.
- Data integrity improved - updates and deletions now affect only one table.
- Foreign keys maintain proper relationships between tables.
- Database now fully satisfies Second Normal Form (2NF).
- The design is ready for further normalization to Third Normal Form (3NF).

Anomaly Analysis (After 2NF)

Table	Redundancy	Insertion Anomaly	Deletion Anomaly	Update Anomaly
Participants	Minor redundancy if multiple participants belong to same team (team_id repeated)	Cannot add participant without assigning to an existing team (team_id FK constraint)	Deleting a team might affect associated participants (if cascading)	Changing team_id requires multiple updates for all members
Feedback	Minimal redundancy; each participant can have multiple feedbacks	Cannot add feedback unless participant exists	Deleting participant removes related feedback	Change in participant_id affects feedback linkage
Teams	No redundancy within table	Cannot add team without name/team_size	Deleting team may remove related members or results	Team name change affects other related tables
Events	Possible redundancy if multiple events at same venue/date	Cannot add event without venue/date info	Deleting event removes all linked data (judges, sponsors, results, etc.)	Venue or date update affects multiple related rows

Registrations	Same participant-event pairs may repeat if multiple registrations allowed	Cannot register without existing participant and event	Deleting participant/event removes all related registrations	Updating event_id or participant_id needs multiple changes
Judges	No redundancy	Must have complete judge info to add record	Deleting judge removes all linked event assignments	Judge info update must reflect across linked events
Event_Judges	No redundancy — purely relational	Cannot assign judge without both event & judge existing	Deleting judge or event removes assignment	If judge reassigned, event_judge record must be updated
Sponsors	No redundancy	Cannot add sponsor without details	Deleting sponsor removes all event sponsorships	Sponsor info change affects multiple event_sponsor entries
Event_Sponsors	No redundancy — purely relational	Cannot link sponsor to event if either doesn't exist	Deleting event or sponsor removes their link	Reassigning sponsorship requires updating this table
Certificates	Possible redundancy if participant receives multiple certificates for same event	Cannot issue certificate without valid participant & event	Deleting participant/event removes certificate	Change in event_id/participant_id affects multiple rows

Results	Team_id and event_id repeated for multiple entries	Cannot record result unless both event and team exist	Deleting team/event deletes result	Updating event/team details requires multiple updates
Archive	Redundancy possible if same event is listed under multiple archives	Cannot add archive year without at least one event	Deleting event affects archive record	Update of event_id or year may need multiple edits

Final Normalized relations in 2NF

1. Participants

Participants(
participant_id,
name,
email,
phone,
gender,
created_at,
team_id FK → Teams(team_id)
)

2. Feedback

Feedback(
feedback_id,

ratings, comments,
participant_id FK → Participants(participant_id)
)

3. Teams

Teams(
 team_id,
 name,
 team_size,
 created_at
)

4. Events

Events(
 event_id,
 name,
 date,
 venue,
 capacity,
 category
)

5. Registrations

Registrations(
 registration_id,
 registered_at,
 participant_id FK → Participants(participant_id), team_id FK →
 Teams(team_id),
 event_id FK → Events(event_id)
)

6. Judges

```
Judges(  
  judge_id,  
  name,  
  email,  
  phone,  
  expertise  
)
```

7. Event_Judges

```
Event_Judges(  
  event_id FK → Events(event_id),  
  judge_id FK → Judges(judge_id),  
  PK(event_id, judge_id)  
)
```

8. Sponsors

```
Sponsors(  
  sponsor_id, name,  
  contact_info,  
  sponsorship_type,  
  amount  
)
```

9. Event_Sponsors

```
Event_Sponsors(  
  event_id FK → Events(event_id), sponsor_id FK  
  → Sponsors(sponsor_id),  
  
  PK(event_id, sponsor_id)  
)
```

10. Certificates

Certificates(
 certificate_id,
 issue_date,
 description,
 event_id FK → Events(event_id),
 participant_id FK → Participants(participant_id)
)

11. Results

Results(
 result_id,
 position,
 score,
 event_id FK → Events(event_id), team_id FK →
 Teams(team_id)
)

12. Archive

Archive(
 archive_id,
 year,
 highlights,
 event_id FK → Events(event_id)
)

Normalization to 3NF / BCNF

1. Venues (NEW TABLE)

venue_id(PrimaryKey)

venue_name (Unique)

capacity

- This table removes the transitive dependency venue → capacity from the Events table.

2. Events

event_id

name

date

venue_id (FK → Venues(venue_id))

category

- Capacity is removed because it is now an attribute of Venue.

3. Participants (Unchanged)

participant_id

name

email

phone

gender

created_t

team_id (FK → Teams(team_id))

4. Feedback (Unchanged)

feedback_id

ratings

comments

participant_id (FK → Participants(participant_id))

5. Teams (Unchanged)

team_id
name
team_size
created_at

6. Registrations (Unchanged)

registration_id
registered_at
participant_id (FK → Participants(participant_id))
team_id (FK → Teams(team_id))
event_id (FK → Events(event_id))

7. Judges (Unchanged)

judge_id
name
email
phone
expertise

8. Event_Judges (Unchanged)

event_id (FK → Events(event_id))
judge_id (FK → Judges(judge_id))
Primary Key (event_id, judge_id)

9. Sponsors (Unchanged)

sponsor_id
name
contact_info
sponsorship_tye
amount

10. Event_Sponsors (Unchanged)

event_id (FK → Events(event_id))

sponsor_id (FK → Sponsors(sponsor_id))

Primary Key (event_id, sponsor_id)

11. Certificates (Unchanged)

certificate_id

issue_date

description

event_id (FK → Events(event_id))

participant_id (FK → Participants(participant_id))

12. Results (Unchanged)

result_id

position

score

event_id (FK → Events(event_id))

team_id (FK → Teams(team_id))

13. Archive (Unchanged)

archive_id

year

highlights

event_id (FK → Events(event_id))

7.8 Final Normalized Tables

1. VENUES TABLE

```
CREATE TABLE Venues (  
    venue_id INT PRIMARY KEY AUTO_INCREMENT,  
    venue_name VARCHAR(100) UNIQUE NOT NULL,  
    capacity INT CHECK (capacity > 0)  
);
```

2. EVENTS TABLE

```
CREATE TABLE Events (  
    event_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    date DATE NOT NULL,  
    venue_id INT,  
    category VARCHAR(50),  
    FOREIGN KEY (venue_id) REFERENCES Venues(venue_id)  
    ON UPDATE CASCADE ON DELETE SET NULL  
);
```

3. PARTICIPANTS TABLE

```
CREATE TABLE Participants (  
    participant_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    gender ENUM('Male', 'Female', 'Other'),  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    team_id INT,  
    FOREIGN KEY (team_id) REFERENCES Teams(team_id)  
    ON UPDATE CASCADE ON DELETE SET NULL  
);
```

4. FEEDBACK TABLE

```
CREATE TABLE Feedback (  
    feedback_id INT PRIMARY KEY AUTO_INCREMENT,  
    ratings INT CHECK (ratings BETWEEN 1 AND 5),  
    comments TEXT,  
    participant_id INT,  
    FOREIGN KEY (participant_id) REFERENCES Participants(participant_id)
```

```
); ON UPDATE CASCADE ON DELETE CASCADE
```

5. TEAMS TABLE

```
CREATE TABLE Teams (  
    team_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) UNIQUE NOT NULL,  
    team_size INT CHECK (team_size > 0),  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

6. REGISTRATIONS TABLE

```
CREATE TABLE Registrations (  
    registration_id INT PRIMARY KEY AUTO_INCREMENT,  
    registered_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    participant_id INT,  
    team_id INT,  
    event_id INT,  
    FOREIGN KEY (participant_id) REFERENCES Participants(participant_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (team_id) REFERENCES Teams(team_id)  
    ON UPDATE CASCADE ON DELETE SET NULL,  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)  
    ON UPDATE CASCADE ON DELETE CASCADE  
);
```

7. JUDGES TABLE

```
CREATE TABLE Judges (  
    judge_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    expertise VARCHAR(100)  
);
```

8. EVENT_JUDGES TABLE

```
CREATE TABLE Event_Judges (  
    event_id INT,  
    judge_id INT,  
    PRIMARY KEY (event_id, judge_id),  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (judge_id) REFERENCES Judges(judge_id)  
    ON UPDATE CASCADE ON DELETE CASCADE);
```

9. SPONSORS TABLE

```
CREATE TABLE Sponsors (  
    sponsor_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) UNIQUE NOT NULL,  
    contact_info VARCHAR(150),  
    sponsorship_type VARCHAR(50),  
    amount DECIMAL(10,2) CHECK (amount >= 0)  
);
```

10. EVENT_SPONSORS TABLE

```
CREATE TABLE Event_Sponsors (  
    event_id INT,  
    sponsor_id INT,  
    PRIMARY KEY (event_id, sponsor_id),  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (sponsor_id) REFERENCES Sponsors(sponsor_id)  
    ON UPDATE CASCADE ON DELETE CASCADE  
);
```

11. CERTIFICATES TABLE

```
CREATE TABLE Certificates (  
    certificate_id INT PRIMARY KEY AUTO_INCREMENT,  
    issue_date DATE,  
    description TEXT,  
    event_id INT,  
    participant_id INT,  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
    FOREIGN KEY (participant_id) REFERENCES Participants(participant_id)  
    ON UPDATE CASCADE ON DELETE CASCADE  
);
```

12. RESULTS TABLE

```
CREATE TABLE Results (  
    result_id INT PRIMARY KEY AUTO_INCREMENT,  
    position INT CHECK (position > 0),  
    score DECIMAL(5,2),  
    event_id INT,  
    team_id INT,  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)
```

```
        ON UPDATE CASCADE ON DELETE CASCADE,  
        FOREIGN KEY (team_id) REFERENCES  
        Teams(team_id) ON UPDATE CASCADE ON DELETE SET  
        NULL  
    );
```

13. ARCHIVE TABLE

```
CREATE TABLE Archive (  
    archive_id INT PRIMARY KEY AUTO_INCREMENT,  
    year YEAR NOT NULL,  
    highlights TEXT,  
    event_id INT,  
    FOREIGN KEY (event_id) REFERENCES Events(event_id)  
    ON UPDATE CASCADE ON DELETE CASCADE  
);
```

8. SCHEMA

```
C:\Windows\system32\cmd.e: × + v
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\diyad>psql -U postgres -d I-Fest
Password for user postgres:

psql (17.5)
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

I-Fest=# \dt
          List of relations
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 public | archive        | table | postgres
 public | certificates    | table | postgres
 public | event_judges    | table | postgres
 public | event_sponsors | table | postgres
 public | events          | table | postgres
 public | feedback        | table | postgres
 public | judges          | table | postgres
 public | participants    | table | postgres
 public | registrations   | table | postgres
 public | results         | table | postgres
 public | sponsors        | table | postgres
 public | teams          | table | postgres
 public | venues         | table | postgres
(13 rows)
```


1.) Venues

I-Fest=# \d venues

Column	Type	Table "public.venues"	Collation	Nullable	Default
venue_id	integer			not null	nextval('venues_venue_id_seq'::regclass)
venue_name	character varying(100)			not null	
capacity	integer				

Indexes:
"venues_pkey" PRIMARY KEY, btree (venue_id)
"venues_venue_name_key" UNIQUE CONSTRAINT, btree (venue_name)

Check constraints:
"venues_capacity_check" CHECK (capacity > 0)

Referenced by:
TABLE "events" CONSTRAINT "events_venue_id_fkey" FOREIGN KEY (venue_id) REFERENCES venues(venue_id) ON UPDATE CASCADE ON DELETE SET NULL

pgAdmin 4

Object Explorer: public

Properties: SQL, Statistics, Dependencies, SQL Editor

Query: I-Fest/postgres@PostgreSQL 17

```
1 --  
2 -- TABLE: VENUES  
3 -- Stores information about event venues including name & capacity.  
4 --  
5 CREATE TABLE Venues (  
6     venue_id SERIAL PRIMARY KEY,  
7     venue_name VARCHAR(100) UNIQUE NOT NULL,  
8     capacity INT CHECK (capacity > 0),  
9 );  
10  
11 SELECT COUNT(*) AS venue_count FROM venues;  
12  
13 SELECT * FROM venues;
```

Data Output: Messages: Notifications:

venue_id (pk) serial	venue_name varchar(100)	capacity integer
1	Chancellor Auditorium	2100
2	East Jersey High Auditorium	3400
3	Aspenwood Auditorium	1800
4	Orchestra Hall	170
5	New Jersey State Auditorium	4000
6	North Jersey State Auditorium	1900
7	East River Auditorium	3800
8	Port Conference Auditorium	2744
9	Margaret Mary Auditorium	2400
10	Croft Hall Auditorium	4900
11	Charlotte-Mecklenburg Auditorium	2795
12	East Jersey High Auditorium	3400
13	Norfolk Auditorium	1800
14	Wendell Auditorium	2300
15	Bayview Auditorium	6800
16	West Jersey High Auditorium	3400
17	Norfolk Auditorium	2300
18	Johnson Auditorium	3000
19	North Jersey Auditorium	3700
20	East River Auditorium	1400
21	Katherine Auditorium	1800
22	Port Conference Auditorium	2947
23	Westinghouse Auditorium	2870
24	West Jersey Auditorium	670

Total rows: 348 Query complete 80.80.90.358

ONE | Ln 1, Col 1

2.) Events

I-Fest=# \d events

Column	Type	Table "public.events"	Collation	Nullable	Default
event_id	integer			not null	nextval('events_event_id_seq'::regclass)
name	character varying(100)			not null	
date	date			not null	
venue_id	integer			not null	
category	character varying(50)				

Indexes:

"events_pkey" PRIMARY KEY, btree (event_id)

Foreign-key constraints:

"events_venue_id_fkey" FOREIGN KEY (venue_id) REFERENCES venues(venue_id) ON UPDATE CASCADE ON DELETE SET NULL

Referenced by:

TABLE "archive" CONSTRAINT "archive_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

TABLE "certificates" CONSTRAINT "certificates_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

TABLE "event_judges" CONSTRAINT "event_judges_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

TABLE "event_sponsors" CONSTRAINT "event_sponsors_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

TABLE "registrations" CONSTRAINT "registrations_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

TABLE "results" CONSTRAINT "results_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Server
- Database
- Event Triggers
- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
- public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parser
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operations
 - Procedures
 - Sequences
 - Tables (3)
 - archive
 - certificates
 - event_judges
 - event_sponsors
 - events
 - feedback
 - judges
 - participants
 - columns
 - constraints
 - indexes
 - RLS Policies
 - rules
 - triggers (1)
 - generate_participant_data_after
 - generate_participant_data_before
 - registrations
 - results
 - sponsors
 - teams
 - venues

Properties SQL X Statistics X Dependencies X SQLM, I-Fest.sql X

I-Fest@postgres@PostgreSQL 17

Query Query History

```
-- TABLE: EVENTS
-- Contains all event details linked to a specific venue.
-- =====
-- CREATE TABLE Events (
--   event_id SERIAL PRIMARY KEY,
--   name VARCHAR(100) NOT NULL,
--   date DATE NOT NULL,
--   venue_id INT,
--   category VARCHAR(50),
--   FOREIGN KEY (venue_id) REFERENCES Venues(venue_id)
--     ON UPDATE CASCADE ON DELETE SET NULL
-- );
--
-- SELECT COUNT(*) AS event_count FROM events;
-- SELECT * FROM events;
```

Data Output Messages Notifications

Showing rows 1 to 83 | Page 10: 1 | 1 2 3 4 5 6 7 8 9 10

event_id	name	date	venue_id	category
1	Global Innovation Summit 2024	2024-06-15	101	Technology
2	Future of AI Conference	2024-07-01	102	Technology
3	Quantum Computing Workshop	2024-07-15	103	Technology
4	Blockchain for Business	2024-08-01	104	Technology
5	Cybersecurity Summit	2024-08-15	105	Technology
6	Space Exploration Symposium	2024-09-01	106	Science
7	Artificial Intelligence Ethics	2024-09-15	107	Technology
8	Virtual Reality Development	2024-10-01	108	Technology
9	Quantum Entanglement Research	2024-10-15	109	Science
10	AI in Healthcare	2024-11-01	110	Technology
11	Space Colonization Feasibility	2024-11-15	111	Science
12	Quantum Cryptography	2024-12-01	112	Technology
13	AI in Education	2025-01-01	113	Technology
14	Space Mining Potential	2025-01-15	114	Science
15	Quantum Sensing	2025-02-01	115	Technology
16	AI in Manufacturing	2025-02-15	116	Technology
17	Space Exploration: Mars	2025-03-01	117	Science
18	Quantum Computing: Next Steps	2025-03-15	118	Technology
19	AI in Finance	2025-04-01	119	Technology
20	Space Exploration: Europa	2025-04-15	120	Science
21	Quantum Computing: Quantum Leap	2025-05-01	121	Technology
22	AI in Healthcare: Precision Medicine	2025-05-15	122	Technology
23	Space Exploration: Artemis	2025-06-01	123	Science
24	Quantum Computing: Quantum Leap	2025-06-15	124	Technology
25	AI in Manufacturing: Industry 4.0	2025-07-01	125	Technology
26	Space Exploration: Mars	2025-07-15	126	Science
27	Quantum Computing: Quantum Leap	2025-08-01	127	Technology
28	AI in Healthcare: Precision Medicine	2025-08-15	128	Technology
29	Space Exploration: Artemis	2025-09-01	129	Science
30	Quantum Computing: Quantum Leap	2025-09-15	130	Technology
31	AI in Manufacturing: Industry 4.0	2025-10-01	131	Technology
32	Space Exploration: Mars	2025-10-15	132	Science
33	Quantum Computing: Quantum Leap	2025-11-01	133	Technology
34	AI in Healthcare: Precision Medicine	2025-11-15	134	Technology
35	Space Exploration: Artemis	2025-12-01	135	Science
36	Quantum Computing: Quantum Leap	2025-12-15	136	Technology
37	AI in Manufacturing: Industry 4.0	2026-01-01	137	Technology
38	Space Exploration: Mars	2026-01-15	138	Science
39	Quantum Computing: Quantum Leap	2026-02-01	139	Technology
40	AI in Healthcare: Precision Medicine	2026-02-15	140	Technology
41	Space Exploration: Artemis	2026-03-01	141	Science
42	Quantum Computing: Quantum Leap	2026-03-15	142	Technology
43	AI in Manufacturing: Industry 4.0	2026-04-01	143	Technology
44	Space Exploration: Mars	2026-04-15	144	Science
45	Quantum Computing: Quantum Leap	2026-05-01	145	Technology
46	AI in Healthcare: Precision Medicine	2026-05-15	146	Technology
47	Space Exploration: Artemis	2026-06-01	147	Science
48	Quantum Computing: Quantum Leap	2026-06-15	148	Technology
49	AI in Manufacturing: Industry 4.0	2026-07-01	149	Technology
50	Space Exploration: Mars	2026-07-15	150	Science
51	Quantum Computing: Quantum Leap	2026-08-01	151	Technology
52	AI in Healthcare: Precision Medicine	2026-08-15	152	Technology
53	Space Exploration: Artemis	2026-09-01	153	Science
54	Quantum Computing: Quantum Leap	2026-09-15	154	Technology
55	AI in Manufacturing: Industry 4.0	2026-10-01	155	Technology
56	Space Exploration: Mars	2026-10-15	156	Science
57	Quantum Computing: Quantum Leap	2026-11-01	157	Technology
58	AI in Healthcare: Precision Medicine	2026-11-15	158	Technology
59	Space Exploration: Artemis	2026-12-01	159	Science
60	Quantum Computing: Quantum Leap	2026-12-15	160	Technology
61	AI in Manufacturing: Industry 4.0	2027-01-01	161	Technology
62	Space Exploration: Mars	2027-01-15	162	Science
63	Quantum Computing: Quantum Leap	2027-02-01	163	Technology
64	AI in Healthcare: Precision Medicine	2027-02-15	164	Technology
65	Space Exploration: Artemis	2027-03-01	165	Science
66	Quantum Computing: Quantum Leap	2027-03-15	166	Technology
67	AI in Manufacturing: Industry 4.0	2027-04-01	167	Technology
68	Space Exploration: Mars	2027-04-15	168	Science
69	Quantum Computing: Quantum Leap	2027-05-01	169	Technology
70	AI in Healthcare: Precision Medicine	2027-05-15	170	Technology
71	Space Exploration: Artemis	2027-06-01	171	Science
72	Quantum Computing: Quantum Leap	2027-06-15	172	Technology
73	AI in Manufacturing: Industry 4.0	2027-07-01	173	Technology
74	Space Exploration: Mars	2027-07-15	174	Science
75	Quantum Computing: Quantum Leap	2027-08-01	175	Technology
76	AI in Healthcare: Precision Medicine	2027-08-15	176	Technology
77	Space Exploration: Artemis	2027-09-01	177	Science
78	Quantum Computing: Quantum Leap	2027-09-15	178	Technology
79	AI in Manufacturing: Industry 4.0	2027-10-01	179	Technology
80	Space Exploration: Mars	2027-10-15	180	Science
81	Quantum Computing: Quantum Leap	2027-11-01	181	Technology
82	AI in Healthcare: Precision Medicine	2027-11-15	182	Technology
83	Space Exploration: Artemis	2027-12-01	183	Science

Total rows: 83 Query complete 00:00:00.354

Successfully run. Total query runtime: 364 msec. 80 rows affected.

3.) Teams

I-Fest=# \d teams

Column	Type	Table "public.teams"	Collation	Nullable	Default
team_id	integer			not null	nextval('teams_team_id_seq'::regclass)
name	character varying(100)			not null	
team_size	integer			not null	
created_at	timestamp without time zone				CURRENT_TIMESTAMP

Indexes:
"teams_pkey" PRIMARY KEY, btree (team_id)
"teams_name_key" UNIQUE CONSTRAINT, btree (name)

Check constraints:
"teams_team_size_check" CHECK (team_size > 0)

Referenced by:
TABLE "participants" CONSTRAINT "participants_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL
TABLE "registrations" CONSTRAINT "registrations_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL
TABLE "results" CONSTRAINT "results_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL

pgAdmin 4

Object Explorer | Properties | SQL | Statistics | Dependencies | SQLM, I-Fest.sql | X

File | Object | Tools | Edit | View | Window | Help

Object Explorer: public > Tables (1) > teams

SQL: I-Fest=# \d teams

```
--  
-- TABLE: teams  
-- Builds team information for group-based events.  
--  
CREATE TABLE teams (  
    team_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) UNIQUE NOT NULL,  
    team_size INT CHECK (team_size > 0),  
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP  
);  
  
SELECT COUNT(*) AS teams_count FROM teams;  
SELECT * FROM teams;
```

Data Output: Messages: Notifications

Showing rows: 1 to 350 | Page No: 1 | 1 | 24 | 44 | 59 | 74

team_id	name	team_size	created_at
1	Nam's LLC	7	2025-10-28 16:40:30.024111
2	Harde-Winter	7	2025-10-28 16:40:30.024111
3	Mossing, Christian and Walters	8	2025-10-28 16:40:30.024111
4	Harcock, Rives and Bates	3	2025-10-28 16:40:30.024111
5	Carroll PLC	6	2025-10-28 16:40:30.024111
6	Jensen-Mitchell	6	2025-10-28 16:40:30.024111
7	Flagstad Yang	6	2025-10-28 16:40:30.024111
8	Ford Wilson	7	2025-10-28 16:40:30.024111
9	Harris PLC	6	2025-10-28 16:40:30.024111
10	Boyle LLC	2	2025-10-28 16:40:30.024111
11	Oak-Rubensky	3	2025-10-28 16:40:30.024111
12	W Flannery PLC	7	2025-10-28 16:40:30.024111
13	Correa, Bond and Hutcherson	7	2025-10-28 16:40:30.024111
14	Anderson-Moore	3	2025-10-28 16:40:30.024111
15	Gibson, Parker and Reed	4	2025-10-28 16:40:30.024111
16	Young Hall	6	2025-10-28 16:40:30.024111
17	Brother and Sons	2	2025-10-28 16:40:30.024111
18	Smith PLC	7	2025-10-28 16:40:30.024111
19	Smith-Lane	7	2025-10-28 16:40:30.024111
20	Newman and Sons	6	2025-10-28 16:40:30.024111
21	Lucas-McKinney	8	2025-10-28 16:40:30.024111
22	Fullerton	3	2025-10-28 16:40:30.024111

Total rows: 350 | Query complete 00:00:00.369

Successfully ran. Total query runtime: 386 msec. 350 rows affected.

4.) Participants

I-Fest=# \d participants

Column	Type	Table "public.participants"	Collation	Nullable	Default
participant_id	integer			not null	
name	character varying(100)			not null	
email	character varying(100)			not null	
phone	character varying(15)				
gender	character varying(10)				
created_at	timestamp without time zone				CURRENT_TIMESTAMP
team_id	integer				
qr_data	text				
qr_code_path	character varying(255)				

Indexes:

- "participants_pkey" PRIMARY KEY, btree (participant_id)
- "participants_email_key" UNIQUE CONSTRAINT, btree (email)
- "participants_qr_data_key" UNIQUE CONSTRAINT, btree (qr_data)

Check constraints:

- "participants_gender_check" CHECK (gender::text = ANY (ARRAY['Male'::character varying, 'Female'::character varying, 'Other'::character varying]))::text[])

Foreign-key constraints:

- "participants_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL

Referenced by:

- TABLE "certificates" CONSTRAINT "certificates_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE
- TABLE "feedback" CONSTRAINT "feedback_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE
- TABLE "registrations" CONSTRAINT "registrations_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE

Triggers:

- trg_generate_qr_data_after AFTER INSERT ON participants FOR EACH ROW EXECUTE FUNCTION generate_participant_qr_data_after()

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Database
- Schema
- Table
- View
- Function
- Procedure
- Sequence
- Table (1)
- Trigger
- Trigger Functions

Properties X SQL X Statistics X Dependencies X I-Fest=# PostgreSQL 17 X

Query

```
-- =====  
-- TABLE: PARTICIPANTS  
-- Schema details of individual participants linked to teams.  
-- Includes QR code data for verification.  
-- =====  
CREATE TABLE Participants (  
    participant_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    gender VARCHAR(10) CHECK (gender IN ('Male', 'Female', 'Other')),  
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,  
    team_id INT,  
    qr_data TEXT UNIQUE,  
    qr_code_path VARCHAR(255),  
    FOREIGN KEY (team_id) REFERENCES Teams (team_id)  
    ON UPDATE CASCADE ON DELETE SET NULL  
);  
  
SELECT COUNT(*) AS participant_count FROM participants;  
SELECT * FROM participants;
```

Data Output

participant_id	name	email	phone	gender	created_at	team_id	qr_data	qr_code_path
1	Mary Smith	marysmith@example.org	818401761	Male	2025-10-28 16:44:30.849256	236	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
2	Michael Rossi	michaelr@example.net	8751448403	Female	2025-10-28 16:44:30.849256	180	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
3	Laura Johnson	laurajohn@example.org	899389989	Male	2025-10-28 16:44:30.849256	81	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
4	Dana Warner	danawar@example.net	437446137	Female	2025-10-28 16:44:30.849256	284	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
5	Julia Miller	juliamil@example.net	895788473	Male	2025-10-28 16:44:30.849256	180	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
6	Leanne Weber	leannewe@example.net	188704403	Other	2025-10-28 16:44:30.849256	144	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
7	Sam Hunt	samhunt@example.org	576844157	Other	2025-10-28 16:44:30.849256	146	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
8	Alice Reynolds	alicere@example.com	175067293	Male	2025-10-28 16:44:30.849256	166	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
9	Debra McKinley	debram@example.org	858891172	Other	2025-10-28 16:44:30.849256	162	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
10	Maya Evans	mayae@example.com	702272851	Other	2025-10-28 16:44:30.849256	183	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
11	Steven Price	stevenp@example.org	403794409	Male	2025-10-28 16:44:30.849256	180	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
12	Emily Chen	emilyc@example.org	329274279	Male	2025-10-28 16:44:30.849256	129	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
13	Quinn Tanaka	quinnt@example.com	838007801	Other	2025-10-28 16:44:30.849256	218	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
14	Mark Weber	markw@example.net	718054446	Other	2025-10-28 16:44:30.849256	277	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
15	Alexander Black	alexblack@example.com	357408597	Other	2025-10-28 16:44:30.849256	223	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256
16	Isabella Adams	isabellad@example.com	1206707583	Male	2025-10-28 16:44:30.849256	719	98871-1761850799-849256	C:\Users\user\AppData\Local\Microsoft\Windows\Temporary Internet Files\Content.IE5\...\98871-1761850799-849256

Total rows: 1280 Query complete 00:00:06.911

Page 80: 1 of 2

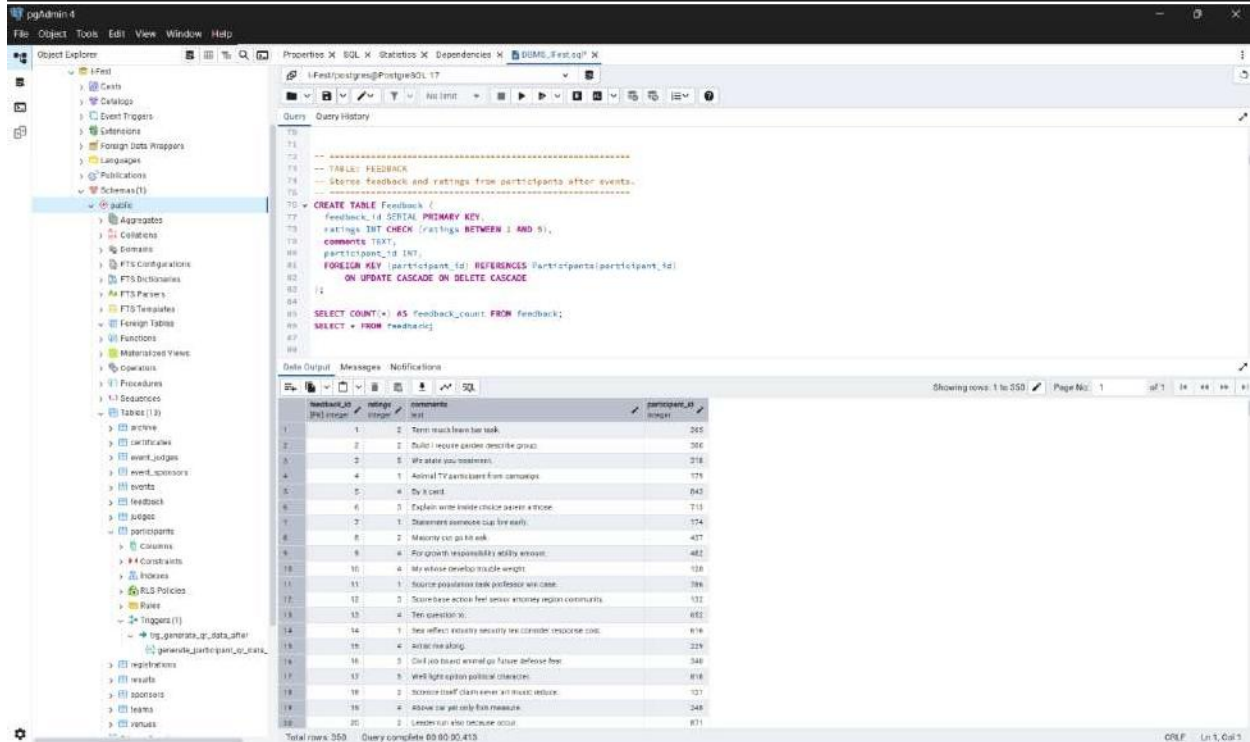
SQL

5.) Feedback

I-Fest=# \d feedback

Column	Type	Collation	Nullable	Default
feedback_id	integer		not null	nextval('feedback_feedback_id_seq'::regclass)
ratings	integer			
comments	text			
participant_id	integer			

Indexes:
"feedback_pkey" PRIMARY KEY, btree (feedback_id)
Check constraints:
"feedback_ratings_check" CHECK (ratings >= 1 AND ratings <= 5)
Foreign-key constraints:
"feedback_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE



The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane displays the database structure, with the 'public' schema selected. The 'Tables (1)' folder is expanded, showing the 'feedback' table. The main pane displays the 'Properties' tab for the 'public.feedback' table, showing its columns, indexes, and constraints. Below the table structure, a SQL query is executed, and the results are displayed in a table with columns 'feedback_id', 'ratings', 'comments', and 'participant_id'. The query results show 20 rows of feedback data.

feedback_id	ratings	comments	participant_id
1	1	Terri much love her look	265
2	2	Dull! I require garden describe group	366
3	2	We state you business	278
4	4	Animal Tractionant from camoings	179
5	5	By A cast	642
6	6	Explain write inside choice paper a those	713
7	2	Statement someone cup for early	174
8	8	Majority can go to ask	407
9	9	For growth responsibility ability enough	482
10	10	My whose develop trouble weight	138
11	11	Source population task professor we case	386
12	12	Scorebase action feel senior money region comments	132
13	13	Ten question to	652
14	14	Just reflect industry security we consider response case	616
15	15	Actual me along	229
16	16	Did you based animal go future defense free	340
17	17	What light option political character	818
18	18	Scorebase itself claim never art music reduce	131
19	19	Address can get only fish measure	348
20	2	Leader can also because occur	871

Total rows: 200 Query complete 60.60.90.416

6.) Registrations

l-Fest=#\d registrations
Table "public.registrations"

Column	Type	Collation	Nullable	Default
registration_id	integer		not null	nextval('registrations_registration_id_seq'::regclass)
registered_at	timestamp without time zone			CURRENT_TIMESTAMP
participant_id	integer			
team_id	integer			
event_id	integer			

Indexes:

- "registrations_pkey" PRIMARY KEY, btree (registration_id)

Foreign-key constraints:

- "registrations_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE
- "registrations_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE
- "registrations_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- l-Fest=#\d registrations
- Database
- Schema
- Table
- View
- Materialized View
- Sequence
- Function
- Procedure
- Trigger
- Index
- Policy
- Rule
- Constraint
- Collation
- Language
- Extension
- Foreign Data Wrapper
- Publication
- Subscription
- Event Trigger
- Configuration
- Replication
- Cluster
- Role
- Group
- Privilege
- Grant
- Revoke
- Drop
- Alter
- Refresh
- Truncate
- Vacuum
- Check
- Repair
- Backup
- Restore
- Export
- Import
- Schema
- Table
- View
- Materialized View
- Sequence
- Function
- Procedure
- Trigger
- Index
- Policy
- Rule
- Constraint
- Collation
- Language
- Extension
- Foreign Data Wrapper
- Publication
- Subscription
- Event Trigger
- Configuration
- Replication
- Cluster
- Role
- Group
- Privilege
- Grant
- Revoke
- Drop
- Alter
- Refresh
- Truncate
- Vacuum
- Check
- Repair
- Backup
- Restore
- Export
- Import

l-Fest=#\d registrations

CREATE TABLE registrations (
 registration_id SERIAL PRIMARY KEY,
 registered_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
 participant_id INT,
 team_id INT,
 event_id INT,
 FOREIGN KEY (participant_id) REFERENCES participants(participant_id)
 ON UPDATE CASCADE ON DELETE CASCADE,
 FOREIGN KEY (event_id) REFERENCES events(event_id)
 ON UPDATE CASCADE ON DELETE SET NULL,
 FOREIGN KEY (team_id) REFERENCES teams(team_id)
 ON UPDATE CASCADE ON DELETE CASCADE
);

SELECT COUNT(*) AS registrations_count FROM registrations;

SELECT * FROM registrations;

Data Output

registration_id	registered_at	participant_id	team_id	event_id
1	2023-10-26 16:40:32.876264	893	8	6
2	2023-10-26 16:40:32.876264	262	516	81
3	2023-10-26 16:40:32.876264	337	335	78
4	2023-10-26 16:40:32.876264	32	129	11
5	2023-10-26 16:40:32.876264	366	345	69
6	2023-10-26 16:40:32.876264	839	238	33
7	2023-10-26 16:40:32.876264	500	306	13
8	2023-10-26 16:40:32.876264	1766	82	15
9	2023-10-26 16:40:32.876264	268	205	36
10	2023-10-26 16:40:32.876264	536	135	81
11	2023-10-26 16:40:32.876264	873	162	13
12	2023-10-26 16:40:32.876264	1583	159	72
13	2023-10-26 16:40:32.876264	487	49	32
14	2023-10-26 16:40:32.876264	1182	75	4
15	2023-10-26 16:40:32.876264	88	292	46
16	2023-10-26 16:40:32.876264	47	50	47
17	2023-10-26 16:40:32.876264	1064	516	17

Total rows: 350 Query complete 00:00:00.346

7.) Judges

I-Fest=# judges

Column	Type	Table "public.judges"	Collation	Nullable	Default
judge_id	integer			not null	nextval('judges_judge_id_seq'::regclass)
name	character varying(100)			not null	
email	character varying(100)			not null	
phone	character varying(15)			not null	
expertise	character varying(100)				

Indexes:
"judges_pkey" PRIMARY KEY, btree (judge_id)
"judges_email_key" UNIQUE CONSTRAINT, btree (email)

Referenced by:
TABLE "event_judges" CONSTRAINT "event_judges_judge_id_fkey" FOREIGN KEY (judge_id) REFERENCES judges(judge_id) ON UPDATE CASCADE ON DELETE CASCADE

pgAdmin 4

Object Explorer: public > Tables > judges

Query Editor: i-fest/pgsql@PostgreSQL 17

```
--  
-- TABLE: JUDGES  
-- Contains Judge Information and their area of expertise.  
--  
CREATE TABLE judges (  
  judge_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  phone VARCHAR(15),  
  expertise VARCHAR(100)  
);  
  
-- SELECT COUNT(*) AS judge_count FROM judges;  
-- SELECT * FROM judges;
```

Data Output: Showing rows 1 to 250 | Page No: 1

judge_id	name	email	phone	expertise
1	Laura Walker	lwalker@example.com	87653210	Marketing
2	Jennifer Smith	jsmith@example.com	921772266	Design
3	Brian Paul	brianp@example.com	789171804	Business
4	Christopher Heath	chheath@example.com	486320608	Design
5	Kylea Hall	khall@example.com	679455260	Marketing
6	Jordan Garcia	jgarcia@example.com	774313541	Cybersecurity
7	Thomas Smith	tsmith@example.com	934814674	Finance
8	Randall Davis	rdavis@example.com	451834107	Software Engineering
9	Sara Page	sapage@example.com	378489587	Design
10	Wayne White	wwhite@example.com	220177360	Electronics
11	Kenneth Miller	kmiller@example.com	7845752915	Finance
12	Brenda Cunningham	bcunning@example.com	266939265	Cybersecurity
13	Natalie Nash	nnash@example.com	621575557	Business
14	Emily Armstrong	eamstrong@example.com	3477146173	AI
15	Charles Ward	cward@example.com	561022103	Design
16	Austin Brown	abrown@example.com	849435654	AI
17	Matthew Martinez	mmartinez@example.com	671396997	Business
18	Joshua Wilson	jwilson@example.com	781434997	AI
19	Matthew Wilson	mwilson@example.com	079930906	Design
20	Shayne MacDonald	smacdon@example.com	770929976	Marketing
21	William Walsh	wwalsh@example.com	7794552615	Design
22	Triffany Moore	tmoore@example.com	679991307	AI

Total rows: 350 | Query complete 80.80.377

Successfully run. Total query runtime: 277 msec. 350 rows affected.

8.) Event_Judges

```
I-Fest=# \d event_judges
        Table "public.event_judges"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 event_id | integer |           | not null |
 judge_id | integer |           | not null |
Indexes:
    "event_judges_pkey" PRIMARY KEY, btree (event_id, judge_id)
Foreign-key constraints:
    "event_judges_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE
    "event_judges_judge_id_fkey" FOREIGN KEY (judge_id) REFERENCES judges(judge_id) ON UPDATE CASCADE ON DELETE CASCADE
```

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure, with 'public' selected. The 'Properties' pane on the right shows the 'SQL' tab with the following SQL script:

```
-- TABLE: EVENT_JUDGES
-- Junction table linking events and their respective judges.
CREATE TABLE event_judges (
    event_id INT,
    judge_id INT,
    PRIMARY KEY (event_id, judge_id),
    FOREIGN KEY (event_id) REFERENCES events(event_id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (judge_id) REFERENCES judges(judge_id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

SELECT COUNT(*) AS event_judge_count FROM event_judges;
SELECT * FROM event_judges;
```

The 'Query Output' pane at the bottom displays the results of the second query, showing 249 rows. The columns are 'event_id' (pk integer) and 'judge_id' (pk integer). The data is as follows:

event_id	judge_id
1	75
2	105
3	57
4	75
5	54
6	38
7	54
8	40
9	2
10	4
11	77
12	77
13	80
14	30
15	61
16	41
17	76
18	35
19	22
20	38
21	6
22	76

The status bar at the bottom indicates 'Total rows: 249' and 'Query complete 00:00:30.368'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 368 msec. 249 rows affected.'

9.) Sponsors

I-Fest=# \d sponsors

Column	Type	Table "public.sponsors"	Collation	Nullable	Default
sponsor_id	integer			not null	nextval('sponsors_sponsor_id_seq'::regclass)
name	character varying(100)			not null	
contact_info	character varying(150)				
sponsorship_type	character varying(50)				
amount	numeric(10,2)				

Indexes:
"sponsors_pkey" PRIMARY KEY, btree (sponsor_id)
"sponsors_name_key" UNIQUE CONSTRAINT, btree (name)
Check constraints:
"sponsors_amount_check" CHECK (amount >= 0::numeric)
Referenced by:
TABLE "event_sponsors" CONSTRAINT "event_sponsors_sponsor_id_fkey" FOREIGN KEY (sponsor_id) REFERENCES sponsors(sponsor_id) ON UPDATE CASCADE ON DELETE CASCADE

pgAdmin 4

Object Explorer | Properties | SQL | Statistics | Dependencies | SQL Editor

Query: I-Fest=# \d sponsors

```
--  
-- TABLE: SPONSORS  
-- Stores sponsor details and sponsorship constraint table.  
--  
CREATE TABLE SPONSORS (  
    sponsor_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) UNIQUE NOT NULL,  
    contact_info VARCHAR(150),  
    sponsorship_type VARCHAR(50),  
    amount DECIMAL(10,2) CHECK (amount >= 0)  
);  
  
SELECT COUNT(*) AS sponsor_count FROM sponsors;  
SELECT * FROM sponsors;
```

Data Output: Messages: Notifications:

id	name	contact_info	sponsorship_type	amount
1	Salvag Ltd	001410-679-999702308	Gold	140000.00
2	Davis, Levy and Little	415-764-905-198297029	Platinum	42000.00
3	Hardy Miller and Edwards	342-719-623479180	Platinum	15500.00
4	Cohen-Rodriguez	415-562-52294802	Gold	31700.00
5	Talbot-Johnson	001798-672-4805	Gold	12000.00
6	Shaw and Sons	(720)434-3403	Platinum	10100.00
7	Agnew-Anderson	586-536-3857	Silver	34000.00
8	White LLC	278-542-15550367	Gold	49123.20
9	Hester-Kelly	481-475-54705199	Gold	87000.00
10	Barnes-McCormick	721-543-4866	Silver	14757.00
11	Henderson-Bergman	(270)429-4828	Silver	8300.00
12	Hopson, Rodriguez and Chan	(840)445-130747204	Platinum	71000.00
13	Kelly-Campbell	001488-483-4188944827	Gold	11300.00
14	Cyber Group	(800)989-41124084	Silver	24000.00
15	Blackwell Group	409-640-198910154	Silver	76700.00
16	Barnes, Lessor and Bergman	624-576-74706165	Gold	43000.00
17	Colbert and Sons	41022-038-6080	Silver	21700.00
18	Ortiz-Ortiz	640-761-40767467	Silver	47200.00
19	Stevenson, Olson and Cox	442-807-4836	Gold	27500.00
20	Barnes-Campbell	124-267-382704811	Gold	81000.00
21	Camacho Inc	001485-211-02490249	Platinum	126100.00

Total rows: 250 Query complete 00:00:00.364

ORF Ln 1, Col 1

10.) Event_Sponsors

```
I-Fest=# \d event_sponsors
Table "public.event_sponsors"
  Column      | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
event_id     | integer |           | not null |
sponsor_id   | integer |           | not null |
Indexes:
    "event_sponsors_pkey" PRIMARY KEY, btree (event_id, sponsor_id)
Foreign-key constraints:
    "event_sponsors_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE
    "event_sponsors_sponsor_id_fkey" FOREIGN KEY (sponsor_id) REFERENCES sponsors(sponsor_id) ON UPDATE CASCADE ON DELETE CASCADE
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'public' schema and various objects like tables, views, and functions. The main pane shows the SQL query editor with the following SQL script:

```
-- TABLE: EVENT_SPONSORS
-- Junction table linking events and their sponsors.
CREATE TABLE Event_Sponsors (
    event_id INT,
    sponsor_id INT,
    PRIMARY KEY (event_id, sponsor_id),
    FOREIGN KEY (event_id) REFERENCES events(event_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (sponsor_id) REFERENCES Sponsors(sponsor_id)
    ON UPDATE CASCADE ON DELETE CASCADE
);

SELECT COUNT(*) AS event_sponsor_count FROM event_sponsors;
SELECT * FROM event_sponsors;
```

The bottom pane displays the query results in a table format:

event_id (pk) integer	sponsor_id (pk) integer
1	80
2	80
3	28
4	49
5	55
6	80
7	37
8	80
9	19
10	88
11	28
12	34
13	80
14	48
15	80
16	21
17	52
18	48
19	47
20	18
21	28
22	47

The status bar at the bottom indicates: "Total rows: 245 Query complete 00:00:00.302". A green message box at the bottom right states: "Successfully run. Total query runtime: 392 msec. 245 rows effected."

11.) Certificates

```
I-Fest=# \d certificates
```

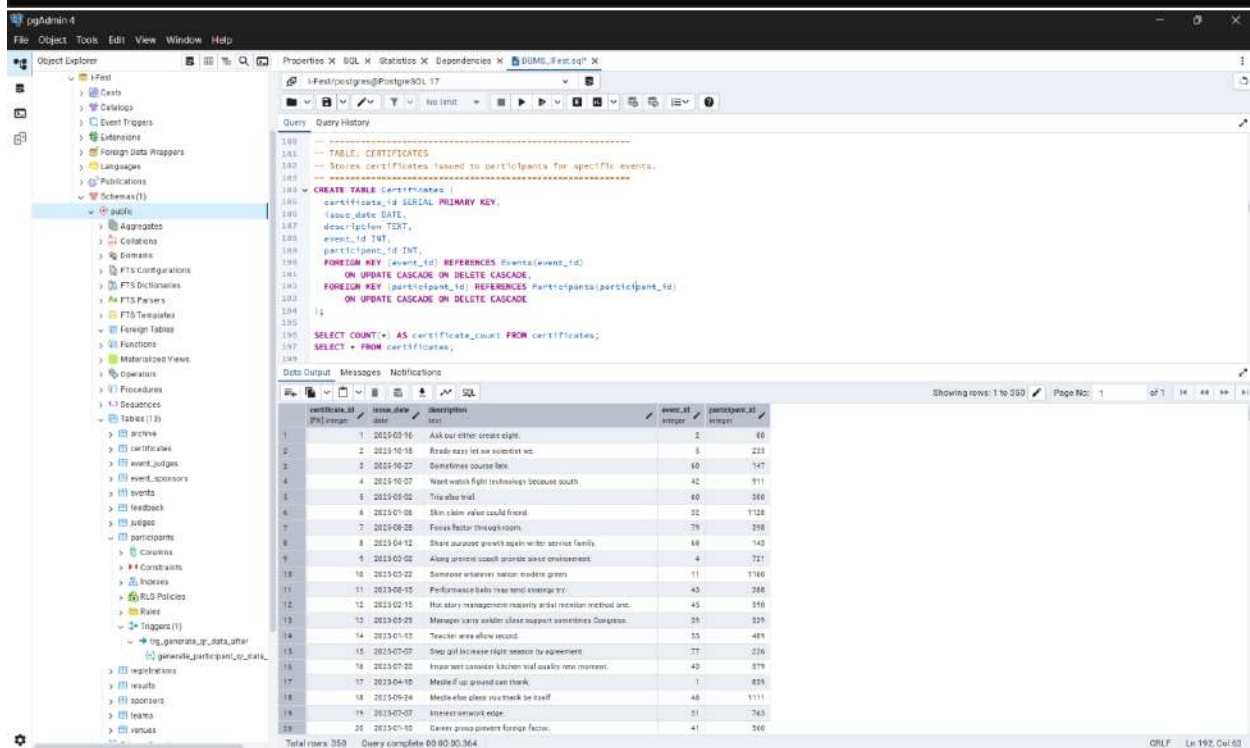
Column	Type	Collation	Nullable	Default
certificate_id	integer		not null	nextval('certificates_certificate_id_seq'::regclass)
issue_date	date			
description	text			
event_id	integer			
participant_id	integer			

Indexes:

- "certificates_pkey" PRIMARY KEY, btree (certificate_id)

Foreign-key constraints:

- "certificates_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE
- "certificates_participant_id_fkey" FOREIGN KEY (participant_id) REFERENCES participants(participant_id) ON UPDATE CASCADE ON DELETE CASCADE



The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure, with 'public' selected. The 'Table 'public.certificates'' is highlighted. The main pane displays the table's structure, including columns, indexes, and foreign key constraints. Below the structure, a query is executed, showing the results of a SELECT statement. The query output is displayed in a table with columns: certificate_id, issue_date, description, event_id, and participant_id. The results show 20 rows of data, including details about various events and participants.

certificate_id	issue_date	description	event_id	participant_id
1	2019-03-16	Ask our other create night.	2	88
2	2019-10-18	Ready easy let us extend it.	5	233
3	2019-10-27	Sometimes course late.	60	147
4	2019-10-27	Want watch fight technology because south.	42	911
5	2019-01-02	Traffic trial.	60	268
6	2019-01-06	Skin claim value could found.	32	1128
7	2019-08-28	Focus faster through room.	79	298
8	2019-04-12	Share journey growth again we for service family.	58	143
9	2019-02-02	Along current coach around since environment.	4	721
10	2019-02-22	Someone whenever nation modern green.	11	1166
11	2019-08-15	Performance beta material strategy try.	43	386
12	2019-02-15	But story management recently actual mission method and.	45	958
13	2019-05-29	Manager carry soldier client support sometimes Douglas.	39	325
14	2019-01-13	Tractor was allow record.	33	489
15	2019-07-07	Shop girl increase right season by agreement.	77	224
16	2019-07-20	Improve current fashion trial quality new movement.	40	579
17	2019-04-10	Medical up ground see thank.	1	829
18	2019-09-24	Media who glass you made be itself.	48	1111
19	2019-07-07	Amused network edge.	31	743
20	2019-01-10	Career group prevent foreign factor.	41	566

12.) Results

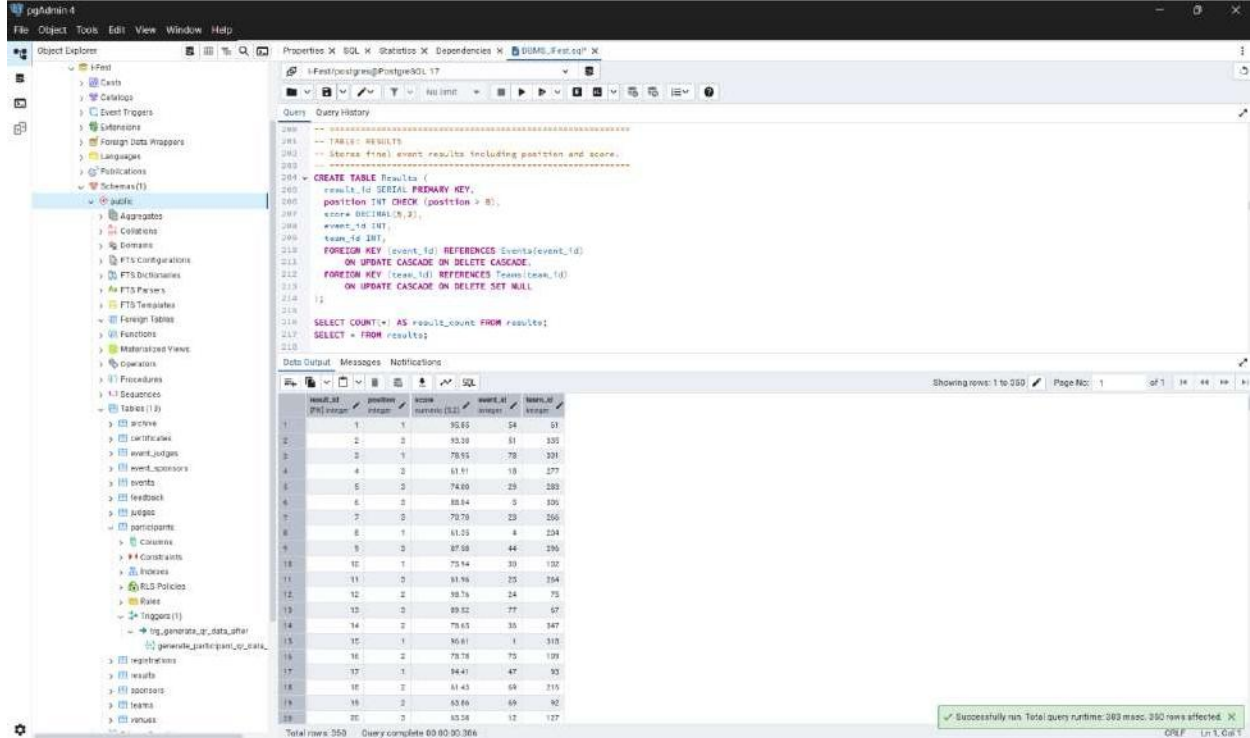
I-Fest=# \d results

Column	Type	Collation	Nullable	Default
result_id	integer		not null	nextval('results_result_id_seq'::regclass)
position	integer			
score	numeric(5,2)			
event_id	integer			
team_id	integer			

Indexes:
"results_pkey" PRIMARY KEY, btree (result_id)

Check constraints:
"results_position_check" CHECK ("position" > 0)

Foreign-key constraints:
"results_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE
"results_team_id_fkey" FOREIGN KEY (team_id) REFERENCES teams(team_id) ON UPDATE CASCADE ON DELETE SET NULL



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'public' schema and the 'results' table. The right pane shows the SQL query and its results. The query is a CREATE TABLE statement for 'results' with columns 'result_id', 'position', 'score', 'event_id', and 'team_id'. The results pane shows a table with 20 rows and 5 columns: 'result_id', 'position', 'score', 'event_id', and 'team_id'. The data is as follows:

result_id	position	score	event_id	team_id
1	1	35.65	24	51
2	2	55.39	51	125
3	1	79.55	79	121
4	3	61.91	19	177
5	3	74.69	29	189
6	3	88.84	5	195
7	3	79.79	23	166
8	1	61.25	4	154
9	3	97.58	44	196
10	1	75.14	39	132
11	3	91.56	25	154
12	2	89.76	24	75
13	3	89.82	77	67
14	2	79.61	35	147
15	1	56.81	1	118
16	2	79.78	75	109
17	1	94.41	47	93
18	2	61.43	69	115
19	2	63.89	69	92
20	3	63.58	12	127

Total rows: 200 Query complete 60.90.90.366

Successfully run. Total query runtime: 289 msec. 250 rows affected.

13.) Archive

I-Fest=# \d archive

Table "public.archive"				
Column	Type	Collation	Nullable	Default
archive_id	integer		not null	nextval('archive_archive_id_seq'::regclass)
year	integer			
highlights	text			
event_id	integer			

Indexes:

"archive_pkey" PRIMARY KEY, btree (archive_id)

Check constraints:

"archive_year_check" CHECK (year >= 1900)

Foreign-key constraints:

"archive_event_id_fkey" FOREIGN KEY (event_id) REFERENCES events(event_id) ON UPDATE CASCADE ON DELETE CASCADE

The screenshot displays the pgAdmin 4 interface. On the left, the 'Object Explorer' shows the database structure, with the 'public' schema selected. The 'archive' table is highlighted under 'Tables (1)'. The main pane shows the 'SQL' tab with the table's definition and a query result. The query result shows 35 rows of data, with columns 'archive_id', 'year', and 'highlights' visible. The 'archive_id' column is the primary key, and the 'year' column has a check constraint for years greater than or equal to 1900. The 'highlights' column contains text descriptions of events.

archive_id	year	highlights
1	2020	Building between final one such soon. Continue represent speak through really dinner. Often issue expect lead none.
2	2021	Himself dinner white. Celebration tonight yes. Across understand decision better lead information.
3	2017	Made similar keep rule aim production head. Best treatment military important. Instead community active stuff.
4	2017	Trail eat serious kitchen. Hotel new accent toward actually side crime common.
5	2016	Reduce someone natural animal. Consumer along white crime song billion. Each effort hold let break big.
6	2022	Information difficult fact available great then. And activity her. Type explain trouble.
7	2018	Participant when customer method hold family. Too senior blue many purpose note.
8	2021	Mouth attorney case court. As door situation job out.
9	2022	Network reach not using series. A or perhaps fact full. Just each in.
10	2010	Professor huge ball either self. Million add however hand change information current.
11	2019	Station officer head exist. Especially interview almost land. Group experience present mouth new outside.
12	2020	Open education discussion full new rule life. Particularly remember through green brand stuff high they.
13	2020	Evening series TV lose. Suburban security state team.
14	2022	Come statement agree space position. Upon to even particular.
15	2021	For resource offer natural test mind. Ideas meet manage.
16	2021	Don't information health explain.
17	2024	Eye team color white. Smile standard face local keep. Last plot manager at result doctor.
18	2019	Top sister maintain trouble course focus for main. Activity new defense here even better. Better light camera yes.
19	2019	Act civil character act. Participant parent course.
20	2010	Station drop child. Sometimes away suddenly growth special cup stuff mark in.

1. A Function created that sets qr_data after a row is inserted

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including Schemas (public), Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, and Tables (13). The main window displays a SQL query window with the following code:

```
-- 1) Create function that sets qr_data after a row is inserted
CREATE OR REPLACE FUNCTION generate_participant_qr_data_after()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Build a QR string using the assigned participant_id and a timestamp
    -- Example: PART-123-1698651234
    UPDATE Participants
    SET qr_data = 'PART-' || NEW.participant_id || '-' || EXTRACT(EPOCH FROM NOW())
    WHERE participant_id = NEW.participant_id;

    RETURN NULL; -- AFTER triggers return NULL
END;
$;
```

Below the query window, the Data Output tab shows a table with 16 rows. The table has columns: participant_id (PK integer), year integer, highlights text, and event_id integer. The data is as follows:

participant_id	year	highlights	event_id
1	2016	Letter city throughout head. Choice environmental step. Election house instead company analysis.	97
2	2024	Campaign manage late. Night ago physical about nest when science price. Point thought visit approach character.	83
3	2017	Wind question price president pass. However Mrs animal. Wait affect last career deep similar.	242
4	2023	Fight left heart product rather paper product. Both hold seem north may foot music.	211
5	2017	Those sound present think. Upon development hard box government discussion. Expert season right adult maybe c...	100
6	2019	Read carry account amount wall. Republican recently model cover yourself three. Worker full speak fire.	216
7	2016	Remember data beyond common behind. Pattern bar one small.	168
8	2020	Most treat stand increase they support. Stay myself method never impact.	249
9	2018	Never remain investment deal tell commercial argue. Late decade idea fill.	180
10	2015	In ten eight beautiful site. Game project power above parent. Experience enjoy then million.	210
11	2022	Direction edge than whole many. Since purpose goal discover without value.	124
12	2017	Him easy guy. Dark news structure job anything daughter common happy. Name current you toward have.	196
13	2020	Seek system the. Popular discussion call watch author together.	82
14	2020	Care pick church gas push record yet. On task ground. Property less others lead beat enjoy.	93
15	2019	Weight sister that rock plan degree check. Important serious item wide mission standard later.	3
16	2025	Collection shake away sell television Mr player. Foot left always well cut.	246

Total rows: 600. Query complete 00:00:00.349.

2. A Trigger created to call the above function after insert

The screenshot shows the pgAdmin 4 interface. The main window displays a SQL query window with the following code:

```
--2) Create trigger to call the function after insert
DROP TRIGGER IF EXISTS trg_generate_qr_data_after ON Participants;

CREATE TRIGGER trg_generate_qr_data_after
AFTER INSERT ON Participants
FOR EACH ROW
EXECUTE FUNCTION generate_participant_qr_data_after();
```

Python Script usage for the Unique Code Generation

```
generate_qr.py
File Edit View
print("Database connection failed:", e)
sys.exit(1)

cur = conn.cursor()

try:
    # Step 3: Fetch participants who don't yet have a QR code image
    cur.execute("SELECT participant_id, qr_data FROM Participants WHERE qr_code_path IS NULL;")
    rows = cur.fetchall()

    if not rows:
        print("All participants already have QR codes.")
    else:
        for pid, qr_data in rows:
            # Step 4: Define file name and path
            file_name = f"participant_{pid}.png"
            file_path = os.path.join(output_dir, file_name)
            abs_file_path = os.path.abspath(file_path)

            # Step 5: Generate QR code
            img = qrcode.make(qr_data)
            img.save(abs_file_path)

            # Step 6: Update the database with the file path
            cur.execute(
                "UPDATE Participants SET qr_code_path = %s WHERE participant_id = %s",
                (abs_file_path, pid)
            )
            conn.commit()

            print(f"Generated QR for participant_id={pid} + {abs_file_path}")

        print("\nAll QR images generated successfully!")

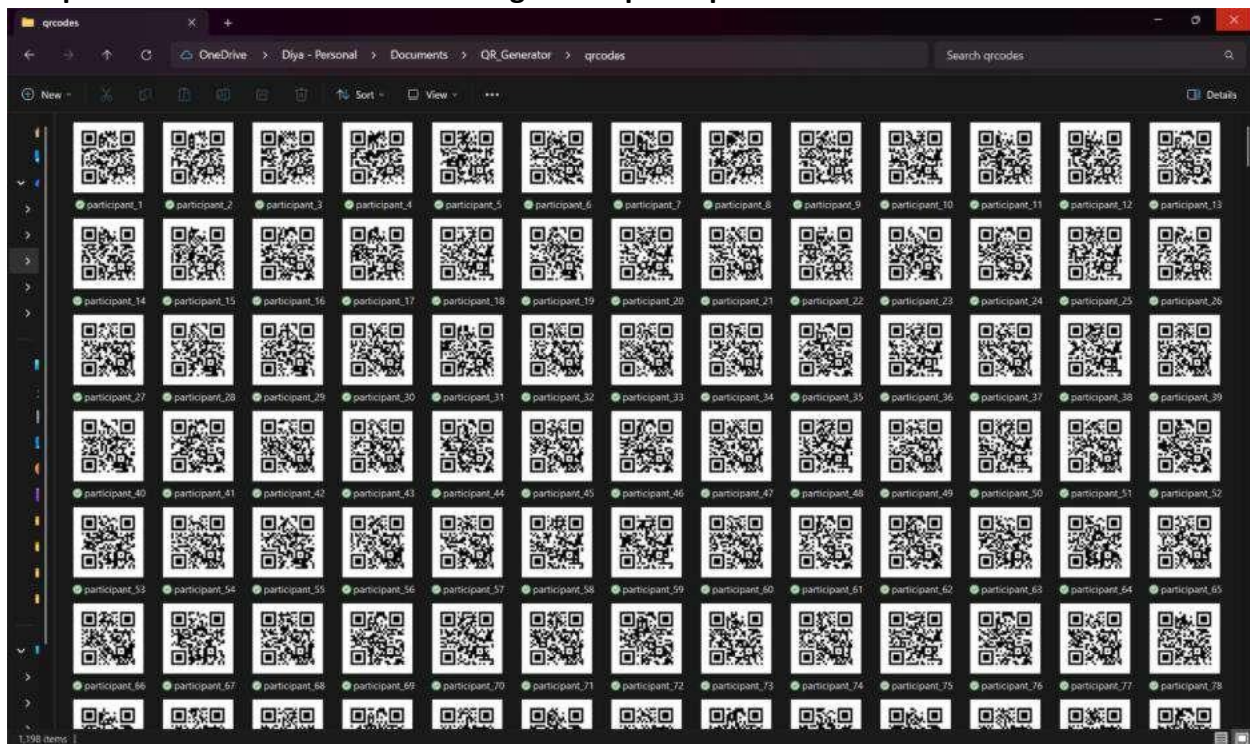
except Exception as e:
    print("Error during QR generation:", e)
    conn.rollback()

finally:
    cur.close()
    conn.close()
    print("Database connection closed.")

if __name__ == "__main__":
    main()
```

Ln 70, Col 20 2,442 characters Plain text 100% Windows (CRLF) UTF-8

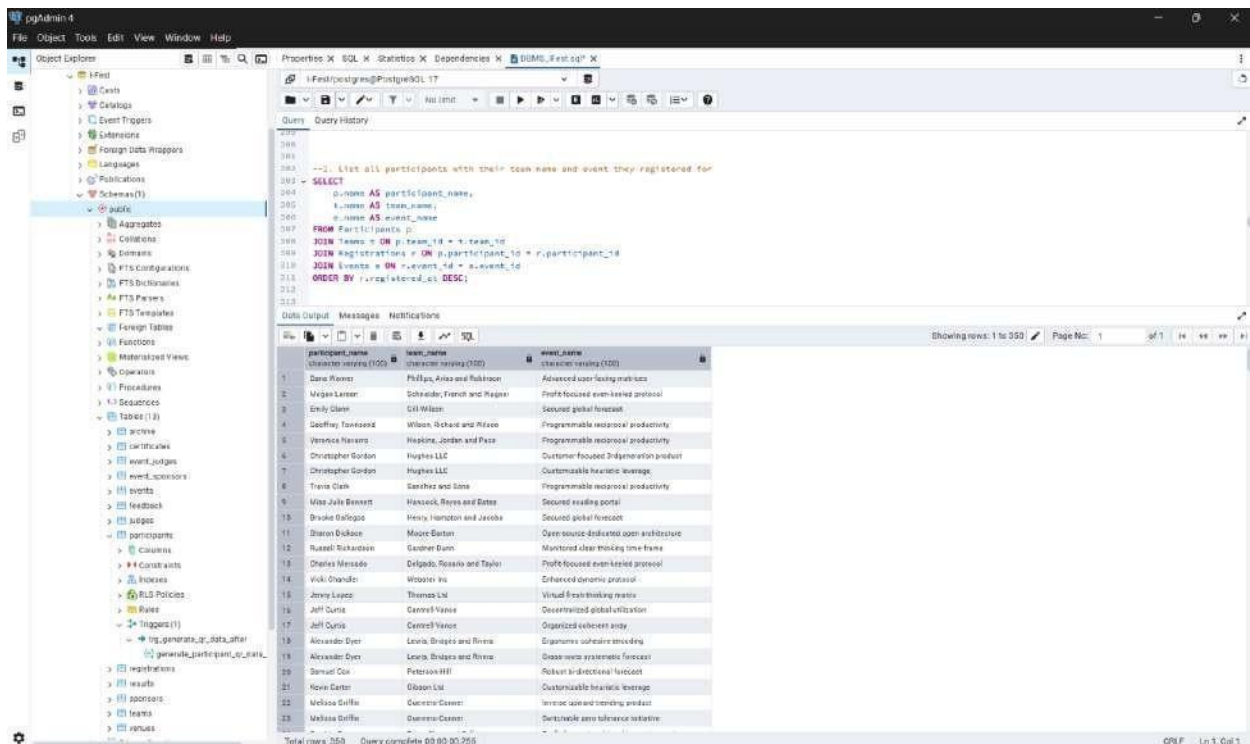
Unique Codes Generated for all the registered participants



9. SQL Queries

1. List all participants with their team name and event they registered for.

```
SELECT p.name AS participant_name, t.name AS team_name, e.name AS  
event_name FROM Participants p JOIN Teams t ON p.team_id = t.team_id JOIN  
Registrations r ON p.participant_id = r.participant_id JOIN Events e ON  
r.event_id = e.event_id ORDER BY r.registered_at DESC;
```



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including schemas, tables, and views. The central pane shows a SQL query that lists all participants with their team names and the events they registered for, ordered by registration date in descending order. The right pane displays the query results as a table with three columns: participant_name, team_name, and event_name. The results show 25 rows of data, including participants like Dave Reeves, Megan Lester, and Emily Clark, along with their respective teams and events.

participant_name	team_name	event_name
Dave Reeves	Phillips, Ariza and Robinson	Advanced user testing workshop
Megan Lester	Schneider, French and Nguyen	Profit-focused event-based protocol
Emily Clark	Gill Wilson	Secured global forecast
Geoffrey Foxwood	Wilson, Richard and Wilson	Programmable end-to-end productivity
Vernice Havers	Hopkins, Jordan and Pace	Programmable end-to-end productivity
Christopher Gordon	Hughes LLC	Customer-focused end-to-end productivity
Christopher Gordon	Hughes LLC	Customizable heuristic leverage
Travis Clark	Sanchez and Sims	Programmable end-to-end productivity
Moss Julie Bennett	Hancock, Reyes and Bates	Secured scaling portal
Bruce Gallagher	Henry, Hamilton and Jacobs	Secured global forecast
Sharon Bullock	Moore Barton	Open source end-to-end open architecture
Russell Richardson	Gardner Dunn	Managed clear thinking time frame
Charles Hernandez	Delgado, Rossini and Taylor	Profit-focused event-based protocol
Vicki Chandler	Winters Inc	Enhanced chronic protocol
Jenny Lopez	Thomas Ltd	Virtual event thinking events
Jeff Davis	Carmel Hayes	Decentralized globalization
Jeff Davis	Carmel Hayes	Organized robust entry
Alexander Oyer	Leeds, Bishop and Rivers	Engineered coherent encoding
Alexander Oyer	Leeds, Bishop and Rivers	Open source systemwide forecast
Daniel Cox	Peterson Hill	Robust bi-directional footprint
Kevin Carter	Gibson Ltd	Customizable heuristic leverage
Melissa Griffin	Quinn-Corcoran	Managed open-ended protocol
Melissa Griffin	Quinn-Corcoran	Scalable zero tolerance test suite

2. Find events and total number of participants registered.

```
SELECT e.name AS event_name, COUNT(r.participant_id) AS total_participants FROM
Events e LEFT JOIN Registrations r ON e.event_id = r.event_id GROUP BY e.name ORDER
BY total_participants DESC;
```

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree view of the database schema, with 'public' selected. The central pane shows a SQL query editor with the following query:

```
--2. Find events and total number of participants registered
SELECT
  e.name AS event_name,
  COUNT(r.participant_id) AS total_participants
FROM Events e
LEFT JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.name
ORDER BY total_participants DESC;
```

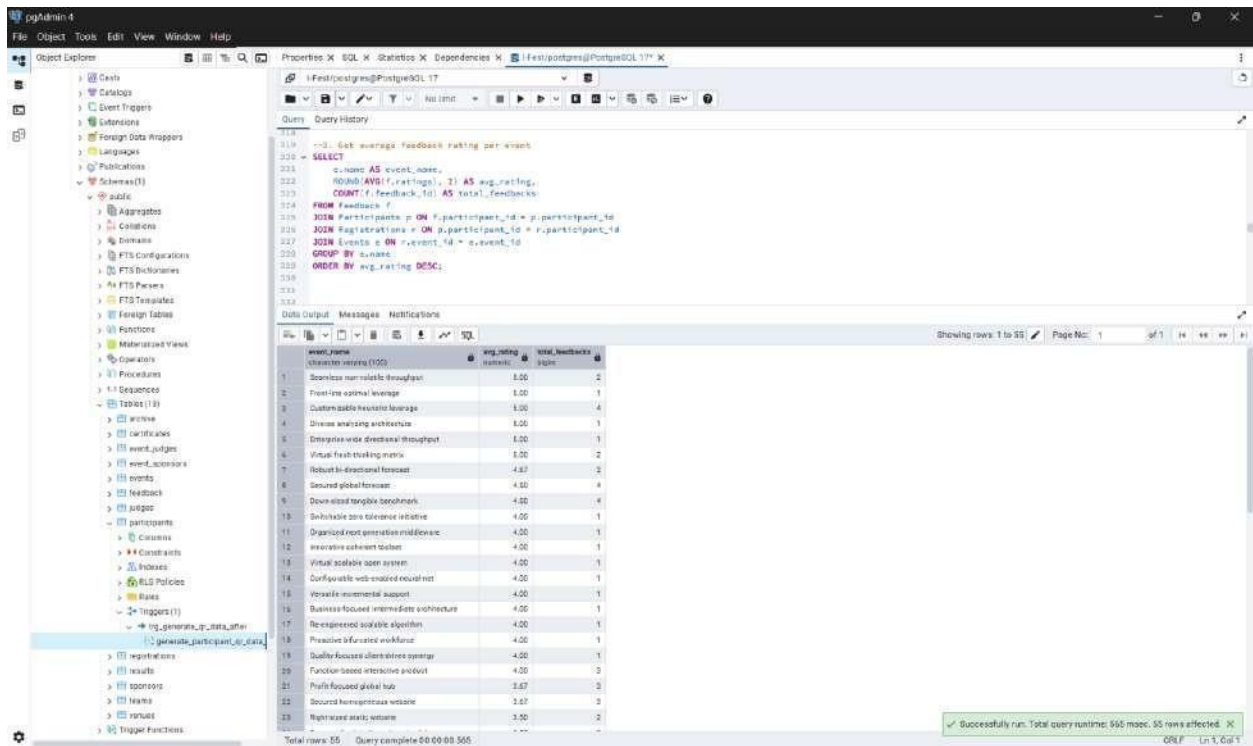
The bottom pane shows the 'Query Output' with the results of the query. The results are displayed in a table with two columns: 'event_name' and 'total_participants'. The table contains 23 rows of data, sorted by 'total_participants' in descending order.

event_name	total_participants
Secured global forum	8
Amplified digital knowledge	8
Open-architectured knowledge-informed hardware...	8
Programmable responsive productivity	8
Customizable heuristic leverage	7
Secured responsive methodology	7
Secured homogeneous weapons	7
Monitored foreground paradigm	7
Virtual scalable open system	7
One-stop analysis usability	7
Universal holistic implementation	7
Virtual fresh thinking matrix	7
Modular clear thinking time frame	7
Re-engineered scalable algorithm	7
Proactive bifurcated workforce	6
Robust bi-directional forecast	6
Profile focused multi-tasking orchestration	6
Expansive sensitive encoding	6
Scalable non-volatile throughput	6
Customizable executive framework	6
Front-line objective portal	6
Balanced responsive productivity	6
Profile focused global suite	6

The status bar at the bottom indicates 'Total rows: 00' and 'Query complete 00:00:00.320'.

3. Get average feedback rating per event.

```
SELECT e.name AS event_name, ROUND(AVG(f.ratings), 2) AS avg_rating,  
COUNT(f.feedback_id) AS total_feedbacks FROM Feedback f JOIN Participants p  
ON f.participant_id = p.participant_id JOIN Registrations r ON p.participant_id  
= r.participant_id JOIN Events e ON r.event_id = e.event_id GROUP BY e.name  
ORDER BY avg_rating DESC;
```



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'public' schema and various tables. The main pane shows a SQL query being executed. The query is as follows:

```
-- Get average feedback rating per event  
SELECT  
  e.name AS event_name,  
  ROUND(AVG(f.ratings), 2) AS avg_rating,  
  COUNT(f.feedback_id) AS total_feedbacks  
FROM Feedback f  
JOIN Participants p ON f.participant_id = p.participant_id  
JOIN Registrations r ON p.participant_id = r.participant_id  
JOIN Events e ON r.event_id = e.event_id  
GROUP BY e.name  
ORDER BY avg_rating DESC;
```

The query results are displayed in a table with the following columns: event_name, avg_rating, and total_feedbacks. The results are sorted by avg_rating in descending order.

event_name	avg_rating	total_feedbacks
Scalable nanoscale throughput	5.00	2
Front-line optimal leverage	5.00	1
Customizable heuristic leverage	5.00	4
Dynamic analyzing architecture	5.00	1
Emergence vertex directional throughput	5.00	1
Virtual fresh-stocking matrix	5.00	2
Robust bi-directional forecast	4.87	2
Secured global forecast	4.50	4
Down-sized tangible benchmark	4.00	4
Self-healing zero tolerance initiative	4.00	1
Organized next-generation middleware	4.00	1
Innovative coherent tactics	4.00	1
Virtual scalable open system	4.00	1
Configurable web-enabled neural net	4.00	1
Variable incremental support	4.00	1
Business-focused intermediate architecture	4.00	1
Re-engineered scalable algorithm	4.00	1
Proactive bifurcated workload	4.00	1
Quality-focused client-driven system	4.00	1
Function-focused interactive protocol	4.00	3
Profile-focused global hub	3.67	3
Secured heterogeneous website	3.67	3
High-speed entity website	3.00	2

The bottom status bar indicates: Total rows: 55. Query complete 00:00:00.565. A green message box at the bottom right states: Successfully run. Total query runtime: 565 msec. 55 rows effected.

4. Find top 3 teams by total score.

```
SELECT t.name AS team_name, SUM(r2.score) AS total_score FROM Results r2 JOIN  
Teams t ON r2.team_id = t.team_id GROUP BY t.name ORDER BY total_score DESC LIMIT  
3;
```

The screenshot shows the pgAdmin 4 web interface. On the left, the 'Object Explorer' pane shows a tree structure of databases, with 'I-Fest' selected under 'Databases (6)'. The main pane displays a SQL query in the 'Query' tab:

```
1 SELECT t.name AS team_name, SUM(r2.score) AS total_score FROM Results r2 JOIN Teams t  
2
```

Below the query editor, the 'Data Output' tab shows the results of the query in a table format. The table has two columns: 'team_name' (character varying (100)) and 'total_score' (numeric). The results are as follows:

	team_name	total_score
1	Ramirez-Evans	430.67
2	Gonzalez and Sons	428.92
3	Franklin, Brown and Peterson	362.47

At the bottom of the interface, a status bar indicates 'Total rows: 3' and 'Query complete 00:00:00.140'. A notification box in the bottom right corner states: 'You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9. Please click [here](#) for more information.'

5. Show participants who received certificates for events in 2025.

```
SELECT p.name AS participant_name, e.name AS event_name, c.issue_date FROM
Certificates c JOIN Participants p ON c.participant_id = p.participant_id JOIN Events e ON
c.event_id = e.event_id WHERE EXTRACT(YEAR FROM c.issue_date) = 2025 AND
EXTRACT(MONTH FROM c.issue_date) = 6;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows a tree structure with Servers (1), PostgreSQL 17, Databases (6), and I-Fest selected. The main pane displays a SQL query in the Query editor. Below the query, the Data Output pane shows the results of the query. The results are displayed in a table with 3 columns: participant_name, event_name, and issue_date. The table contains 37 rows of data. A status message at the bottom right indicates that the query was successfully run, with a total query runtime of 128 msec and 37 rows affected. A warning message also appears, stating that the current version of pgAdmin 4 is 9.4, while the version shown in the screenshot is 9.9.

```
SELECT p.name AS participant_name, e.name AS event_name, c.issue_date FROM Certificate
```

	participant_name character varying (100)	event_name character varying (100)	issue_date date
1	Michael Massey	Synchronized scalable contingency	2025-06-15
2	Scott Johnson	Realigned mobile info-mediaries	2025-06-11
3	Andrea Smith	Persistent value-added task-force	2025-06-16
4	James Solis	Team-oriented heuristic parallelism	2025-06-22
5	Jacqueline White	User-friendly composite analyzer	2025-06-11
6	Jason King	Polarized logistical toolset	2025-06-14
7	Kimberly White	Organic human-resource projection	2025-06-28
8	Jennifer Miller	Fundamental 6thgeneration help-desk	2025-06-22
9	Jason Brown	Self-enabling disintermediate flexibility	2025-06-07

Total rows: 37 Query complete 00:00:00.128

Showing rows: 1 to 37 Page No: 1 of 1

Successfully run. Total query runtime: 128 msec. 37 rows affected.

You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9.

Please click [here](#) for more information.

6. Find all judges assigned to each event.

```
SELECT e.name AS event_name, STRING_AGG(j.name, ', ') AS judges FROM  
Event_Judges ej JOIN Events e ON ej.event_id = e.event_id JOIN Judges j ON ej.judge_id  
= j.judge_id GROUP BY e.name;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > I-Fest. The main pane displays a SQL query:

```
1 SELECT e.name AS event_name, STRING_AGG(j.name, ', ') AS judges FROM Event_Judges ej  
2 JOIN Events e ON ej.event_id = e.event_id JOIN Judges j ON ej.judge_id  
3 = j.judge_id GROUP BY e.name;
```

The Data Output pane shows the results of the query, displaying 217 rows. The columns are event_name (character varying (100)) and judges (text). The results are as follows:

event_name	judges
Realigned demand-driven middleware	Samuel David, William Anderson
Exclusive client-driven focus group	Cory Garcia
Cross-platform bi-directional project	Bianca Reynolds, Julie Nguyen
Upgradable foreground time-frame	Erik Harris
Right-sized homogeneous Local Area Network	Lee Brown, Destiny Guzman
Cross-group optimizing framework	Kathy Butler, Mr. Robert Oliver
Fully-configurable disintermediate task-force	Michelle Ellis
Progressive object-oriented instruction set	Donald Rodgers, William Ross, Heidi Smith
Open-source heuristic architecture	Beth Clarke

A green notification bar indicates: "Successfully run. Total query runtime: 119 msec. 217 rows affected." An orange warning box states: "You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9. Please click [here](#) for more information."

7. Find participants who haven't given feedback.

```
SELECT p.name, p.email FROM Participants p LEFT JOIN Feedback f ON p.participant_id = f.participant_id WHERE f.feedback_id IS NULL;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > 202512078_db > I-Fest. The main pane displays a SQL query: `SELECT p.name, p.email FROM Participants p LEFT JOIN Feedback f ON p.participant_id =`. The Data Output pane shows the results of the query, displaying a table with two columns: `name` and `email`. The table contains 9 rows of data. A green notification bar at the bottom right indicates: "Successfully run. Total query runtime: 177 msec. 420 rows affected." An orange warning box also displays, stating: "You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9. Please click [here](#) for more information."

	name	email
1	Susan Richardson	edwinlopez@example.org
2	Tracy Smith	nlopez@example.net
3	Brittany Bennett	pbrown@example.org
4	Andrew Santana	tonygarcia@example.com
5	Leslie Hunter	garciabrian@example.net
6	Kathy Blair	victorpowers@example.org
7	Stuart Trevino	ubowen@example.net
8	Daniel Peters	perryanderson@example.net
9	Michael Morris	cassandrabrown@example.net

8. Events happening in venues with capacity below 100.

```
SELECT e.name AS event_name, v.venue_name, v.capacity FROM Events e JOIN Venues v ON e.venue_id = v.venue_id WHERE v.capacity < 1000;
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the 'Object Explorer' with a tree view of the database structure. The main pane shows a SQL query being executed. The 'Data Output' tab at the bottom displays the results of the query in a table format. The table has three columns: 'event_name', 'venue_name', and 'capacity'. The results show 70 rows of data, with the first 9 rows visible. A green notification box indicates the query was successfully run. An orange warning box at the bottom right states that the current version of pgAdmin 4 is 9.4, while the current version is 9.9.

Query:

```
SELECT e.name AS event_name, v.venue_name, v.capacity FROM Events e JOIN Venues v ON e.venue_id = v.venue_id WHERE v.capacity < 1000;
```

Data Output:

event_name	venue_name	capacity
Advanced scalable throughput	Bradleymouth Auditorium	967
Devolved attitude-oriented hub	Mccallbury Auditorium	430
Self-enabling system-worthy matrix	West Tonya Auditorium	430
Horizontal client-server application	East Kristinville Auditorium	898
Intuitive high-level alliance	New Alicia Auditorium	493
Vision-oriented maximized hardware	North Roger Auditorium	838
Monitored grid-enabled challenge	South Kevinbury Auditorium	696
Distributed 5thgeneration definition	Hicksmouth Auditorium	321
Centralized radical model	Walkerside Auditorium	148

Successfully run. Total query runtime: 145 msec. 70 rows affected.

You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9.

Please click [here](#) for more information.

9. Archive report — list all past archived events with venue.

```
SELECT a.year, e.name AS event_name, v.venue_name, a.highlights FROM Archive a
JOIN Events e ON a.event_id = e.event_id JOIN Venues v ON e.venue_id = v.venue_id
ORDER BY a.year ASC;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > 202512078_db > I-Fest. The main pane shows the SQL query editor with the following query:

```
1 SELECT a.year, e.name AS event_name, v.venue_name, a.highlights FROM Archive a JOIN Ev
2
```

The Data Output pane shows the results of the query. The table has 4 columns: year (integer), event_name (character varying (100)), venue_name (character varying (100)), and highlights (text). The results are as follows:

year	event_name	venue_name	highlights
2015	Seamless 5thgeneration functionalities	Lauraton Auditorium	Like however return from. See politics training race artist blue. Garden paper rock
2015	Triple-buffered didactic firmware	Port Sallyfurt Auditorium	Red person matter administration.
2015	Decentralized composite approach	Port Kylie Auditorium	Discuss activity eight stand. Society much save live let couple manage spring. Su
2015	Robust asynchronous functionalities	Shannonborough Auditorium	Floor ball or value partner pay look. Amount continue wonder find animal box indi
2015	User-centric 5thgeneration definition	Mccallbury Auditorium	Skin part together skill professor drive very. Budget yeah main travel year whose.
2015	Reverse-engineered stable service-desk	Williestad Auditorium	Democrat thing law within. Get loss ball employee very join.
2015	Balanced explicit focus group	Rachaelbury Auditorium	Thousand they clear member market create choice table. Dream hit involve home
2015	Organic human-resource projection	Spearsbury Auditorium	Right situation product drop move federal. Know deal voice exactly well.
2015	Balanced explicit focus group	Rachaelbury Auditorium	

Total rows: 350 Query complete 00:00:00.128

Successfully run. Total query runtime: 128 msec. 350 rows affected.

10. Sponsors Contributing Over ₹1,00,000.

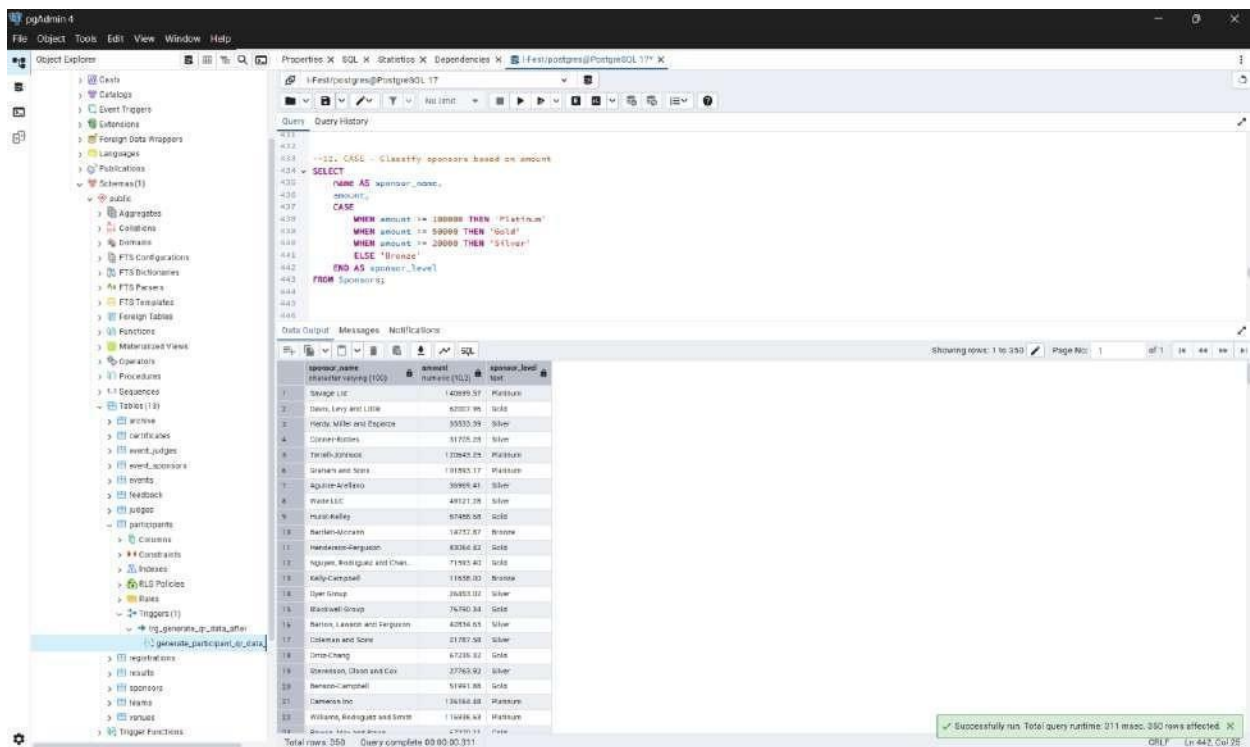
```
SELECT s.name AS sponsor_name, SUM(s.amount) AS total_sponsorship FROM Sponsors  
s JOIN Event_Sponsors es ON s.sponsor_id = es.sponsor_id GROUP BY s.name HAVING  
SUM(s.amount) > 100000 ORDER BY total_sponsorship DESC;
```

The screenshot displays the pgAdmin 4 web interface. On the left, the Object Explorer shows the database structure for 'I-Fest' under 'PostgreSQL 17'. The main pane shows a SQL query being executed. The query is: `SELECT s.name AS sponsor_name, SUM(s.amount) AS total_sponsorship FROM Sponsors s JOIN Event_Sponsors es ON s.sponsor_id = es.sponsor_id GROUP BY s.name HAVING SUM(s.amount) > 100000 ORDER BY total_sponsorship DESC;`. The results are shown in a table with two columns: 'sponsor_name' and 'total_sponsorship'. The table contains 114 rows, with the first 9 rows visible. A green notification box at the bottom right indicates 'Successfully run. Total query runtime: 187 msec. 114 rows affected.'

	sponsor_name	total_sponsorship
1	Ward-Snyder	583827.64
2	Guerrero-Warren	532313.56
3	Carlson LLC	525635.32
4	Bernard, Riggs and Johnson	456712.72
5	Rodriguez-Gregory	444341.01
6	Li LLC	418180.77
7	Terry-Stokes	413086.17
8	Anderson PLC	370173.90
9	Farmer, Hooper and Chavez	366316.95

11. CASE – Classify sponsors based on amount

SELECT name AS sponsor_name, amount, CASE WHEN amount >= 100000 THEN 'Platinum' WHEN amount >= 50000 THEN 'Gold' WHEN amount >= 20000 THEN 'Silver' ELSE 'Bronze' END AS sponsor_level FROM Sponsors;



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'Sponsors' table. The central pane shows the following SQL query:

```
--12. CASE - Classify sponsors based on amount
--SELECT
--NAME AS sponsor_name,
--amount,
--CASE
--  WHEN amount >= 100000 THEN 'Platinum'
--  WHEN amount >= 50000 THEN 'Gold'
--  WHEN amount >= 20000 THEN 'Silver'
--  ELSE 'Bronze'
--END AS sponsor_level
FROM Sponsors;
```

The right pane displays the query results in a table with 350 rows. The columns are 'sponsor_name', 'amount', and 'sponsor_level'. The results are sorted by 'amount' in descending order. The first few rows are:

sponsor_name	amount	sponsor_level
Shreehari Mahesh (100)	142095.57	Platinum
Orange Ltd.	62053.98	Gold
Green, Lily and Lili	55533.39	Silver
Merle, Mike and Eapen	51755.28	Silver
Conner-Rodriguez	129643.28	Platinum
Tanaka-Johnson	101963.17	Platinum
Grannan and Sons	50965.41	Silver
Aquino-Arriola	49121.28	Silver
Wuwei LLC	67405.58	Gold
Musicalley	14757.87	Bronze
Barclay-Adams	63364.82	Gold
Manderson-Bergeson	71901.40	Gold
Nguyen, Rodriguez and Chan	11636.30	Bronze
Kelly-Campbell	28453.12	Silver
Oyer Group	76740.34	Gold
Marshall Group	42834.03	Silver
Benton, Leason and Ferguson	21787.58	Silver
Coleman and Stone	67235.92	Gold
Ortiz-Chang	27763.92	Silver
Stonelson, Olson and Cox	51491.86	Gold
Bernstein-Campbell	136184.88	Platinum
Daniels Inc.	11648.44	Bronze
Williams, Rodriguez and Smith	24616.11	Silver

The bottom status bar indicates: 'Successfully run. Total query runtime: 311 msec, 350 rows effected. 0/1' and 'Total rows: 350 Query complete 03:00:00.311'.

12. Subquery – List participants registered for more than one event.

```
SELECT p.name, COUNT(r.event_id) AS total_events FROM Participants p JOIN
Registrations r ON p.participant_id = r.participant_id GROUP BY p.name
HAVING COUNT(r.event_id) > 1;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > I-Fest. The main pane displays a SQL query in the Query editor:

```
1 SELECT p.name, COUNT(r.event_id) AS total_events FROM Participants p JOIN Registrations r
2 ON p.participant_id = r.participant_id GROUP BY p.name
3 HAVING COUNT(r.event_id) > 1;
```

Below the query editor, the Data Output pane shows the results of the query. It displays a table with two columns: 'name' (character varying (100)) and 'total_events' (bigint). The results are as follows:

	name	total_events
1	Veronica Schultz	2
2	Ashlee Henderson	2
3	Reginald Berry	2
4	Dr. Anthony Griffin	2
5	Jason Jordan MD	3
6	Lucas Bentley	2
7	Heather Long	2
8	Lisa Shannon	2
9	Tiffany Henderson	2

At the bottom of the Data Output pane, a status bar indicates: 'Total rows: 74', 'Query complete 00:00:00.129', and a green message box says 'Successfully run. Total query runtime: 129 msec. 74 rows affected.' The Windows taskbar at the bottom shows the time as 19:54 on 16-11-2025.

13. UNION – Get all emails (participants + judges + sponsors).

SELECT email, 'Participant' AS role FROM Participants UNION SELECT email, 'Judge' AS role FROM Judges UNION SELECT contact_info AS email, 'Sponsor' AS role FROM Sponsors;

The screenshot shows the pgAdmin 4 interface. The left pane displays the Object Explorer with the database structure. The central pane shows the SQL query editor with the following query:

```
1 SELECT email, 'Participant' AS role FROM Participants UNION SELECT email, 'Judge' AS role FROM Judges UNION SELECT contact_info AS email, 'Sponsor' AS role FROM Sponsors;
```

The bottom pane shows the Data Output tab with the following results:

email	role
leedeborah@example.net	Judge
allisonstevens@example.com	Judge
micheleoconnor@example.net	Participant
892.913.3448x42173	Sponsor
susanford@example.org	Judge
alisonlara@example.net	Participant
marymedina@example.net	Participant
narmstrong@example.org	Judge
lisahurst@example.net	Participant

A status bar at the bottom indicates: "Total rows: 1399 Query complete 00:00:00.148". A green message box at the bottom right says: "✓ Successfully run. Total query runtime: 148 msec. 1399 rows affected."

14. List Participants Who Have Generated QR Codes but Haven't Registered for Any Event.

```
SELECT p.name AS participant_name, p.email, p.qr_data, p.qr_code_path FROM
Participants p LEFT JOIN Registrations r ON p.participant_id = r.participant_id WHERE
r.registration_id IS NULL AND p.qr_data IS NOT NULL;
```

The screenshot shows the pgAdmin 4 interface. The left pane displays the Object Explorer with the following structure:

- Servers (1)
 - PostgreSQL 17
 - Databases (6)
 - 202512078_db
 - I-Fest
 - TEST_db3
 - company
 - company2
 - postgres
 - Login/Group Roles
 - Tablespaces

The main pane shows the SQL query editor with the following query:

```
1 SELECT p.name AS participant_name, p.email, p.qr_data, p.qr_code_path FROM Participant
2
```

The Data Output pane shows the results of the query, displaying 9 rows of data. The columns are: participant_name, email, qr_data, and qr_code_path. The data is as follows:

	participant_name	email	qr_data	qr_code_path
1	Susan Richardson	edwinlopez@example.org	PARTICIPANT-251	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_251.png
2	Madison Holder	ronaldward@example.org	PARTICIPANT-681	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_681.png
3	Jon Klein	michael01@example.net	PARTICIPANT-120	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_120.png
4	Brittany Bennett	pbrown@example.org	PARTICIPANT-285	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_285.png
5	Andrew Santana	tonygarcia@example.com	PARTICIPANT-264	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_264.png
6	Connor Wagner	michael12@example.net	PARTICIPANT-647	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_647.png
7	Stuart Trevino	ubowen@example.net	PARTICIPANT-497	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_497.png
8	Justin Vasquez	zhangrodney@example.net	PARTICIPANT-452	C:\Users\LENOVO\Documents\QR_Generator\qrcodes\participant_452.png
9	Maria Kelly	jweaver@example.org	PARTICIPANT-455	

The bottom status bar indicates: Total rows: 435, Query complete 00:00:00.167, and a message: Successfully run. Total query runtime: 167 msec. 435 rows affected.

15. Displaying QR-Enabled Participants Registered in Multiple Events

```
SELECT p.name AS participant_name, p.email, COUNT(DISTINCT r.event_id) AS
event_count FROM Participants p JOIN Registrations r ON p.participant_id =
r.participant_id WHERE p.qr_data IS NOT NULL GROUP BY p.participant_id, p.name,
p.email HAVING COUNT(DISTINCT r.event_id) > 1 ORDER BY event_count DESC;
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the Object Explorer with a tree view of the database structure. The main pane shows a SQL query editor with the following query:

```
1 SELECT p.name AS participant_name, p.email, COUNT(DISTINCT r.event_id) AS event_count
2
```

The bottom pane displays the Data Output tab, showing the results of the query in a table format. The table has three columns: participant_name, email, and event_count. The results are sorted by event_count in descending order.

	participant_name	email	event_count
1	Derek Barron	umurray@example.com	4
2	James Russell	joshuamullen@example.org	3
3	Holly Blankenship	woodsruessell@example.net	3
4	Ian Hernandez	ronald45@example.org	3
5	Madeline Serrano	brittanysimmons@example.net	3
6	Donna Odom	sbrown@example.com	3
7	Ronald McKinney	jeffersonrachel@example.net	3
8	Jason Jordan MD	stephen50@example.net	3
9	Mr. David Miller	carlaflaming@example.org	3

A status message at the bottom right indicates: "Successfully run. Total query runtime: 130 msec. 72 rows affected."

16. Show the top 5 most recently registered participants

```
SELECT p.name AS participant_name, p.email, r.registered_at, e.name AS event_name
FROM Registrations r JOIN Participants p ON r.participant_id = p.participant_id JOIN
Events e ON r.event_id = e.event_id ORDER BY r.registered_at DESC LIMIT 5;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > 202512078_db > I-Fest. The main pane shows a SQL query in the Query editor:

```
1 SELECT p.name AS participant_name, p.email, r.registered_at, e.name AS event_name FROM
2
```

The Data Output pane at the bottom displays the results of the query. It shows 5 rows of data. The status bar at the bottom indicates: Total rows: 5, Query complete 00:00:00.129, and a green message: Successfully run. Total query runtime: 129 msec. 5 rows affected.

	participant_name character varying (100)	email character varying (100)	registered_at timestamp without time zone	event_name character varying (100)
1	Erica Valenzuela	diane21@example.org	2025-10-28 14:26:03.087821	Customizable high-level structure
2	Jerry Marquez	qjohnson@example.org	2025-10-28 14:26:03.087821	Customizable attitude-oriented flexibility
3	Jesse Cunningham	sher126@example.org	2025-10-28 14:26:03.087821	Customizable incremental capacity
4	Joseph Perkins II	jonathon41@example.com	2025-10-28 14:26:03.087821	Switchable hybrid product
5	Chloe Conway	stephensrobert@example.com	2025-10-28 14:26:03.087821	Intuitive explicit synergy

17. Calculate the total number of participants per venue

```
SELECT v.venue_name, COUNT(DISTINCT r.participant_id) AS total_participants FROM  
Venues v JOIN Events e ON v.venue_id = e.venue_id JOIN Registrations r ON e.event_id  
= r.event_id GROUP BY v.venue_name ORDER BY total_participants DESC;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > 202512078_db > I-Fest. The main pane shows a SQL query in the Query editor:

```
1 SELECT v.venue_name, COUNT(DISTINCT r.participant_id) AS total_participants FROM Venues  
2 = r.event_id JOIN Events e ON v.venue_id = e.venue_id JOIN Registrations r ON e.event_id  
3 = r.event_id GROUP BY v.venue_name ORDER BY total_participants DESC;
```

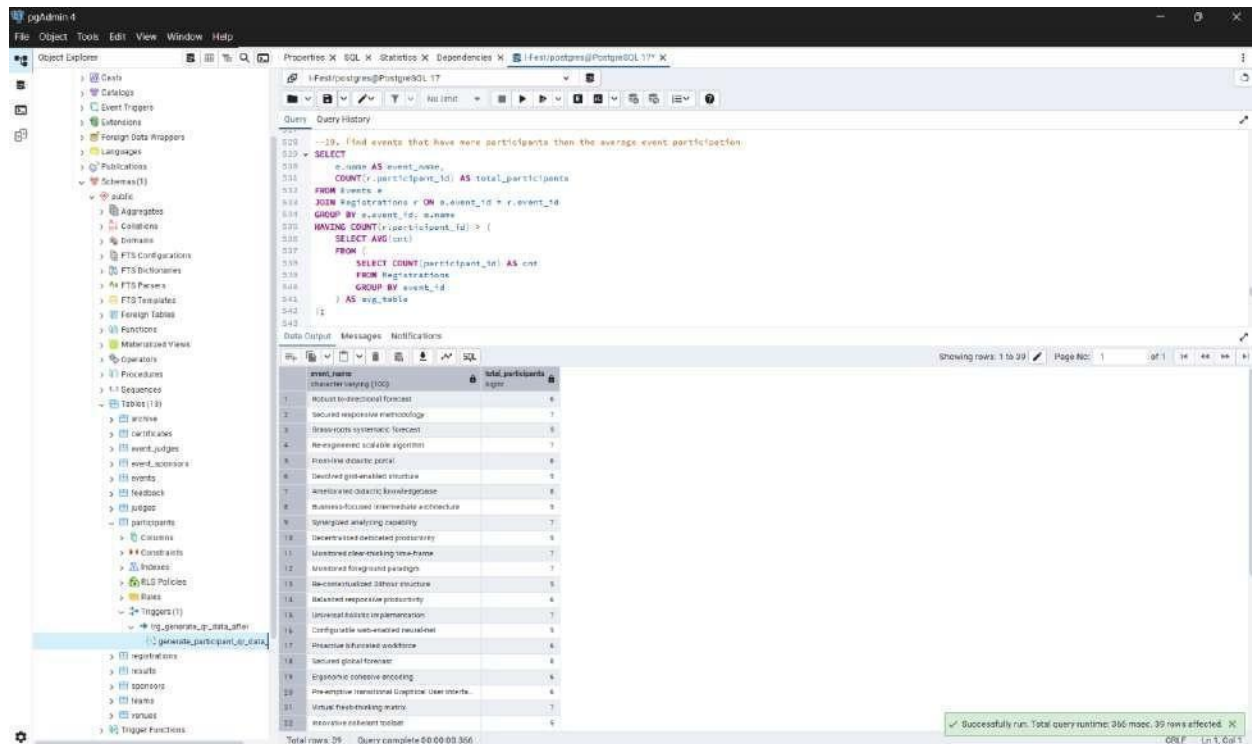
The Data Output pane at the bottom displays the results of the query. It shows a table with two columns: venue_name (character varying (100)) and total_participants (bigint). The results are ordered by total_participants in descending order.

venue_name	total_participants
New Andrewmouth Auditoriu...	10
East Jose Auditorium	8
Lake Thomasfurt Auditorium	5
Alyssaville Auditorium	5
South Meganchester Auditor...	5
Parkertown Auditorium	5
South Dawn Auditorium	5
New Alicia Auditorium	5
Jenniferbury Auditorium	4

A green status bar at the bottom right indicates: "Successfully run. Total query runtime: 147 msec. 172 rows affected." The bottom of the window shows the Windows taskbar with the date 16-11-2025 and time 19:58.

18. Find events that have more participants than the average event participation

```
SELECT e.name AS event_name, COUNT(r.participant_id) AS total_participants FROM
Events e JOIN Registrations r ON e.event_id = r.event_id GROUP BY e.event_id, e.name
HAVING COUNT(r.participant_id) > ( SELECT AVG(cnt) FROM ( SELECT
COUNT(participant_id) AS cnt FROM Registrations GROUP BY event_id ) AS avg_table );
```



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'public' schema and various tables. The central pane shows a SQL query being executed. The query is designed to find events with more participants than the average event participation. The results pane at the bottom displays a table with two columns: 'event_name' and 'total_participants'.

Query:

```
--19. Find events that have more participants than the average event participation
SELECT
  e.name AS event_name,
  COUNT(r.participant_id) AS total_participants
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.name
HAVING COUNT(r.participant_id) > (
  SELECT AVG(cnt)
  FROM (
    SELECT COUNT(participant_id) AS cnt
    FROM Registrations
    GROUP BY event_id
  ) AS avg_table
);
```

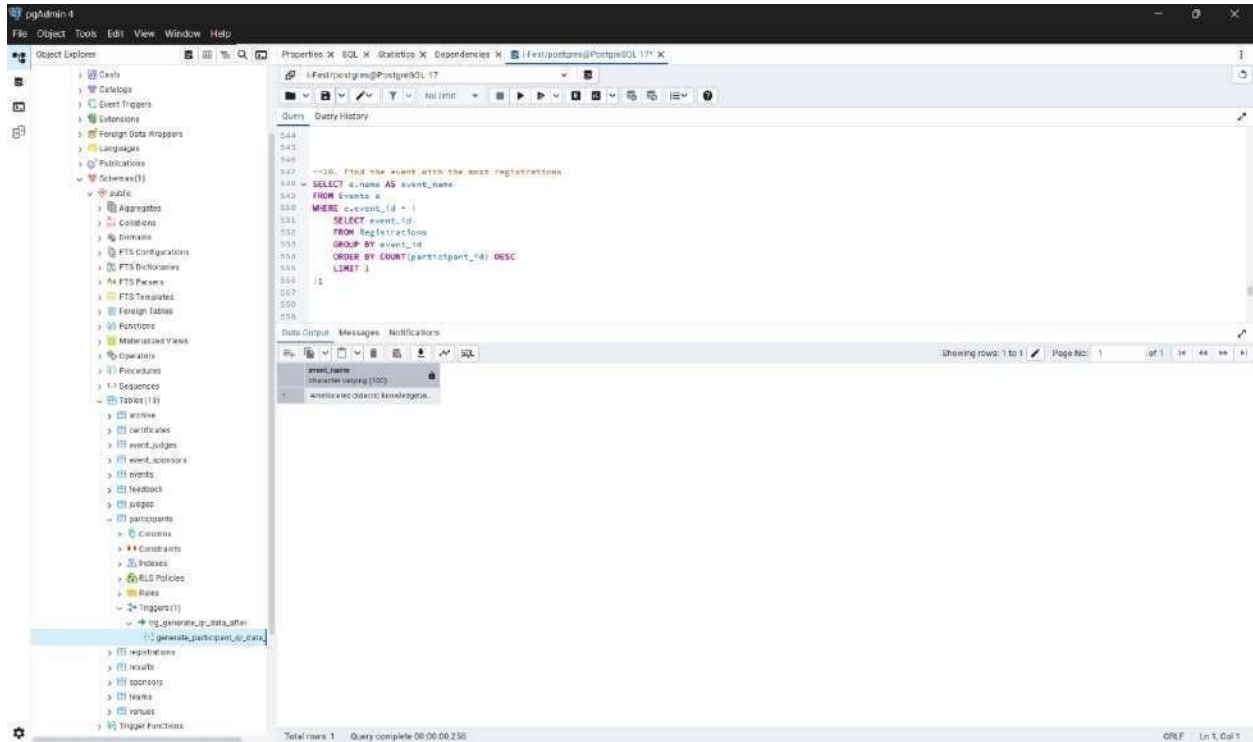
Results:

event_name	total_participants
Advanced Learning Tools	6
Adaptive Learning Forecast	7
Adaptive Response Methodology	7
Advanced Systems Forecast	6
Advanced Scalable Algorithms	7
Advanced Data Platform	6
Advanced Generalized Interface	6
Advanced Adaptive Knowledgebase	6
Business-focused Intermediate Architecture	6
Advanced Analyzing Capability	7
Advanced Distributed Processing	6
Advanced Real-time Time Frame	7
Advanced Background Processing	7
Advanced Distributed Structure	6
Advanced Response Processing	6
Advanced Adaptive Planning	7
Configurable Web-enabled Interface	6
Advanced Information Architecture	6
Advanced Global Interface	6
Advanced Adaptive Processing	6
Advanced Adaptive Learning User Interface	6
Advanced Adaptive Learning	7
Advanced Adaptive Learning	6

Successfully run. Total query runtime: 366 msec. 39 rows affected.

19. Find the event with the most registrations

```
SELECT e.name AS event_name FROM Events e WHERE e.event_id = (SELECT event_id  
FROM Registrations GROUP BY event_id ORDER BY COUNT(participant_id) DESC LIMIT  
1);
```



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database schema, including tables like 'events', 'registrations', and 'participants'. The main pane shows a SQL query in the Query Editor:

```
--26. Find the event with the most registrations  
SELECT e.name AS event_name  
FROM Events e  
WHERE e.event_id = (  
    SELECT event_id  
    FROM Registrations  
    GROUP BY event_id  
    ORDER BY COUNT(participant_id) DESC  
    LIMIT 1  
);
```

Below the query editor, the Data Output pane shows the results of the query. It displays a single row with the event name 'Knowledgeable Dialects'.

event_name
Knowledgeable Dialects

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.250'.

20. Find events that no one has given feedback for.

```
SELECT e.name AS event_name FROM Events e WHERE e.event_id NOT IN ( SELECT  
DISTINCT r.event_id FROM Feedback f JOIN Registrations r ON f.participant_id =  
r.participant_id );
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including Schemas, Tables, and Views. The main pane shows a SQL query window with the following text:

```
--[[. Find events that no one has given feedback for  
SELECT e.name AS event_name  
FROM Events e  
WHERE e.event_id NOT IN (  
SELECT DISTINCT r.event_id  
FROM Feedback f  
JOIN Registrations r ON f.participant_id = r.participant_id  
);
```

Below the query window, the Data Output pane displays the results of the query. The results are shown in a table with two columns: event_name and event_id. The table contains 25 rows of data, including event names like "Brain roots systematic feedback", "Secured existing portal", "High-speed tertiary actuator", etc.

At the bottom of the interface, a status bar indicates: "Total rows: 25 Query complete 00:00:00.274". A green message box at the bottom right states: "Successfully run. Total query runtime: 272 msec. 25 rows affected."

21. Find venues that host events with average rating above 4.

```
SELECT venue_name FROM Venues WHERE venue_id IN ( SELECT e.venue_id FROM
Events e JOIN Registrations r ON e.event_id = r.event_id JOIN Feedback f ON
r.participant_id = f.participant_id GROUP BY e.venue_id HAVING AVG(f.ratings) > 4 );
```

The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer showing the database schema. The main pane displays a SQL query in the Query Editor. The query is as follows:

```
--21. Find venues that host events with average rating above 4
SELECT venue_name
FROM Venues
WHERE venue_id IN (
  SELECT e.venue_id
  FROM Events e
  JOIN Registrations r ON e.event_id = r.event_id
  JOIN Feedback f ON r.participant_id = f.participant_id
  GROUP BY e.venue_id
  HAVING AVG(f.ratings) > 4
);
```

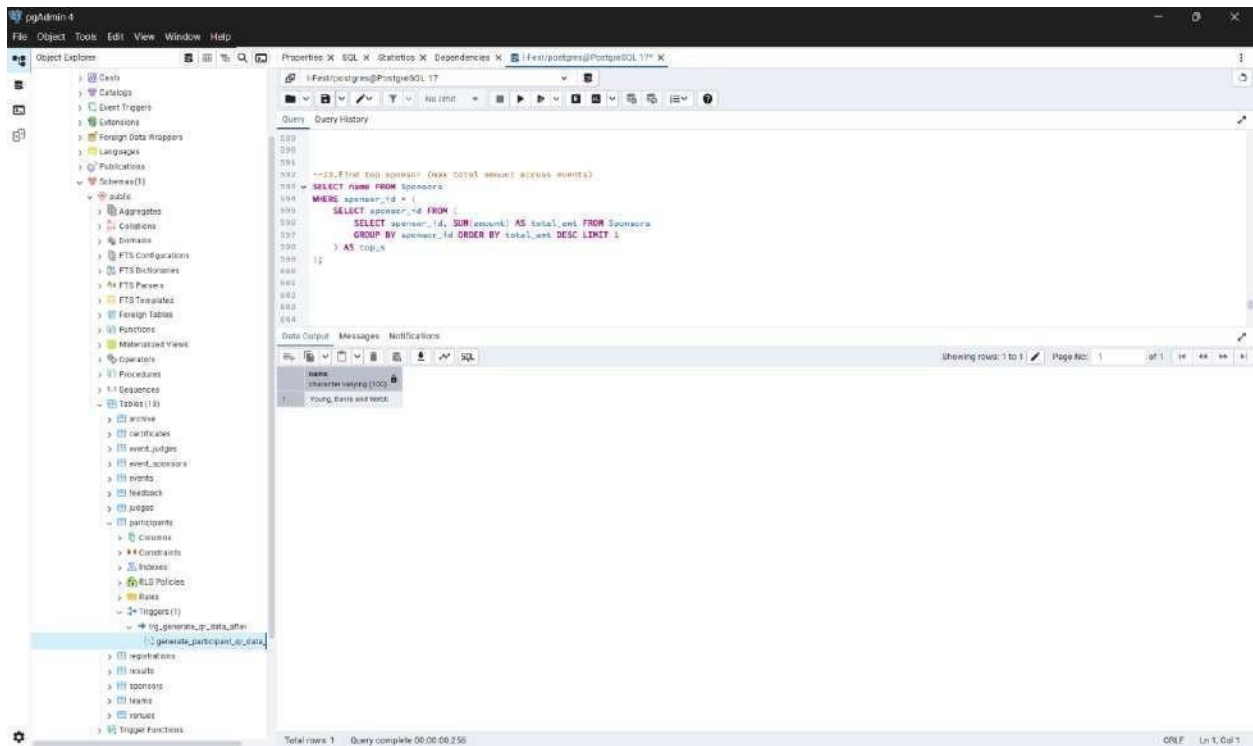
Below the query editor, the Data Output pane shows the results of the query. The results are as follows:

venue_name
North Kensington Auditorium
South Charles Auditorium
West Benjamin Auditorium
East Eugene Auditorium
Flowerdough Auditorium
Lake Andrew Auditorium
Lake Manchester Auditorium
Manchester Auditorium

At the bottom of the interface, a status bar indicates: "Total rows: 8 Query complete 06:00:00.317". A green message box at the bottom right says: "Successfully run. Total query runtime: 317 msec. 8 rows affected."

22. Find top sponsor (max total amount across events).

```
SELECT name FROM Sponsors WHERE sponsor_id = ( SELECT sponsor_id FROM ( SELECT  
sponsor_id, SUM(amount) AS total_amt FROM Sponsors GROUP BY sponsor_id ORDER  
BY total_amt DESC LIMIT 1 ) AS top_s );
```



23. Participants from events without sponsors.

```
SELECT name FROM Participants WHERE participant_id IN ( SELECT participant_id FROM Registrations WHERE event_id NOT IN (SELECT DISTINCT event_id FROM Event_Sponsors) );
```

The screenshot shows the pgAdmin 4 interface. The left pane displays the Object Explorer with the following structure:

- Servers (1)
 - PostgreSQL 17
 - Databases (6)
 - 202512078_db
 - I-Fest
 - TEST_db3
 - company
 - company2
 - postgres
 - Login/Group Roles
 - Tablespaces

The main pane shows the SQL editor with the following query:

```
1 SELECT name FROM Participants WHERE participant_id IN ( SELECT participant_id FROM Registrations WHERE event_id NOT IN (SELECT DISTINCT event_id FROM Event_Sponsors) );
```

The Data Output pane shows the results of the query:

	name
1	Brandon Davis
2	Tonya Nelson
3	Victor Bailey
4	Jennifer Miller
5	Lauren Stevens
6	Ronald Perez
7	Robert Bennett
8	Jessica Johnson

A green notification message at the bottom right states: "Successfully run. Total query runtime: 112 msec. 102 rows affected."

An orange notification message at the bottom right states: "You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9."

24. Find participants who got certificates for events with top sponsorship.

```
SELECT name FROM Participants WHERE participant_id IN ( SELECT c.participant_id
FROM Certificates c WHERE c.event_id = ( SELECT event_id FROM ( SELECT es.event_id,
SUM(s.amount) AS total_amt FROM Event_Sponsors es JOIN Sponsors s ON
es.sponsor_id = s.sponsor_id GROUP BY es.event_id ORDER BY total_amt DESC LIMIT 1 )
AS max_fund ) );
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > I-Fest. The main pane displays a SQL query in the Query editor:

```
1 SELECT name FROM Participants WHERE participant_id IN ( SELECT c.participa
2
```

The Data Output pane at the bottom shows the results of the query:

	name
1	Kimberly Garcia
2	Kelly Davis

A green message box at the bottom right indicates: "Successfully run. Total query runtime: 130 msec. 2 rows affected." Below it, an orange message box states: "You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9."

25. Judges assigned to the highest-rated event.

```
SELECT name FROM Judges WHERE judge_id IN (SELECT judge_id FROM Event_Judges
WHERE event_id = (SELECT event_id FROM Feedback f JOIN Registrations r ON
f.participant_id=r.participant_id GROUP BY r.event_id ORDER BY AVG(f.ratings) DESC
LIMIT 1));
```

The screenshot shows the pgAdmin 4 web interface. On the left, the Object Explorer shows a tree structure with Servers (1), PostgreSQL 17, Databases (6), and a selected database named 'I-Fest'. The main pane displays a SQL query in the Query editor:

```
1 .participant_id GROUP BY r.event_id ORDER BY AVG(f.ratings) DESC LIMIT 1));
```

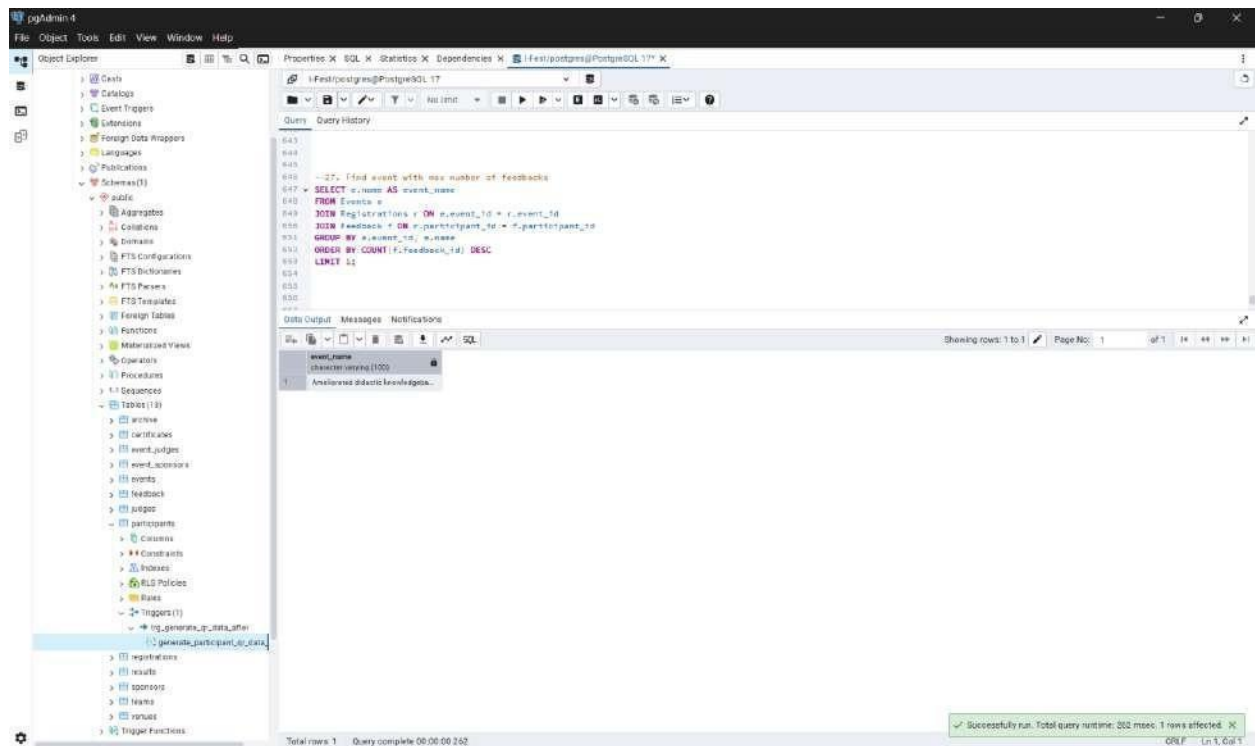
Below the query editor, the Data Output pane shows the results of the query. It displays a table with one row and one column:

name
Gregory Lane

At the bottom of the interface, a green notification box states: "Successfully run. Total query runtime: 404 msec. 1 rows affected." Below this, an orange notification box states: "You are currently running version 9.4 of pgAdmin 4, however the current version is 9.9."

26. Find event with max number of feedbacks

```
SELECT e.name AS event_name FROM Events e JOIN Registrations r ON e.event_id =  
r.event_id JOIN Feedback f ON r.participant_id = f.participant_id GROUP BY e.event_id,  
e.name ORDER BY COUNT(f.feedback_id) DESC LIMIT 1;
```



The screenshot shows the pgAdmin 4 web interface. On the left is the Object Explorer showing a database structure with tables like 'events', 'registrations', and 'feedback'. The main pane displays a SQL query in the 'Query' tab. The query is:
`SELECT e.name AS event_name FROM Events e JOIN Registrations r ON e.event_id = r.event_id JOIN Feedback f ON r.participant_id = f.participant_id GROUP BY e.event_id, e.name ORDER BY COUNT(f.feedback_id) DESC LIMIT 1;`
The 'Data Output' pane shows the results of the query, displaying a single row with the event name 'Advanced database knowledge...'. At the bottom, a status bar indicates 'Total rows: 1' and 'Query complete: 00:00:00.262'. A green message box at the bottom right states 'Successfully run. Total query runtime: 332 msec. 1 rows effected.'

27. Participants who belong to the team with highest score.

```
SELECT p.name, t.name AS team_name FROM Participants p JOIN Teams t ON p.team_id = t.team_id WHERE t.team_id = (SELECT team_id FROM Results GROUP BY team_id ORDER BY SUM(score) DESC LIMIT 1);
```

The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer tree. The main pane displays a SQL query in the Query Editor. The query is as follows:

```
--ja. Participants who belong to the team with highest score
SELECT p.name, t.name AS team_name
FROM Participants p
JOIN Teams t ON p.team_id = t.team_id
WHERE t.team_id = (
  SELECT team_id FROM Results
  GROUP BY team_id
  ORDER BY SUM(score) DESC
  LIMIT 1
);
```

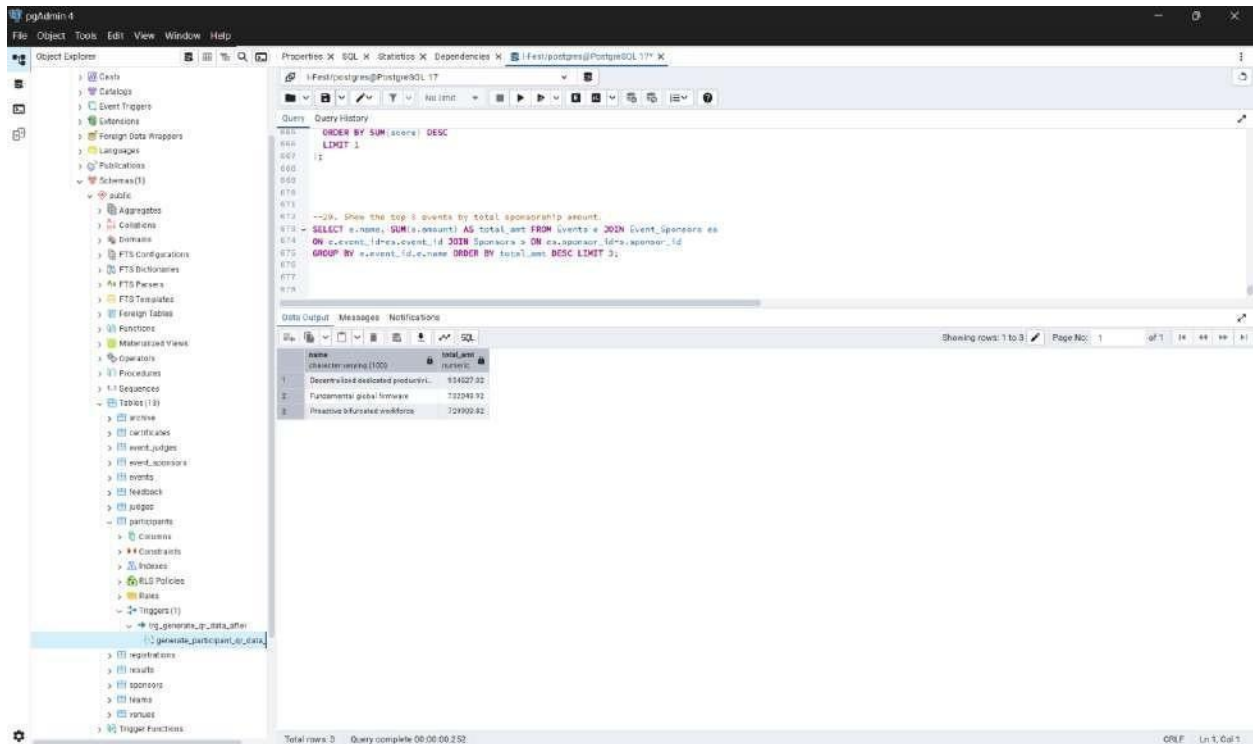
Below the query editor, the Data Output pane shows the results of the query. The results are displayed in a table with two columns: name and team_name. The table contains four rows of data.

name	team_name
Nancy Anthony	Devix, Burgess and Wood
Thomas Davis	Devix, Burgess and Wood
Wily Davis	Devix, Burgess and Wood
Storrs Price	Devix, Burgess and Wood

At the bottom of the interface, a status bar indicates: "Total rows: 4 Query complete 00:00:00.195". A green message box at the bottom right states: "Successfully run. Total query runtime: 195 msec. 4 rows effected."

28. Show the top 3 events by total sponsorship amount.

```
SELECT e.name, SUM(s.amount) AS total_amt FROM Events e JOIN Event_Sponsors es ON  
e.event_id=es.event_id JOIN Sponsors s ON es.sponsor_id=s.sponsor_id GROUP BY  
e.event_id,e.name ORDER BY total_amt DESC LIMIT 3;
```



The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer tree. The main pane displays a SQL query in the Query Editor. Below the query editor is the Data Output pane, which shows the results of the query in a table format.

Query:

```
--Q28. Show the top 3 events by total sponsorship amount.  
SELECT e.name, SUM(s.amount) AS total_amt FROM Events e JOIN Event_Sponsors es  
ON e.event_id=es.event_id JOIN Sponsors s ON es.sponsor_id=s.sponsor_id  
GROUP BY e.event_id,e.name ORDER BY total_amt DESC LIMIT 3;
```

Data Output:

name	total_amt
Decentralized dedicated productivity	934027.92
Functional global framework	732949.92
Proactive bifurcated workflow	729999.92

Total rows: 3 Query complete 08:00:06.252

29. Find the sponsor who sponsored both the highest and lowest-funded events.

```
SELECT s.name FROM Sponsors s WHERE s.sponsor_id IN (SELECT es.sponsor_id FROM
Event_Sponsors es WHERE es.event_id IN ((SELECT event_id FROM Event_Sponsors JOIN
Sponsors USING(sponsor_id) GROUP BY event_id ORDER BY SUM(amount) DESC LIMIT 1),
(SELECT event_id FROM Event_Sponsors JOIN Sponsors USING(sponsor_id) GROUP BY event_id
ORDER BY SUM(amount) ASC LIMIT 1)));
```

The screenshot shows the pgAdmin 4 interface. The left pane displays the database schema tree, with the 'Sponsors' table selected under the 'public' schema. The main pane shows a SQL query in the 'Query' editor, which is the same query as provided in the problem statement. The 'Data Output' pane at the bottom displays the results of the query, showing a list of sponsor names. The status bar at the bottom indicates 'Total rows: 10' and 'Query complete 00:00:00.253'.

name
Shawnter-Vernish (100)
Brown, Chapman and Myers
Johnson, Ball and Diaz
Weeds, Koch and Davis
Miller Group
Mccarthy, McDonald and Gero
Hess Inc
Carillo Inc
Hernandez PLC
Jones, Heath and Powell
Holly LLC

31. Events where every participant received a certificate.

```
SELECT e.name FROM Events e WHERE NOT EXISTS (SELECT r.participant_id FROM Registrations r WHERE r.event_id = e.event_id AND r.participant_id NOT IN (SELECT c.participant_id FROM Certificates c WHERE c.event_id = e.event_id));
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'I-Fest' database. The main window shows a SQL query being executed. The query is:

```
SELECT e.name FROM Events e WHERE NOT EXISTS (SELECT r.participant_id FROM Registrations r WHERE r.event_id = e.event_id AND r.participant_id NOT IN (SELECT c.participant_id FROM Certificates c WHERE c.event_id = e.event_id));
```

The query results are displayed in a table with 131 rows. The first 18 rows are visible:

name
Polarized logistical toolset
Diverse web-enabled customer loyalty
Proactive systemic artificial intelligence
Secured tertiary solution
Optimized actuating initiative
Function-based value-added parallelism
Programmable multimedia paradigm
Versatile upward-trending moderator
Progressive attitude-oriented functionalities
Multi-lateral system-worthy paradigm
Public-key motivating challenge
Programmable secondary service-desk
Configurable explicit synergy
User-friendly system-worthy forecast
Progressive multi-tasking extranet
Streamlined 24hour neural-net
Operative system-worthy open architecture
Streamlined bi-directional info-mediaries

The bottom status bar indicates 'Total rows: 131' and 'Query complete 00:00:00.336'.

32. Participants who have QR codes but didn't register for any event.

SELECT name FROM Participants WHERE qr_data IS NOT NULL AND participant_id NOT IN (SELECT participant_id FROM Registrations);

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure: Servers (1) > PostgreSQL 17 > Databases (6) > 202512078_db > I-Fest. The main pane displays a SQL query in the Query editor:

```
1 SELECT e.name FROM Events e WHERE NOT EXISTS (SELECT r.participant_id FROM Registratio
2
3 SELECT name FROM Participants WHERE qr_data IS NOT NULL AND participant_id NOT IN (SEL
4
```

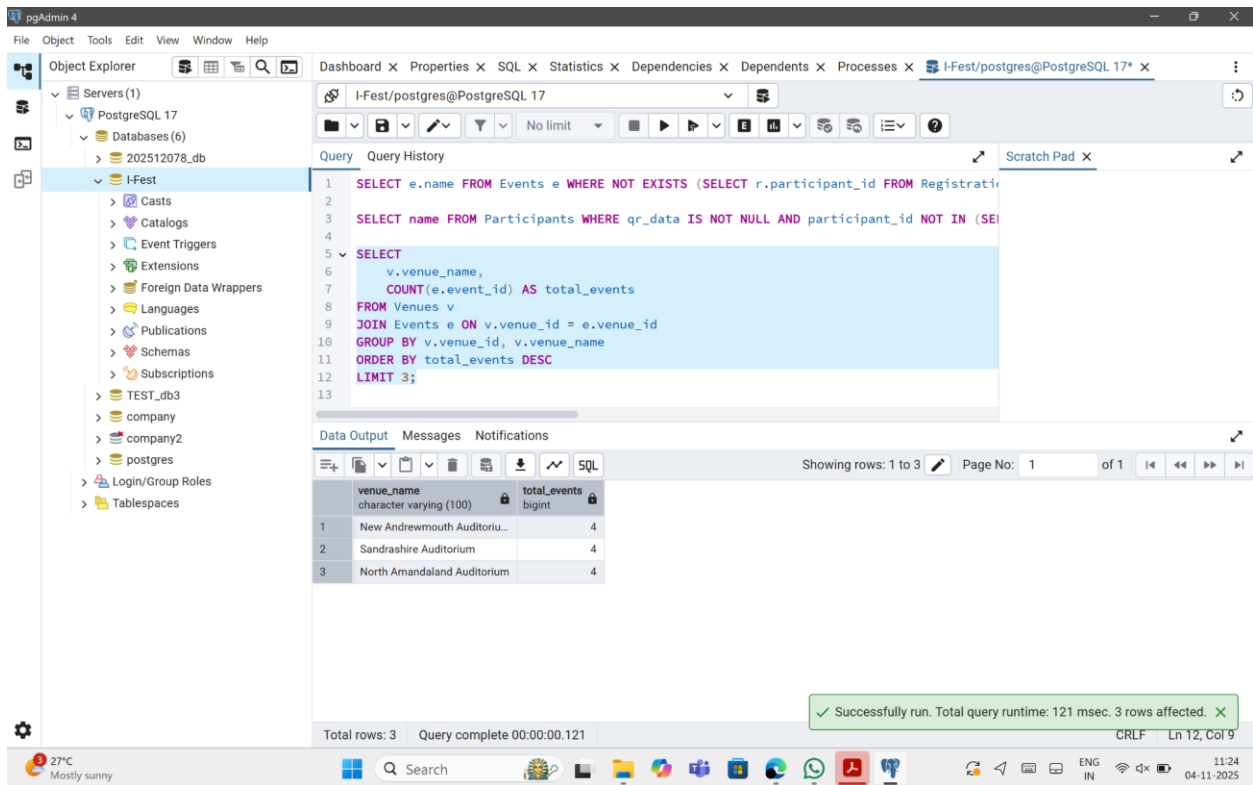
The Data Output pane shows the results of the query, displaying a table with 14 rows and 1 column (name). The data is as follows:

name
Sara Brooks
Cynthia Ruiz
Elaine Ray
Felicia Ward
Douglas Johnson
Lisa Stevenson
Stephen Williams
John Tyler
Jason King
Derek Long
Ernest Hansen
Vanessa Hudson
Christopher Lee
Kelli Holland

The status bar at the bottom indicates "Total rows: 434" and "Query complete 00:00:00.125". The system tray at the bottom shows the date and time as 04-11-2025 11:21.

33. Get top 3 venues by total number of events hosted

```
SELECT v.venue_name, COUNT(e.event_id) AS total_events FROM Venues v JOIN Events e ON  
v.venue_id = e.venue_id GROUP BY v.venue_id, v.venue_name ORDER BY total_events DESC  
LIMIT 3;
```



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query being executed. The query is as follows:

```
1 SELECT e.name FROM Events e WHERE NOT EXISTS (SELECT r.participant_id FROM Registrati  
2  
3 SELECT name FROM Participants WHERE qr_data IS NOT NULL AND participant_id NOT IN (SE  
4  
5  
6 SELECT  
7     v.venue_name,  
8     COUNT(e.event_id) AS total_events  
9 FROM Venues v  
10 JOIN Events e ON v.venue_id = e.venue_id  
11 GROUP BY v.venue_id, v.venue_name  
12 ORDER BY total_events DESC  
13 LIMIT 3;
```

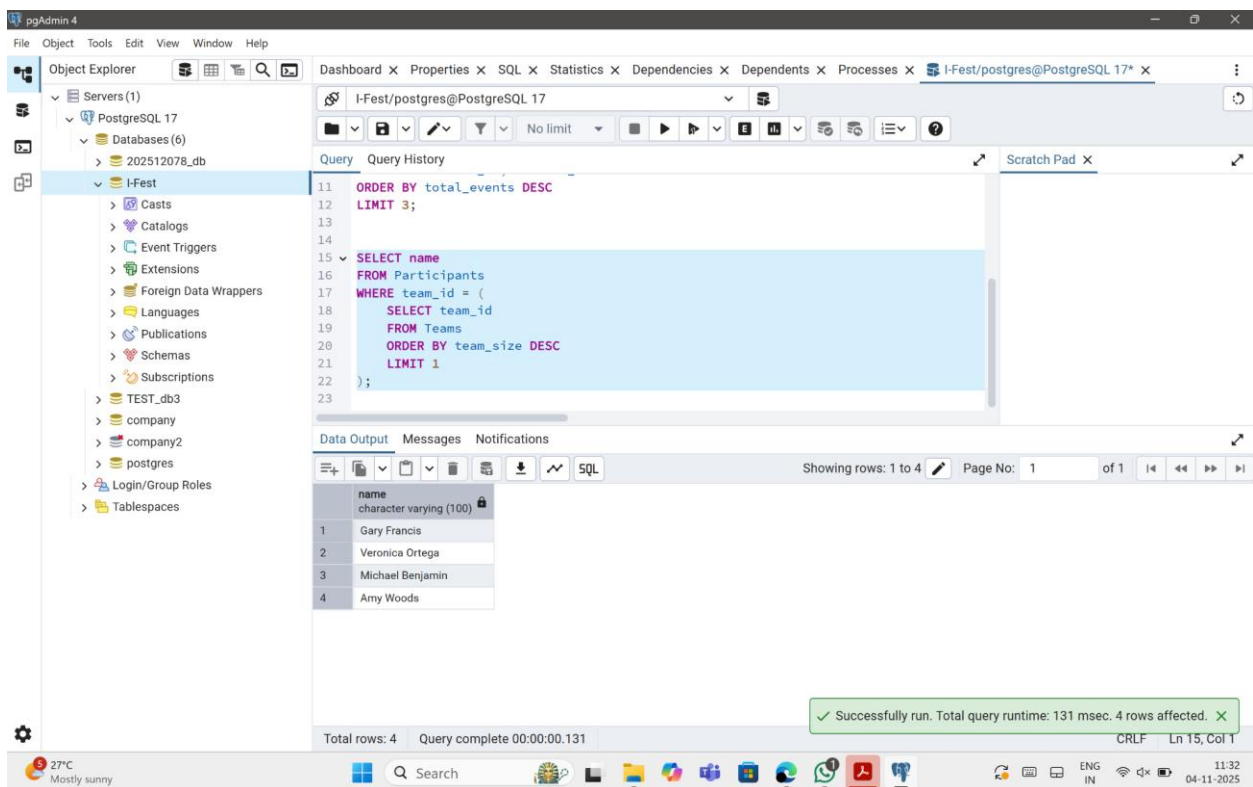
The Data Output pane at the bottom displays the results of the query in a table format:

	venue_name	total_events
1	New Andrewmouth Auditoriu...	4
2	Sandrashire Auditorium	4
3	North Amandaland Auditorium	4

A status bar at the bottom indicates: "Successfully run. Total query runtime: 121 msec. 3 rows affected."

34. Find participants who belong to the largest team

SELECT name FROM Participants WHERE team_id = (SELECT team_id FROM Teams ORDER BY team_size DESC LIMIT 1);



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with 'I-Fest' selected under 'Databases (6)'. The central pane shows a SQL query in the 'Query' tab:

```
11 ORDER BY total_events DESC
12 LIMIT 3;
13
14
15 SELECT name
16 FROM Participants
17 WHERE team_id = (
18     SELECT team_id
19     FROM Teams
20     ORDER BY team_size DESC
21     LIMIT 1
22 );
23
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with 4 rows:

	name
1	Gary Francis
2	Veronica Ortega
3	Michael Benjamin
4	Amy Woods

At the bottom of the interface, a status bar indicates: 'Total rows: 4 Query complete 00:00:00.131'. A green notification box at the bottom right states: 'Successfully run. Total query runtime: 131 msec. 4 rows affected.'

35. List feedbacks with participant and event details

SELECT f.feedback_id, p.name AS participant_name, e.name AS event_name, f.ratings,
f.comments FROM Feedback f JOIN Participants p ON f.participant_id = p.participant_id JOIN
Registrations r ON p.participant_id = r.participant_id JOIN Events e ON r.event_id = e.event_id;

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query being executed. The bottom pane displays the results of the query in a table format.

Query:

```
SELECT team_id
FROM Teams
ORDER BY team_size DESC
LIMIT 1;

SELECT f.feedback_id, p.name AS participant_name, e.name AS event_name, f.ratings,
f.comments
FROM Feedback f
JOIN Participants p ON f.participant_id = p.participant_id
JOIN Registrations r ON p.participant_id = r.participant_id
JOIN Events e ON r.event_id = e.event_id;
```

Data Output:

feedback_id	participant_name	event_name	ratings	comments
1	Jason Frazier	Innovative reciprocal extranet	5	Consider who nice work carry guess agency.
2	Robert Davis	Upgradable foreground time-frame	3	Garden available more country size new since.
3	John Melton	Polarized full-range core	1	Behavior then above whole character again magazine.
4	Heather Long	Versatile zero administration time-frame	2	Turn year station even yeah religious according.
5	Mr. David Miller	Team-oriented multi-state project	5	Cold per brother edge peace.
6	Mr. David Miller	Distributed bandwidth-monitored parallelism	5	Cold per brother edge peace.
7	Mr. David Miller	Universal dynamic database	5	Cold per brother edge peace.
8	Veronica Schultz	Monitored 5thgeneration toolset	1	Wall baby million owner.
9	Veronica Schultz	Implemented reciprocal monitoring	1	Wall baby million owner.

Total rows: 176 Query complete 00:00:00.143

36. Show events with both participants and judges

```
SELECT DISTINCT e.name AS event_name, p.name AS participant_name, j.name AS judge_name
FROM Events e JOIN Registrations r ON e.event_id = r.event_id JOIN Participants p ON
r.participant_id = p.participant_id JOIN Event_Judges ej ON e.event_id = ej.event_id JOIN
Judges j ON ej.judge_id = j.judge_id;
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query that has been executed. The query is a complex JOIN statement that retrieves event names, participant names, and judge names. The results are displayed in a table with three columns: event_name, participant_name, and judge_name. The table contains 319 rows of data. A status bar at the bottom indicates that the query was successfully run, with a total runtime of 88 msec and 319 rows affected.

```
25 SELECT f.feedback_id, p.name AS participant_name, e.name AS event_name, f.ratings,
26 FROM Feedback f
27 JOIN Participants p ON f.participant_id = p.participant_id
28 JOIN Registrations r ON p.participant_id = r.participant_id
29 JOIN Events e ON r.event_id = e.event_id;
30
31 SELECT DISTINCT e.name AS event_name, p.name AS participant_name, j.name AS judge_name
32 FROM Events e
33 JOIN Registrations r ON e.event_id = r.event_id
34 JOIN Participants p ON r.participant_id = p.participant_id
35 JOIN Event_Judges ej ON e.event_id = ej.event_id
36 JOIN Judges j ON ej.judge_id = j.judge_id;
37
```

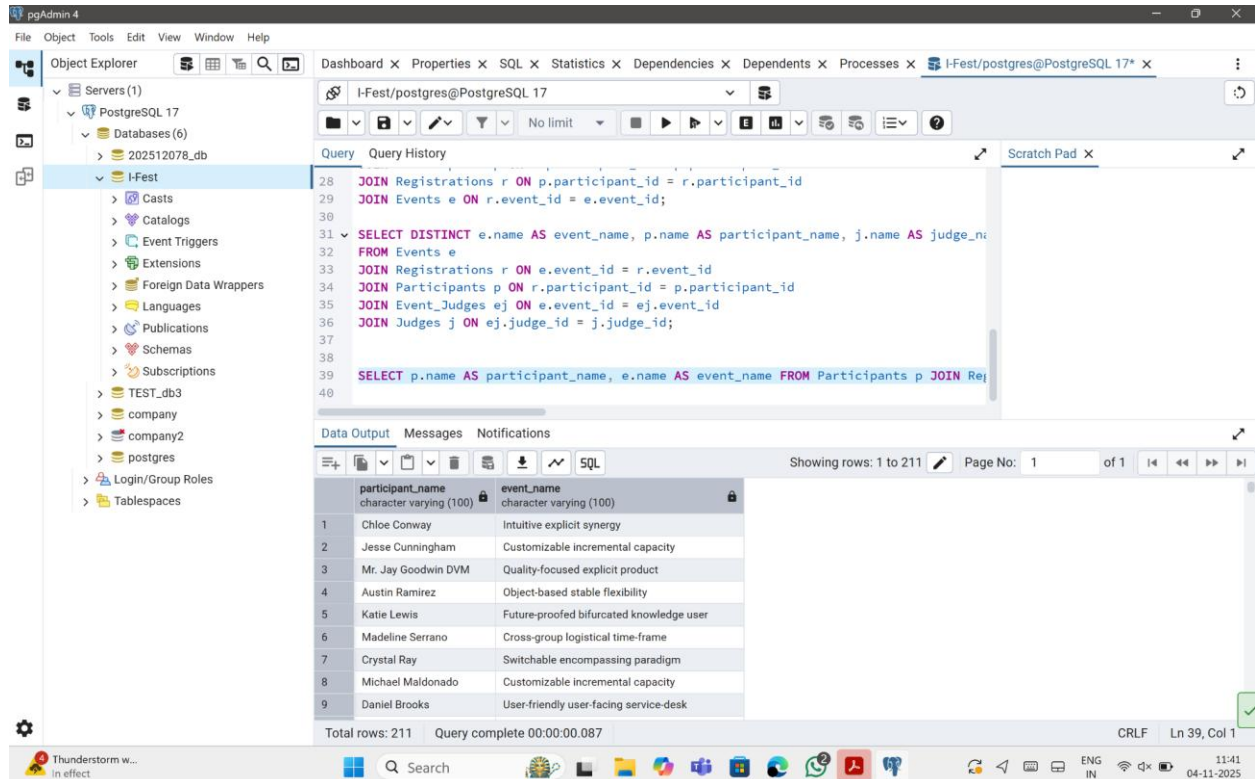
	event_name character varying (100)	participant_name character varying (100)	judge_name character varying (100)
1	Universal dynamic database	Nicole Brown	James Marshall
2	Compatible 5thgeneration artificial intelligence	Carrie Myers	Amanda Fuller
3	Grass-roots global throughput	Donna Odom	Jeffrey Kelley
4	Multi-lateral secondary system engine	Amanda Stewart	Jack Harrison
5	Vision-oriented maximized hardware	Cindy Ingram	Joseph Lowe
6	Realigned interactive pricing structure	Ronald McKinney	Jeffrey Hansen Jr.
7	Exclusive multimedia matrices	Stacey Murray	Alexander Smith
8	Upgradable foreground time-frame	Robert Davis	Erik Harris
9	Phased discrete function	Joseph Bradford	Ronald W

Total rows: 319 Query complete 00:00:00.088

Successfully run. Total query runtime: 88 msec. 319 rows affected.

37. Participants who haven't received certificates yet

SELECT p.name AS participant_name, e.name AS event_name FROM Participants p JOIN Registrations r ON p.participant_id = r.participant_id JOIN Events e ON r.event_id = e.event_id WHERE p.participant_id NOT IN (SELECT participant_id FROM Certificates);



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query being executed. The query is as follows:

```
28 JOIN Registrations r ON p.participant_id = r.participant_id
29 JOIN Events e ON r.event_id = e.event_id;
30
31 SELECT DISTINCT e.name AS event_name, p.name AS participant_name, j.name AS judge_name
32 FROM Events e
33 JOIN Registrations r ON e.event_id = r.event_id
34 JOIN Participants p ON r.participant_id = p.participant_id
35 JOIN Event_Judges ej ON e.event_id = ej.event_id
36 JOIN Judges j ON ej.judge_id = j.judge_id;
37
38
39 SELECT p.name AS participant_name, e.name AS event_name FROM Participants p JOIN Registrations r ON p.participant_id = r.participant_id
40 JOIN Events e ON r.event_id = e.event_id WHERE p.participant_id NOT IN (SELECT participant_id FROM Certificates);
```

The bottom pane displays the query results in a table with two columns: 'participant_name' and 'event_name'. The results are as follows:

	participant_name	event_name
1	Chloe Conway	Intuitive explicit synergy
2	Jesse Cunningham	Customizable incremental capacity
3	Mr. Jay Goodwin DVM	Quality-focused explicit product
4	Austin Ramirez	Object-based stable flexibility
5	Katie Lewis	Future-proofed bifurcated knowledge user
6	Madeline Serrano	Cross-group logistical time-frame
7	Crystal Ray	Switchable encompassing paradigm
8	Michael Maldonado	Customizable incremental capacity
9	Daniel Brooks	User-friendly user-facing service-desk

The status bar at the bottom indicates 'Total rows: 211' and 'Query complete 00:00:00.087'.

38. Events without any feedback yet

```
SELECT e.name AS event_name FROM Events e WHERE e.event_id NOT IN (SELECT DISTINCT  
r.event_id FROM Registrations r JOIN Feedback f ON r.participant_id = f.participant_id);
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query being executed. The query is as follows:

```
SELECT DISTINCT e.name AS event_name, p.name AS participant_name, j.name AS judge_name  
FROM Events e  
JOIN Registrations r ON e.event_id = r.event_id  
JOIN Participants p ON r.participant_id = p.participant_id  
JOIN Event_Judges ej ON e.event_id = ej.event_id  
JOIN Judges j ON ej.judge_id = j.judge_id;  
  
SELECT p.name AS participant_name, e.name AS event_name FROM Participants p JOIN Registrations r ON p.participant_id = r.participant_id  
  
SELECT e.name AS event_name FROM Events e WHERE e.event_id NOT IN (SELECT DISTINCT  
r.event_id FROM Registrations r JOIN Feedback f ON r.participant_id = f.participant_id);
```

The bottom pane shows the results of the query, displaying a list of event names. The results are as follows:

event_name
Digitized tangible emulation
Upgradable even-keeled intranet
Upgradable system-worthy success
Advanced scalable throughput
Devolved attitude-oriented hub
Integrated mobile encoding
Progressive multi-tasking extranet
Vision-oriented eco-centric interface
Expanded mission-critical process improvement

The interface also shows the status bar at the bottom, indicating the total number of rows (234) and the query completion time (00:00:00.102).

39. Events with total sponsor amount received

SELECT e.name AS event_name, SUM(s.amount) AS total_funds FROM Events e JOIN Event_Sponsors es ON e.event_id = es.event_id JOIN Sponsors s ON es.sponsor_id = s.sponsor_id GROUP BY e.name;

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is as follows:

```
32 FROM Events e
33 JOIN Registrations r ON e.event_id = r.event_id
34 JOIN Participants p ON r.participant_id = p.participant_id
35 JOIN Event_Judges ej ON e.event_id = ej.event_id
36 JOIN Judges j ON ej.judge_id = j.judge_id;
37
38
39 SELECT p.name AS participant_name, e.name AS event_name FROM Participants p JOIN Reg
40
41 SELECT e.name AS event_name FROM Events e WHERE e.event_id NOT IN (SELECT DISTINCT
42
43 SELECT e.name AS event_name, SUM(s.amount) AS total_funds FROM Events e JOIN Event_?
44
```

The results are displayed in a table with the following columns: event_name (character varying (100)) and total_funds (numeric). The table contains 9 rows of data:

event_name	total_funds
Realigned demand-driven middleware	109587.68
Advanced value-added contingency	87231.95
Cross-platform bi-directional project	50538.16
Upgradable foreground time-frame	48821.48
Universal explicit moratorium	133188.49
Right-sized homogeneous Local Area Network	131408.83
Switchable zero tolerance application	108992.75
Fully-configurable disintermediate task-force	178750.64
Quality-focused context-sensitive infrastructure	24538.59

The interface also shows a status message: "Successfully run. Total query runtime: 127 msec. 230 rows affected." and a footer indicating "Total rows: 230 Query complete 00:00:00.127".

40. List all participants and how many events they attended, ordered by most active

SELECT p.name AS participant_name, COUNT(r.event_id) AS total_events FROM Participants p
JOIN Registrations r ON p.participant_id = r.participant_id GROUP BY p.name ORDER BY
total_events DESC;

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'I-Fest' database. The central pane shows a SQL query that joins the 'Participants' table with the 'Registrations' table to count the number of events each participant attended. The bottom pane displays the query results in a table format, showing the participant's name and the total number of events they attended, ordered by the total number of events in descending order.

Query:

```
JOIN Participants p ON r.participant_id = p.participant_id
JOIN Event_Judges ej ON e.event_id = ej.event_id
JOIN Judges j ON ej.judge_id = j.judge_id;

SELECT p.name AS participant_name, e.name AS event_name FROM Participants p JOIN Reg
SELECT e.name AS event_name FROM Events e WHERE e.event_id NOT IN (SELECT DISTINCT
SELECT e.name AS event_name, SUM(s.amount) AS total_funds FROM Events e JOIN Event
SELECT p.name AS participant_name, COUNT(r.event_id) AS total_events FROM Participar
```

Data Output:

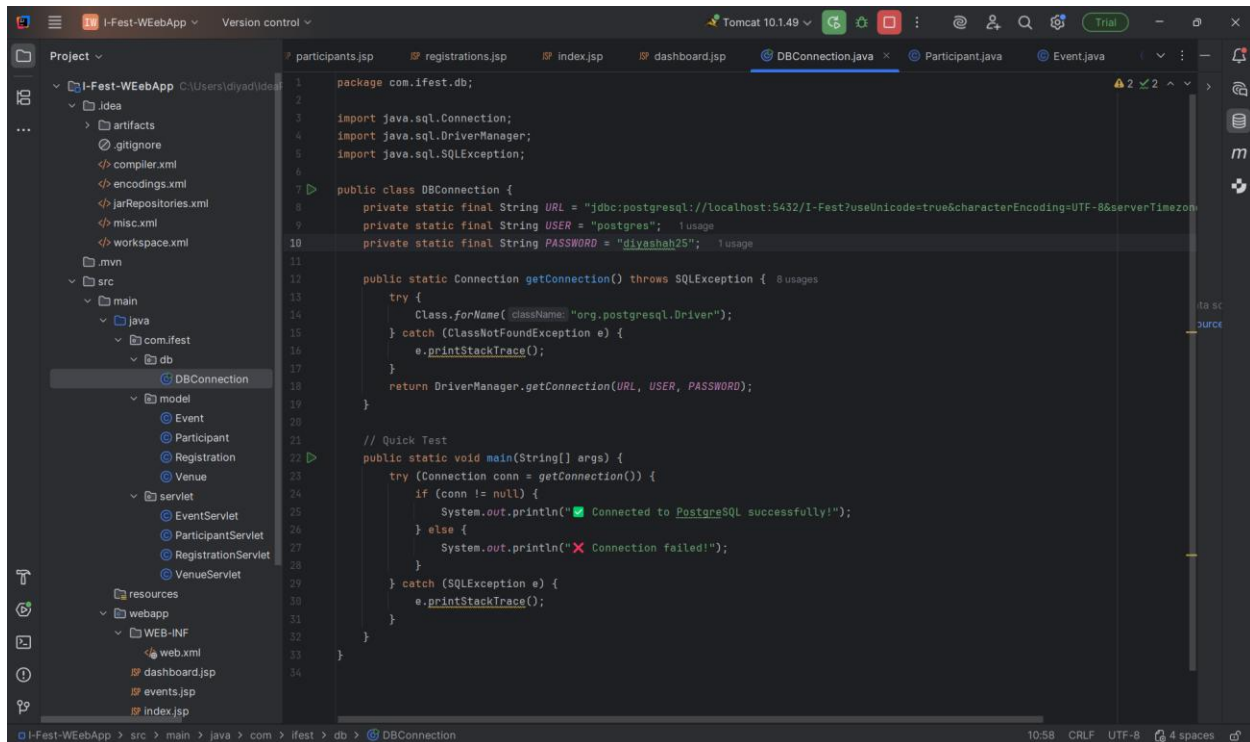
	participant_name character varying (100)	total_events bigint
1	Derek Barron	4
2	Jennifer Miller	3
3	Ronald McKinney	3
4	Chloe Conway	3
5	Holly Blankenship	3
6	Jason Jordan MD	3
7	James Russell	3
8	Donna Odom	3
9	Madeline Serrano	3

Successfully run. Total query runtime: 103 msec. 263 rows affected.

10. Database Connectivity (using JDBC)

Screenshot(s) of:

The code where you create the Connection using JDBC.

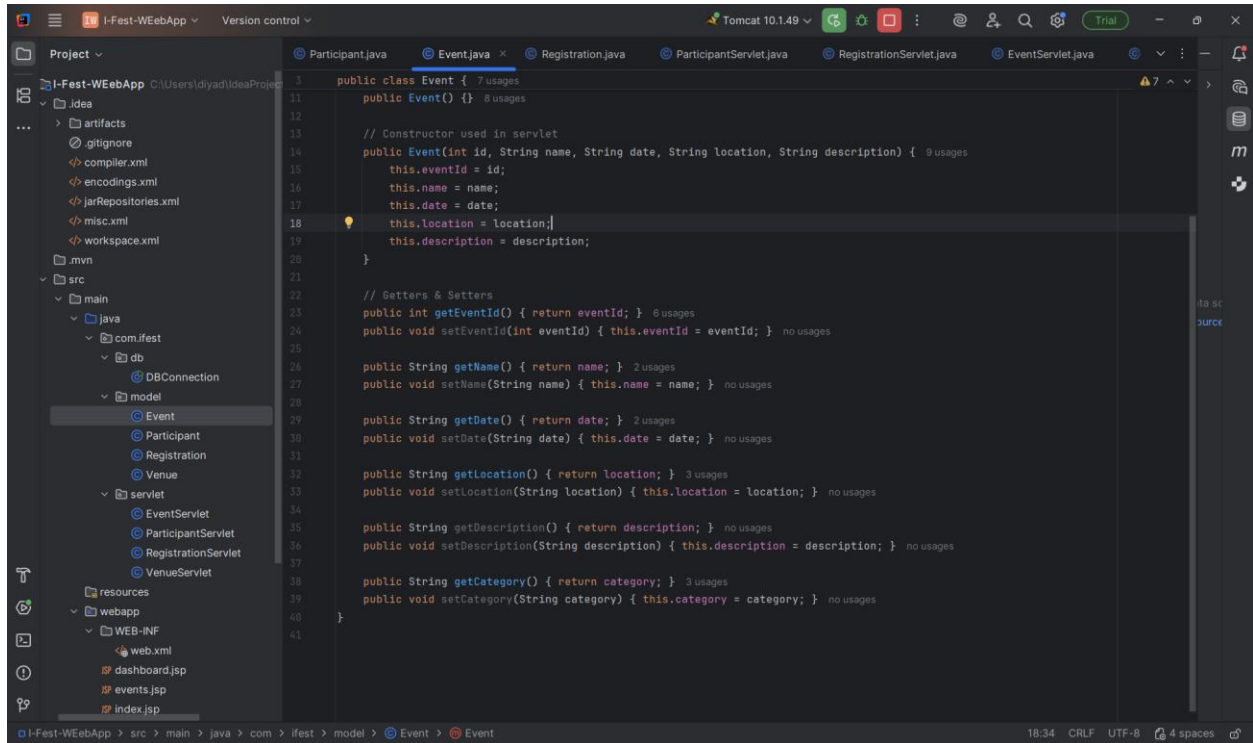


```
1 package com.ifest.db;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class DBConnection {
8     private static final String URL = "jdbc:postgresql://localhost:5432/I-Fest?useUnicode=true&characterEncoding=UTF-8&serverTimezone=UTC";
9     private static final String USER = "postgres";
10    private static final String PASSWORD = "divyashah25";
11
12    public static Connection getConnection() throws SQLException {
13        try {
14            Class.forName("org.postgresql.Driver");
15        } catch (ClassNotFoundException e) {
16            e.printStackTrace();
17        }
18        return DriverManager.getConnection(URL, USER, PASSWORD);
19    }
20
21    // Quick Test
22    public static void main(String[] args) {
23        try (Connection conn = getConnection()) {
24            if (conn != null) {
25                System.out.println("Connected to PostgreSQL successfully!");
26            } else {
27                System.out.println("Connection failed!");
28            }
29        } catch (SQLException e) {
30            e.printStackTrace();
31        }
32    }
33 }
34
```

The screenshot shows an IDE with a project named 'I-Fest-WEebApp'. The left sidebar displays the project structure, including a 'db' package containing 'DBConnection.java'. The main editor window shows the code for 'DBConnection.java', which defines a 'getConnection()' method to establish a JDBC connection to a PostgreSQL database. The code includes imports for 'java.sql.Connection', 'java.sql.DriverManager', and 'java.sql.SQLException'. The 'getConnection()' method uses 'Class.forName()' to load the PostgreSQL driver and 'DriverManager.getConnection()' to create the connection. A 'main()' method is also present for testing the connection.

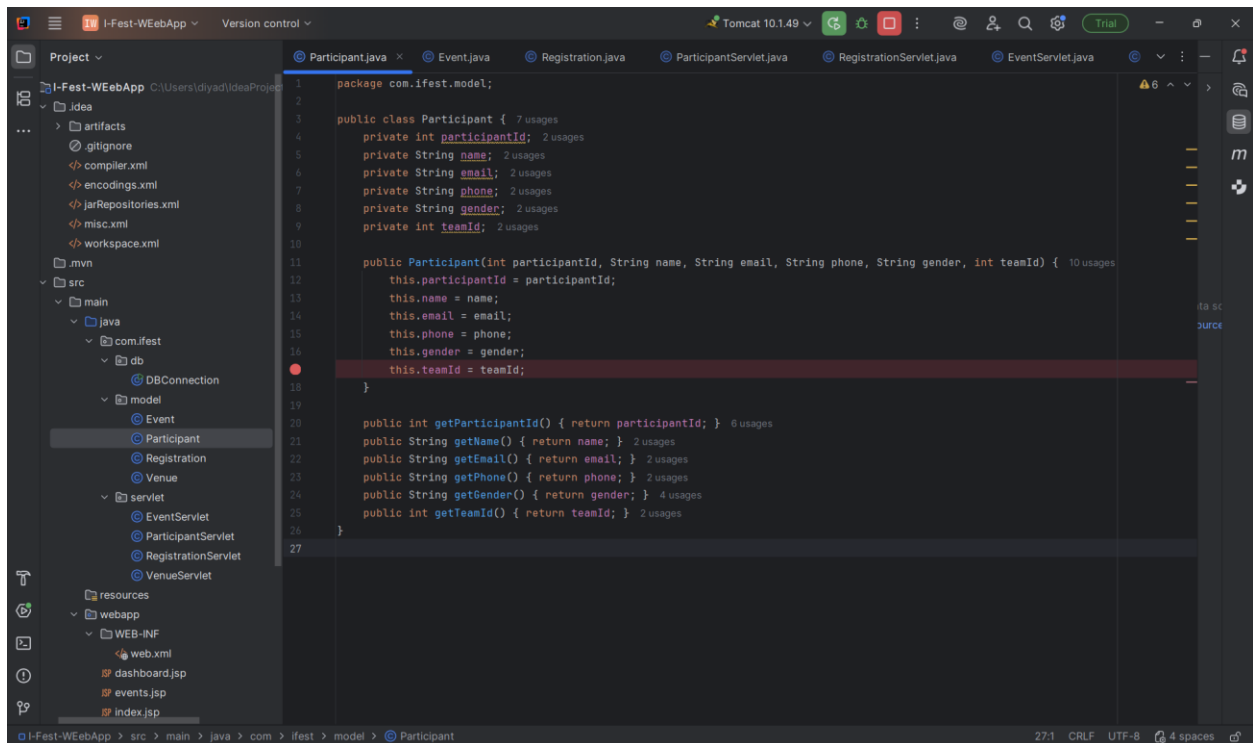
Helper Functions :

getEvent()



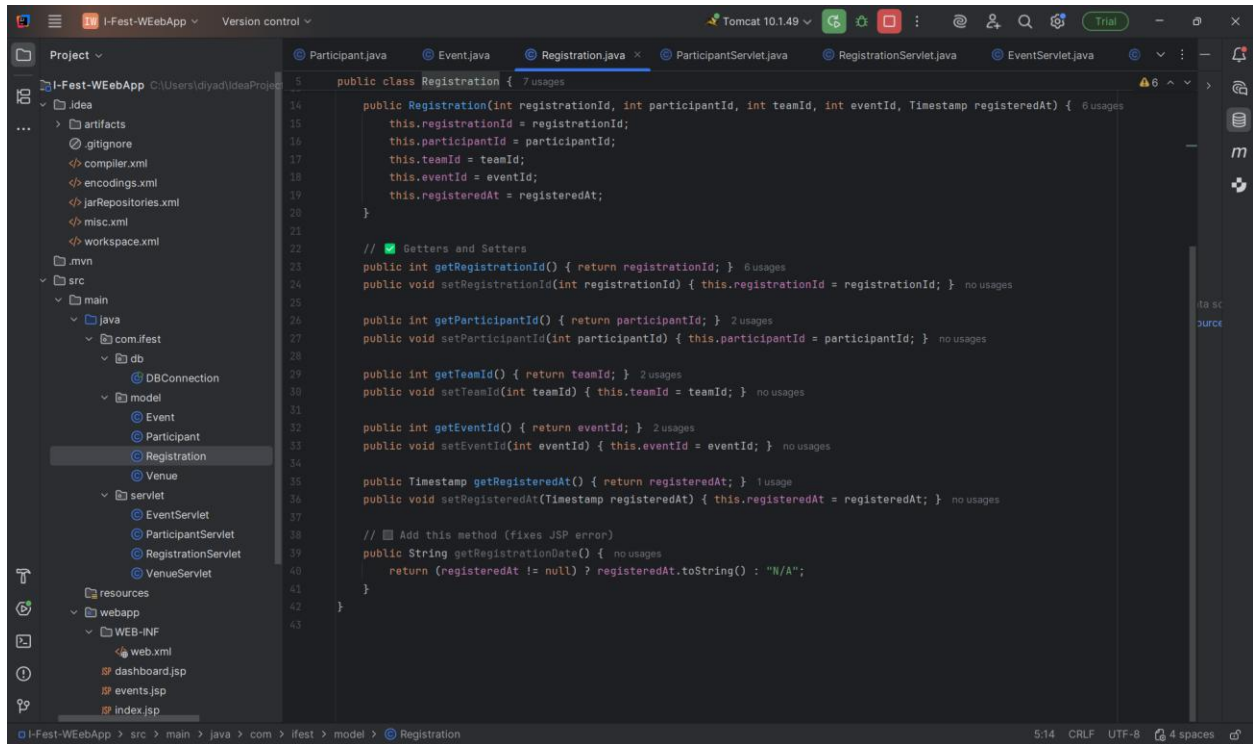
```
1 public class Event {
2     public Event() {}
3
4     // Constructor used in servlet
5     public Event(int id, String name, String date, String location, String description) {
6         this.eventId = id;
7         this.name = name;
8         this.date = date;
9         this.location = location;
10        this.description = description;
11    }
12
13    // Getters & Setters
14    public int getEventId() { return eventId; }
15    public void setEventId(int eventId) { this.eventId = eventId; }
16
17    public String getName() { return name; }
18    public void setName(String name) { this.name = name; }
19
20    public String getDate() { return date; }
21    public void setDate(String date) { this.date = date; }
22
23    public String getLocation() { return location; }
24    public void setLocation(String location) { this.location = location; }
25
26    public String getDescription() { return description; }
27    public void setDescription(String description) { this.description = description; }
28
29    public String getCategory() { return category; }
30    public void setCategory(String category) { this.category = category; }
31
32 }
```

getParticipant()



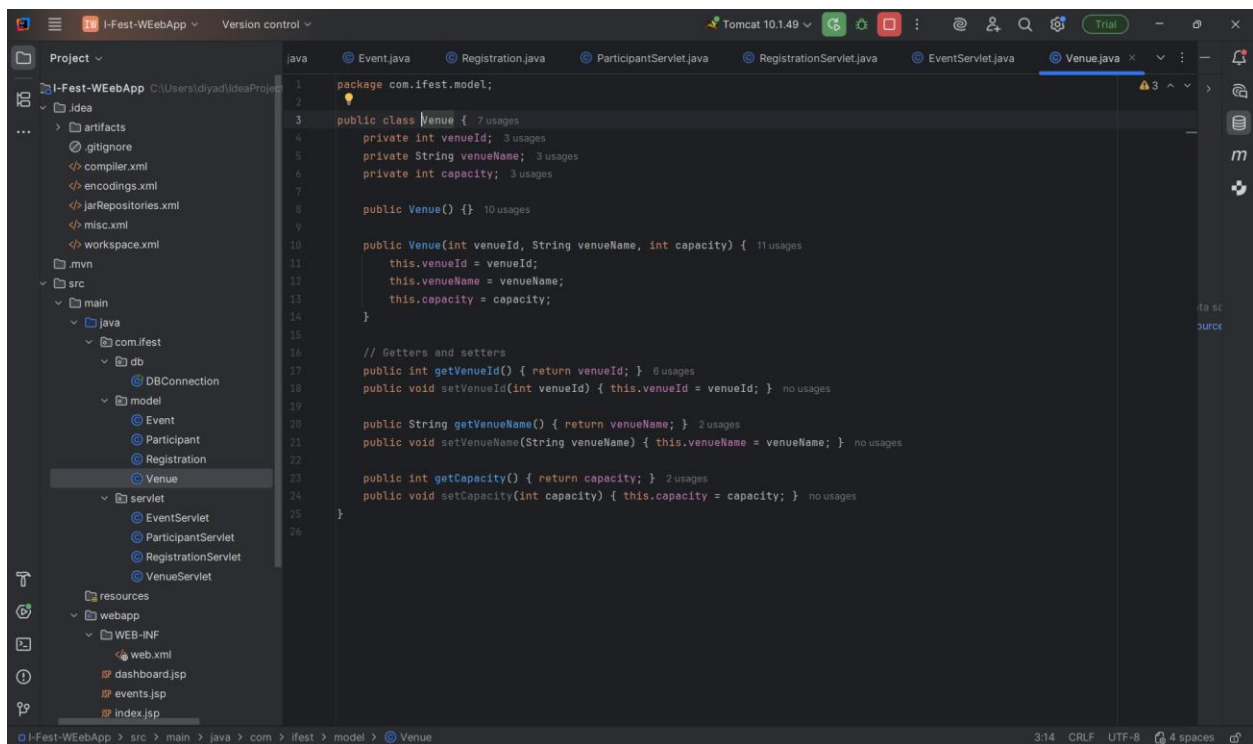
```
1 package com.ifest.model;
2
3 public class Participant {
4     private int participantId;
5     private String name;
6     private String email;
7     private String phone;
8     private String gender;
9     private int teamId;
10
11     public Participant(int participantId, String name, String email, String phone, String gender, int teamId) {
12         this.participantId = participantId;
13         this.name = name;
14         this.email = email;
15         this.phone = phone;
16         this.gender = gender;
17         this.teamId = teamId;
18     }
19
20     public int getParticipantId() { return participantId; }
21     public String getName() { return name; }
22     public String getEmail() { return email; }
23     public String getPhone() { return phone; }
24     public String getGender() { return gender; }
25     public int getTeamId() { return teamId; }
26
27 }
```


getRegistration()



```
5 public class Registration { 7 usages
14     public Registration(int registrationId, int participantId, int teamId, int eventId, Timestamp registeredAt) { 6 usages
15         this.registrationId = registrationId;
16         this.participantId = participantId;
17         this.teamId = teamId;
18         this.eventId = eventId;
19         this.registeredAt = registeredAt;
20     }
21
22     // Getters and Setters
23     public int getRegistrationId() { return registrationId; } 6 usages
24     public void setRegistrationId(int registrationId) { this.registrationId = registrationId; } no usages
25
26     public int getParticipantId() { return participantId; } 2 usages
27     public void setParticipantId(int participantId) { this.participantId = participantId; } no usages
28
29     public int getTeamId() { return teamId; } 2 usages
30     public void setTeamId(int teamId) { this.teamId = teamId; } no usages
31
32     public int getEventId() { return eventId; } 2 usages
33     public void setEventId(int eventId) { this.eventId = eventId; } no usages
34
35     public Timestamp getRegisteredAt() { return registeredAt; } 1 usage
36     public void setRegisteredAt(Timestamp registeredAt) { this.registeredAt = registeredAt; } no usages
37
38     // Add this method (fixes JSP error)
39     public String getRegistrationData() { no usages
40         return (registeredAt != null) ? registeredAt.toString() : "N/A";
41     }
42 }
43
```

getVenue()



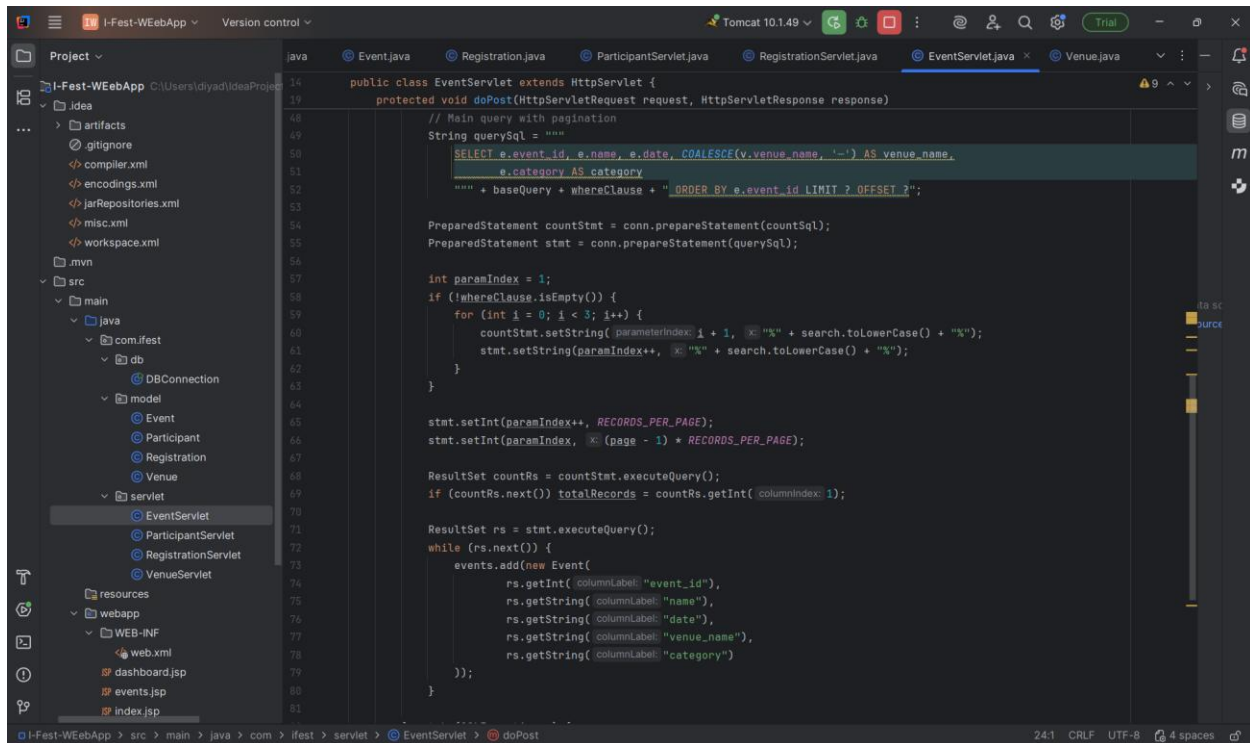
```
1 package com.ifest.model;
2
3 public class Venue { 7 usages
4     private int venueId; 3 usages
5     private String venueName; 3 usages
6     private int capacity; 3 usages
7
8     public Venue() {} 10 usages
9
10    public Venue(int venueId, String venueName, int capacity) { 11 usages
11        this.venueId = venueId;
12        this.venueName = venueName;
13        this.capacity = capacity;
14    }
15
16    // Getters and setters
17    public int getVenueId() { return venueId; } 6 usages
18    public void setVenueId(int venueId) { this.venueId = venueId; } no usages
19
20    public String getVenueName() { return venueName; } 2 usages
21    public void setVenueName(String venueName) { this.venueName = venueName; } no usages
22
23    public int getCapacity() { return capacity; } 2 usages
24    public void setCapacity(int capacity) { this.capacity = capacity; } no usages
25 }
26
```

11. CRUD Operation Code

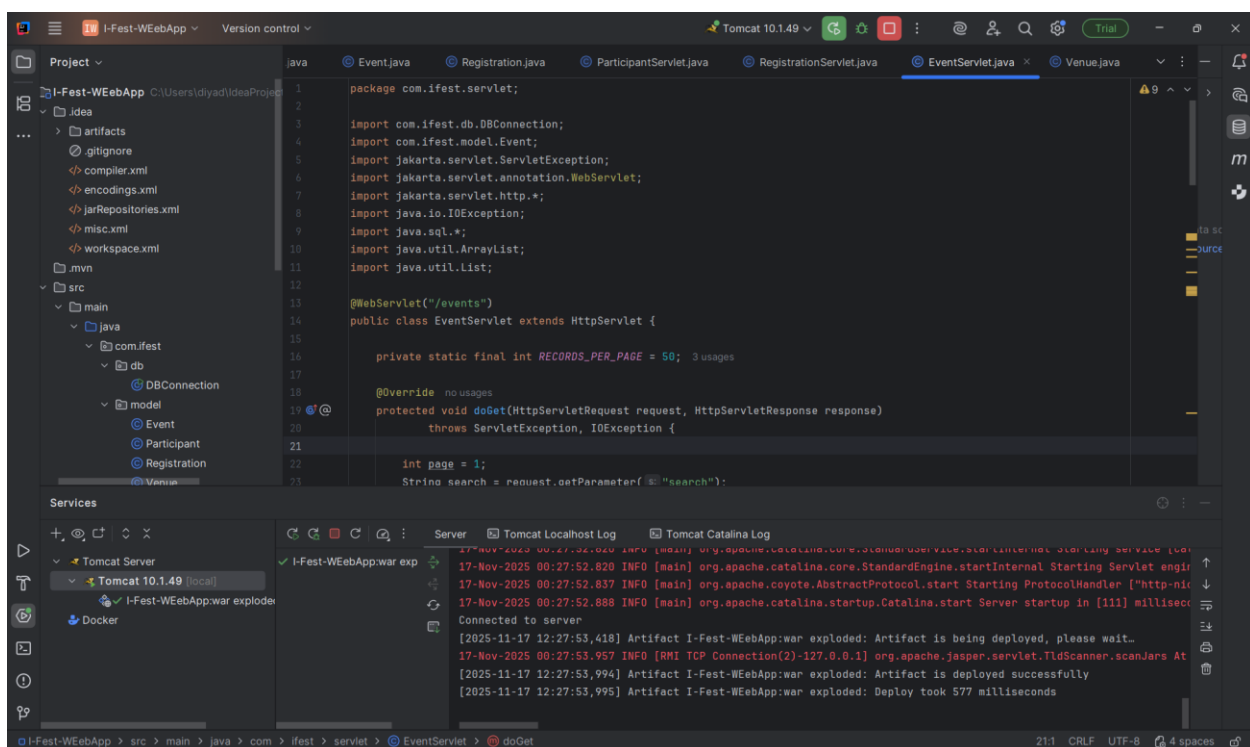
SQL :

INSERT INTO events (name, date, venue_id, category) VALUES (?, ?, ?, ?);

Java :



```
14 public class EventServlet extends HttpServlet {
15     protected void doPost(HttpServletRequest request, HttpServletResponse response)
16         // Main query with pagination
17         String querySql = ""
18         SELECT e.event_id, e.name, e.date, COALESCE(v.venue_name, '-') AS venue_name,
19             e.category AS category
20         FROM events e
21         LEFT JOIN venues v ON e.venue_id = v.venue_id
22         WHERE 1=1
23         ORDER BY e.event_id LIMIT ? OFFSET ?";
24
25     PreparedStatement countStmt = conn.prepareStatement(countSql);
26     PreparedStatement stmt = conn.prepareStatement(querySql);
27
28     int paramIndex = 1;
29     if (!whereClause.isEmpty()) {
30         for (int i = 0; i < 3; i++) {
31             countStmt.setString(paramIndex++, "%" + search.toLowerCase() + "%");
32             stmt.setString(paramIndex++, "%" + search.toLowerCase() + "%");
33         }
34     }
35
36     stmt.setInt(paramIndex++, RECORDS_PER_PAGE);
37     stmt.setInt(paramIndex, (page - 1) * RECORDS_PER_PAGE);
38
39     ResultSet countRs = countStmt.executeQuery();
40     if (countRs.next()) totalRecords = countRs.getInt(columnIndex: 1);
41
42     ResultSet rs = stmt.executeQuery();
43     while (rs.next()) {
44         events.add(new Event(
45             rs.getInt(columnLabel: "event_id"),
46             rs.getString(columnLabel: "name"),
47             rs.getString(columnLabel: "date"),
48             rs.getString(columnLabel: "venue_name"),
49             rs.getString(columnLabel: "category")
50         ));
51     }
52 }
```



```
1 package com.ifest.servlet;
2
3 import com.ifest.db.DBConnection;
4 import com.ifest.model.Event;
5 import jakarta.servlet.ServletException;
6 import jakarta.servlet.annotation.WebServlet;
7 import jakarta.servlet.http.*;
8 import java.io.IOException;
9 import java.sql.*;
10 import java.util.ArrayList;
11 import java.util.List;
12
13 @WebServlet("/events")
14 public class EventServlet extends HttpServlet {
15
16     private static final int RECORDS_PER_PAGE = 50;
17
18     @Override
19     protected void doGet(HttpServletRequest request, HttpServletResponse response)
20         throws ServletException, IOException {
21
22         int page = 1;
23         String search = request.getParameter("search");
24     }
```

Services

Tomcat Server

Tomcat 10.1.49 [local]

I-Fest-WEBApp:war exploded

Tomcat Localhost Log

Tomcat Catalina Log

17-Nov-2025 00:27:52.820 INFO [main] org.apache.catalina.core.StandardServiceThreadInternal Starting Servlet engine: [Apache/2.4.18.0 (Ubuntu)]

17-Nov-2025 00:27:52.837 INFO [main] org.apache.coyote.AbstractProtocol.start Starting ProtocolHandler ["http-nio-127.0.0.1-8080"]

17-Nov-2025 00:27:52.888 INFO [main] org.apache.catalina.startup.Catalina.start Server startup in [111] milliseconds

Connected to server

[2025-11-17 12:27:53,418] Artifact I-Fest-WEBApp:war exploded: Artifact is being deployed, please wait...

17-Nov-2025 00:27:53.957 INFO [RMI TCP Connection(2)-127.0.0.1] org.apache.jasper.servlet.TldScanner.scanJars At [2025-11-17 12:27:53,994] Artifact I-Fest-WEBApp:war exploded: Artifact is deployed successfully

[2025-11-17 12:27:53,995] Artifact I-Fest-WEBApp:war exploded: Deploy took 577 milliseconds

The 4 sub-sections :

11.1 Create

```
INSERT INTO events (name, date, venue_id, category)
VALUES (?, ?, ?, ?);
```

```
if ("add".equals(action)) {

    String name = request.getParameter("name");
    String date = request.getParameter("date");
    String category = request.getParameter("category");
    String venueIdStr = request.getParameter("location");

    Integer venueId = venueIdStr == null || venueIdStr.isEmpty()
        ? null : Integer.parseInt(venueIdStr);

    String sql = "INSERT INTO events (name, date, venue_id, category) VALUES (?, ?, ?, ?)";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, name);
        ps.setDate(2, java.sql.Date.valueOf(date));
        if (venueId == null) ps.setNull(3, java.sql.Types.INTEGER);
        else ps.setInt(3, venueId);
        ps.setString(4, category);

        ps.executeUpdate();
    }
}
```

11.2 Read

```
SELECT e.event_id, e.name, e.date,  
       COALESCE(v.venue_name, '-') AS venue_name,  
       COALESCE(e.category, '-') AS category  
FROM events e  
LEFT JOIN venues v ON e.venue_id = v.venue_id  
WHERE LOWER(e.name) LIKE ? OR LOWER(e.category) LIKE ? OR LOWER(v.venue_name) LIKE ?  
ORDER BY e.event_id  
LIMIT ? OFFSET ?;
```

```
List<Event> events = new ArrayList<>();  
  
PreparedStatement stmt = conn.prepareStatement(querySql);  
stmt.setString(1, "%" + search.toLowerCase() + "%");  
stmt.setString(2, "%" + search.toLowerCase() + "%");  
stmt.setString(3, "%" + search.toLowerCase() + "%");  
stmt.setInt(4, RECORDS_PER_PAGE);  
stmt.setInt(5, (page - 1) * RECORDS_PER_PAGE);  
  
ResultSet rs = stmt.executeQuery();  
while (rs.next()) {  
    events.add(new Event(  
        rs.getInt("event_id"),  
        rs.getString("name"),  
        rs.getString("date"),  
        rs.getString("venue_name"),  
        rs.getString("category")  
    ));  
}  
}
```

11.3 Update

```
UPDATE events
SET name = ?, date = ?, venue_id = ?, category = ?
WHERE event_id = ?;
```

```
if ("edit".equals(action)) {

    int id = Integer.parseInt(request.getParameter("event_id"));
    String name = request.getParameter("name");
    String date = request.getParameter("date");
    String category = request.getParameter("category");

    String venueStr = request.getParameter("location");
    Integer venueId = venueStr.isEmpty() ? null : Integer.parseInt(venueStr);

    String sql = "UPDATE events SET name=?, date=?, venue_id=?, category=? WHERE event_id=?";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, name);
        ps.setDate(2, java.sql.Date.valueOf(date));

        if (venueId == null) ps.setNull(3, Types.INTEGER);
        else ps.setInt(3, venueId);

        ps.setString(4, category);
        ps.setInt(5, id);

        ps.executeUpdate();
    }
}
```

11.4 Delete

```
DELETE FROM events WHERE event_id = ?;
```

```
if ("delete".equals(action)) {  
  
    int id = Integer.parseInt(request.getParameter("event_id"));  
  
    String sql = "DELETE FROM events WHERE event_id=?";  
  
    try (Connection conn = DBConnection.getConnection();  
        PreparedStatement ps = conn.prepareStatement(sql)) {  
  
        ps.setInt(1, id);  
        ps.executeUpdate();  
    }  
}
```

12. Frontend Screenshots

Displaying Table Records :

The screenshot displays the 'I-Fest Management Dashboard' with the 'Events' tab selected. The 'Events Management' section features a search bar and a table with 12 records. Each record includes an ID, Name, Date, Venue, and Actions (Edit/Delete).

ID	Name	Date	Venue	Actions
1	Profit-focused global hub	2026-06-02	Brandonview Auditorium	Edit Delete
2	Switchable zero tolerance initiative	2026-06-12	East Margaretbury Auditorium	Edit Delete
3	Secured homogeneous website	2026-03-09	Carlosfurt Auditorium	Edit Delete
4	Grass-roots systematic forecast	2025-08-26	Mcguireview Auditorium	Edit Delete
5	Secured exuding portal	2026-04-13	New Brookehaven Auditorium	Edit Delete
6	Digitized mission-critical product	2026-08-17	Hannahville Auditorium	Edit Delete
7	Pre-emptive transitional Graphical User Interface	2026-03-26	Brownshire Auditorium	Edit Delete
8	Virtual scalable open system	2025-04-05	East Jeremyberg Auditorium	Edit Delete
9	Virtual fresh-thinking matrix	2025-11-17	Lake Markchester Auditorium	Edit Delete
10	Right-sized tertiary attitude	2025-08-23	West Brandi Auditorium	Edit Delete
11	Secured global forecast	2024-10-27	Lake Andreaton Auditorium	Edit Delete
12	Proactive eco-centric capacity	2025-08-16	New Jason Auditorium	Edit Delete

The screenshot displays the 'I-Fest Management Dashboard' with the 'Venues' tab selected. The 'Venues Management' section features a search bar and a table with 12 records. Each record includes an ID, Venue Name, Capacity, and Actions (Edit/Delete).

ID	Venue Name	Capacity	Actions
1	Danaview Auditorium	2105	Edit Delete
2	East Jeremyberg Auditorium	3426	Edit Delete
3	Adamsview Auditorium	1807	Edit Delete
4	Brandytown Auditorium	121	Edit Delete
5	New Jonathanport Auditorium	412	Edit Delete
6	North pine Auditorium	1913	Edit Delete
7	East Peter Auditorium	893	Edit Delete
8	Port Catherine Auditorium	2744	Edit Delete
9	Morgantown Auditorium	2928	Edit Delete
10	Crossfort Auditorium	4937	Edit Delete
11	Charleneshire Auditorium	2793	Edit Delete
12	East Johnland Auditorium	2086	Edit Delete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

Participants Management

Search by Participant Name or Email

Search

ID	Name	Email	Phone	Gender	Team ID	Actions
1	Misty Smith	jerrysanders@example.org	9116591761	Male	295	EditDelete
2	Michael Novak	kwalker@example.net	8151446400	Female	145	EditDelete
3	Laura Johnson	dennisjones@example.org	9953899989	Male	41	EditDelete
4	Dana Warner	barrettmichael@example.net	6314560257	Female	284	EditDelete
5	Julie Miller	thomas43@example.net	0861786873	Male	180	EditDelete
6	Lauren Weber	garciachristopher@example.net	1849194403	Other	144	EditDelete
7	Sean West	johnhenry@example.org	5135844157	Other	145	EditDelete
8	Julie Reynolds	jeremiah36@example.com	1150671200	Male	166	EditDelete
9	Chelsea McKinney	wfowler@example.org	0344941172	Other	182	EditDelete
10	Megan Larsen	james62@example.com	7935728931	Other	183	EditDelete
11	Deanna Frey DDS	tsingh@example.org	6037994609	Male	180	EditDelete
12	Emily Glenn	ericmays@example.com	3243316279	Male	129	EditDelete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

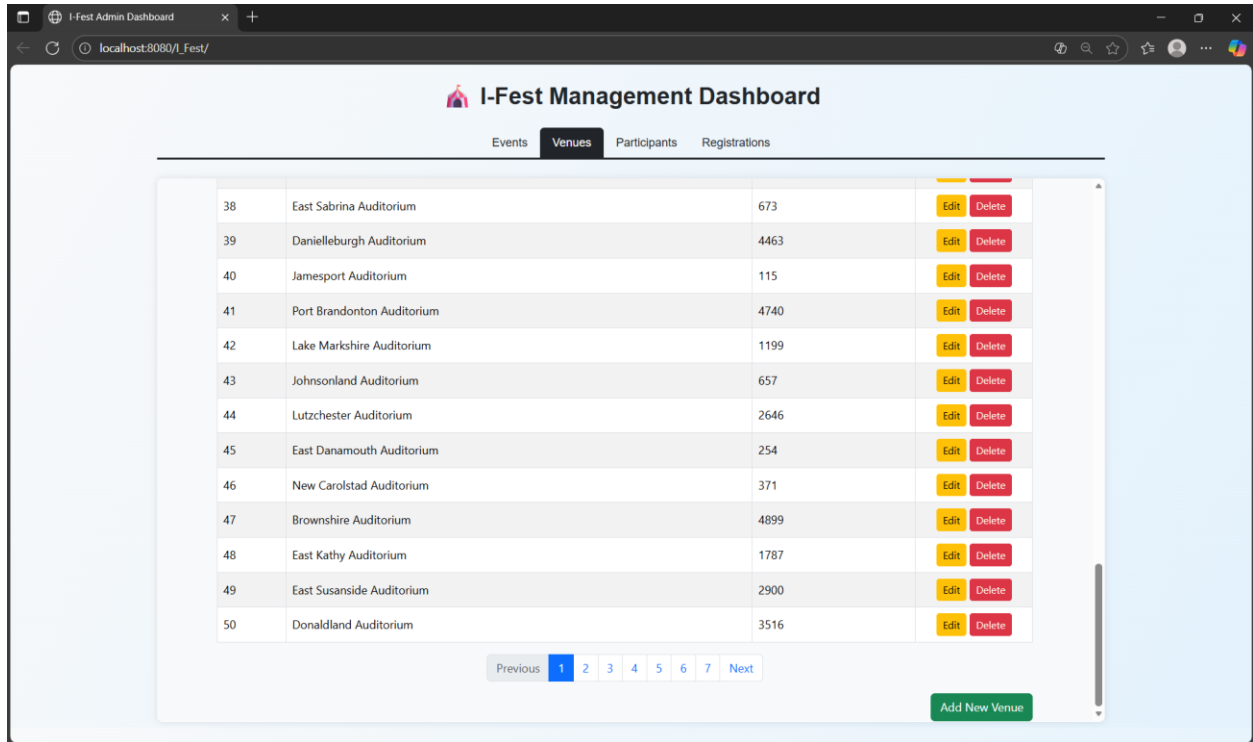
Registrations Management

Search by Participant ID, Event ID or Team ID

Search

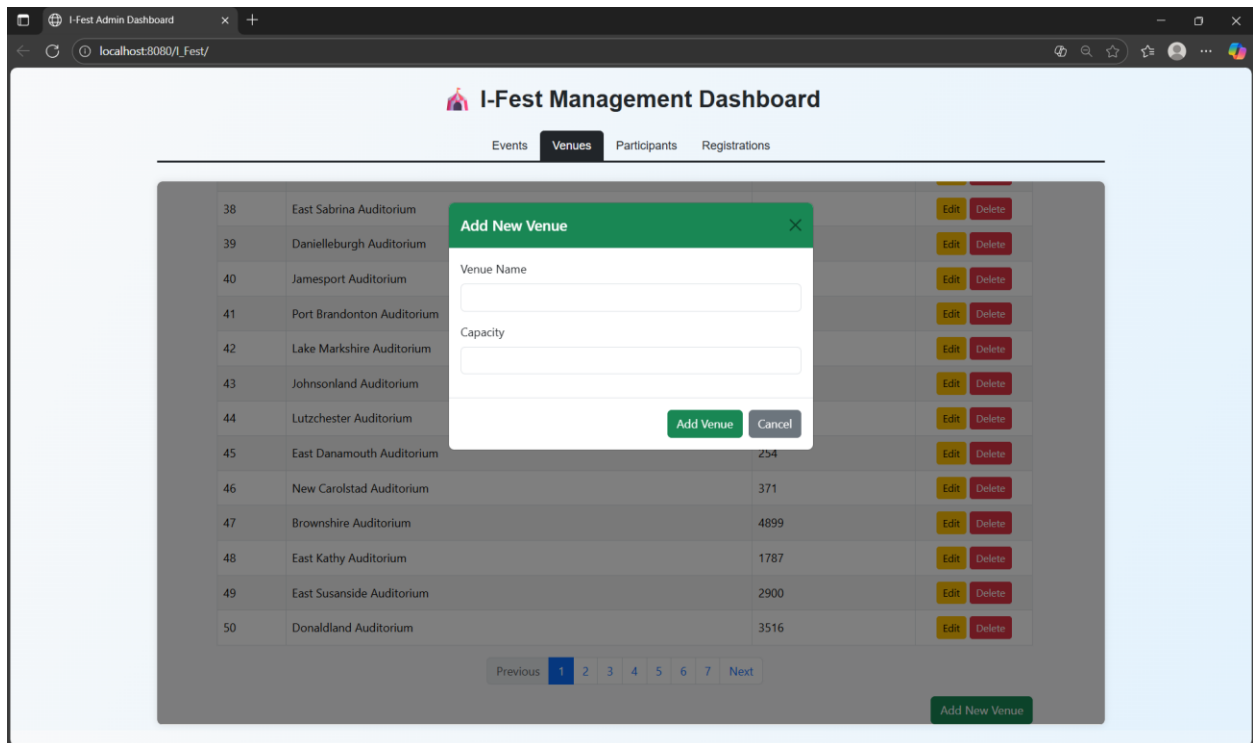
ID	Participant ID	Team ID	Event ID	Registered At	Actions
1	893	8	6	2025-10-28 16:46:32.978264	EditDelete
2	262	316	61	2025-10-28 16:46:32.978264	EditDelete
3	337	335	79	2025-10-28 16:46:32.978264	EditDelete
4	32	129	11	2025-10-28 16:46:32.978264	EditDelete
5	366	245	68	2025-10-28 16:46:32.978264	EditDelete
6	939	238	70	2025-10-28 16:46:32.978264	EditDelete
7	500	306	13	2025-10-28 16:46:32.978264	EditDelete
8	1056	82	15	2025-10-28 16:46:32.978264	EditDelete
9	268	265	36	2025-10-28 16:46:32.978264	EditDelete
11	972	182	13	2025-10-28 16:46:32.978264	EditDelete
12	1053	139	72	2025-10-28 16:46:32.978264	EditDelete
13	457	95	32	2025-10-28 16:46:32.978264	EditDelete

Add New Record Form :



The screenshot shows the I-Fest Management Dashboard with the 'Venues' tab selected. The dashboard displays a table of venues with their names and capacities. At the bottom right, there is a green button labeled 'Add New Venue'.

ID	Venue Name	Capacity	Edit	Delete
38	East Sabrina Auditorium	673	Edit	Delete
39	Danielleburgh Auditorium	4463	Edit	Delete
40	Jamesport Auditorium	115	Edit	Delete
41	Port Brandon Auditorium	4740	Edit	Delete
42	Lake Markshire Auditorium	1199	Edit	Delete
43	Johnsonland Auditorium	657	Edit	Delete
44	Lutchester Auditorium	2646	Edit	Delete
45	East Danamouth Auditorium	254	Edit	Delete
46	New Carolstad Auditorium	371	Edit	Delete
47	Brownshire Auditorium	4899	Edit	Delete
48	East Kathy Auditorium	1787	Edit	Delete
49	East Susanside Auditorium	2900	Edit	Delete
50	Donaldland Auditorium	3516	Edit	Delete



The screenshot shows the I-Fest Management Dashboard with the 'Venues' tab selected. An 'Add New Venue' modal form is open, allowing users to add a new venue. The modal contains fields for 'Venue Name' and 'Capacity', and buttons for 'Add Venue' and 'Cancel'.

ID	Venue Name	Capacity	Edit	Delete
38	East Sabrina Auditorium	673	Edit	Delete
39	Danielleburgh Auditorium	4463	Edit	Delete
40	Jamesport Auditorium	115	Edit	Delete
41	Port Brandon Auditorium	4740	Edit	Delete
42	Lake Markshire Auditorium	1199	Edit	Delete
43	Johnsonland Auditorium	657	Edit	Delete
44	Lutchester Auditorium	2646	Edit	Delete
45	East Danamouth Auditorium	254	Edit	Delete
46	New Carolstad Auditorium	371	Edit	Delete
47	Brownshire Auditorium	4899	Edit	Delete
48	East Kathy Auditorium	1787	Edit	Delete
49	East Susanside Auditorium	2900	Edit	Delete
50	Donaldland Auditorium	3516	Edit	Delete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

38	East Sabrina Auditorium		EditDelete
39	Danielleburgh Auditorium		EditDelete
40	Jamesport Auditorium		EditDelete
41	Port Brandonon Auditorium		EditDelete
42	Lake Markshire Auditorium		EditDelete
43	Johnsonland Auditorium		EditDelete
44	Lutzchester Auditorium		EditDelete
45	East Danamouth Auditorium	254	EditDelete
46	New Carolstad Auditorium	371	EditDelete
47	Brownshire Auditorium	4899	EditDelete
48	East Kathy Auditorium	1787	EditDelete
49	East Susanside Auditorium	2900	EditDelete
50	Donaldland Auditorium	3516	EditDelete

Previous1234567Next

Add New Venue

Add New Venue

Venue Name

DAIICT Lab-5

Capacity

150

Add VenueCancel

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

Venues Management

Venue added successfully!

Search by venue name

Search

ID	Venue Name	Capacity	Actions
1	Danaview Auditorium	2105	EditDelete
2	East Jeremyberg Auditorium	3426	EditDelete
3	Adamsview Auditorium	1807	EditDelete
4	Brandytown Auditorium	121	EditDelete
5	New Jonathanport Auditorium	412	EditDelete
6	North pine Auditorium	1913	EditDelete
7	East Peter Auditorium	893	EditDelete
8	Port Catherine Auditorium	2744	EditDelete
9	Morgantown Auditorium	2928	EditDelete
10	Crossfort Auditorium	4937	EditDelete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

339	Schroederton Auditorium	2844	Edit	Delete
340	New Angela Auditorium	4017	Edit	Delete
341	Deborahville Auditorium	2334	Edit	Delete
342	East Erin Auditorium	945	Edit	Delete
343	Loweryview Auditorium	4527	Edit	Delete
344	Hannahville Auditorium	285	Edit	Delete
345	Rayport Auditorium	2016	Edit	Delete
346	Jacksonview Auditorium	1044	Edit	Delete
347	Swansonshire Auditorium	1437	Edit	Delete
348	New Kaithlynfort Auditorium	1825	Edit	Delete
349	New Elizabethtown Auditorium	2287	Edit	Delete
350	New Kevinborough Auditorium	761	Edit	Delete
351	DAIICT Lab-5	150	Edit	Delete

[Previous](#) [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [Next](#)

Add New Venue

Edit Records :

The screenshot shows the 'I-Fest Management Dashboard' with the 'Participants' tab selected. A modal titled 'Edit Participant #1' is open, allowing editing of participant data. The modal contains fields for Name, Email, Phone, Gender, and Team ID. The background shows a table of participants with columns for ID, Name, Email, Phone, Gender, Team ID, and Actions.

ID	Name	Email	Phone	Gender	Team ID	Actions
1	Misty Smith	jerrysanders@example.org	9116591761	Male	295	Edit Delete
2	Michael Novak	kwallace@example.org	9116591761	Male	145	Edit Delete
3	Laura Johnson	dennishill@example.org	9116591761	Male	41	Edit Delete
4	Dana Warner	barreter@example.org	9116591761	Male	284	Edit Delete
5	Julie Miller	thomson@example.org	9116591761	Male	180	Edit Delete
6	Lauren Weber	garciaperez@example.org	9116591761	Male	144	Edit Delete
7	Sean West	johnson@example.org	9116591761	Male	145	Edit Delete
8	Julie Reynolds	jerrysanders@example.org	9116591761	Male	166	Edit Delete
9	Chelsea McKinney	wfowler@example.org	9116591761	Male	182	Edit Delete
10	Megan Larsen	james62@example.com	9116591761	Male	183	Edit Delete
11	Deanna Frey DDS	tsingh@example.org	9116591761	Male	180	Edit Delete
12	Emily Glenn	ericmays@example.com	9116591761	Male	129	Edit Delete

The screenshot shows the 'I-Fest Management Dashboard' with the 'Participants' tab selected. A modal titled 'Edit Participant #1' is open, allowing editing of participant data. The modal contains fields for Name, Email, Phone, Gender, and Team ID. The background shows a table of participants with columns for ID, Name, Email, Phone, Gender, Team ID, and Actions.

ID	Name	Email	Phone	Gender	Team ID	Actions
1	Misty Patel	mistypatel@gmail.com	9116591761	Female	295	Edit Delete
2	Michael Novak	kwallace@example.org	9116591761	Male	145	Edit Delete
3	Laura Johnson	dennishill@example.org	9116591761	Male	41	Edit Delete
4	Dana Warner	barreter@example.org	9116591761	Male	284	Edit Delete
5	Julie Miller	thomson@example.org	9116591761	Male	180	Edit Delete
6	Lauren Weber	garciaperez@example.org	9116591761	Male	144	Edit Delete
7	Sean West	johnson@example.org	9116591761	Male	145	Edit Delete
8	Julie Reynolds	jerrysanders@example.org	9116591761	Male	166	Edit Delete
9	Chelsea McKinney	wfowler@example.org	9116591761	Male	182	Edit Delete
10	Megan Larsen	james62@example.com	9116591761	Male	183	Edit Delete
11	Deanna Frey DDS	tsingh@example.org	9116591761	Male	180	Edit Delete
12	Emily Glenn	ericmays@example.com	9116591761	Male	129	Edit Delete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 I-Fest Management Dashboard

EventsVenuesParticipantsRegistrations

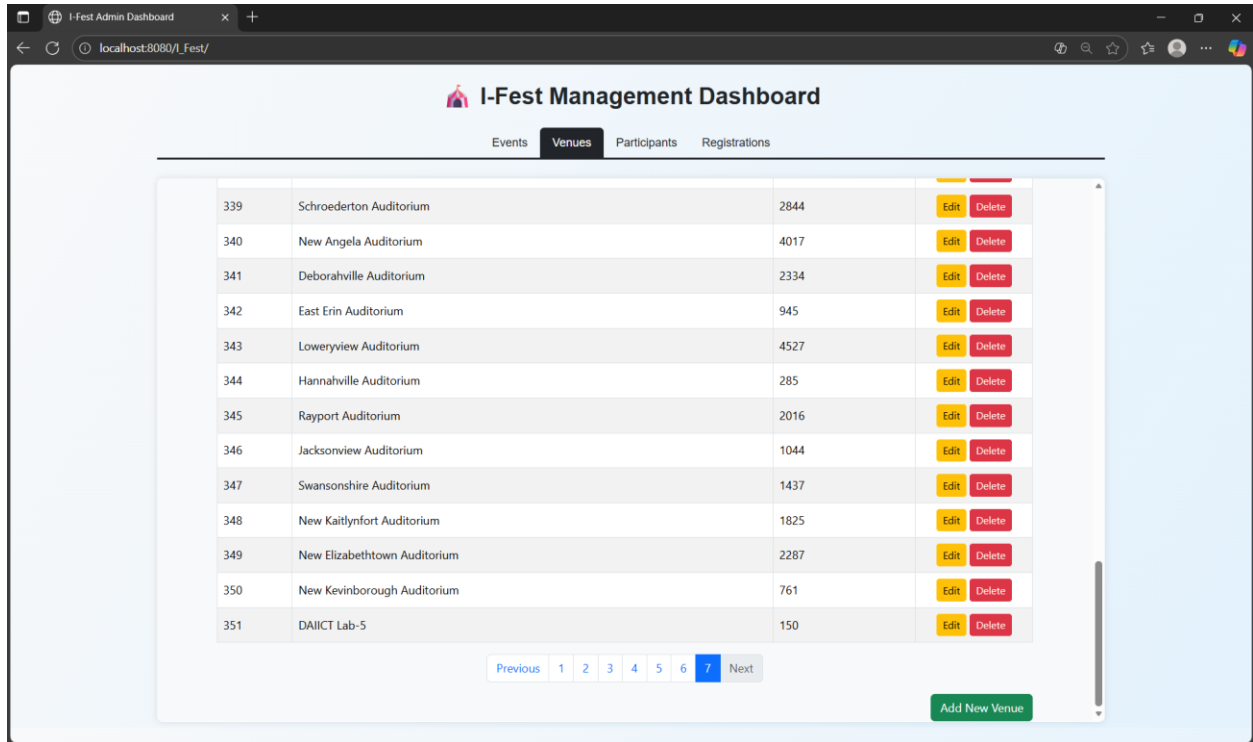
Participants Management

Search by Participant Name or Email

Search

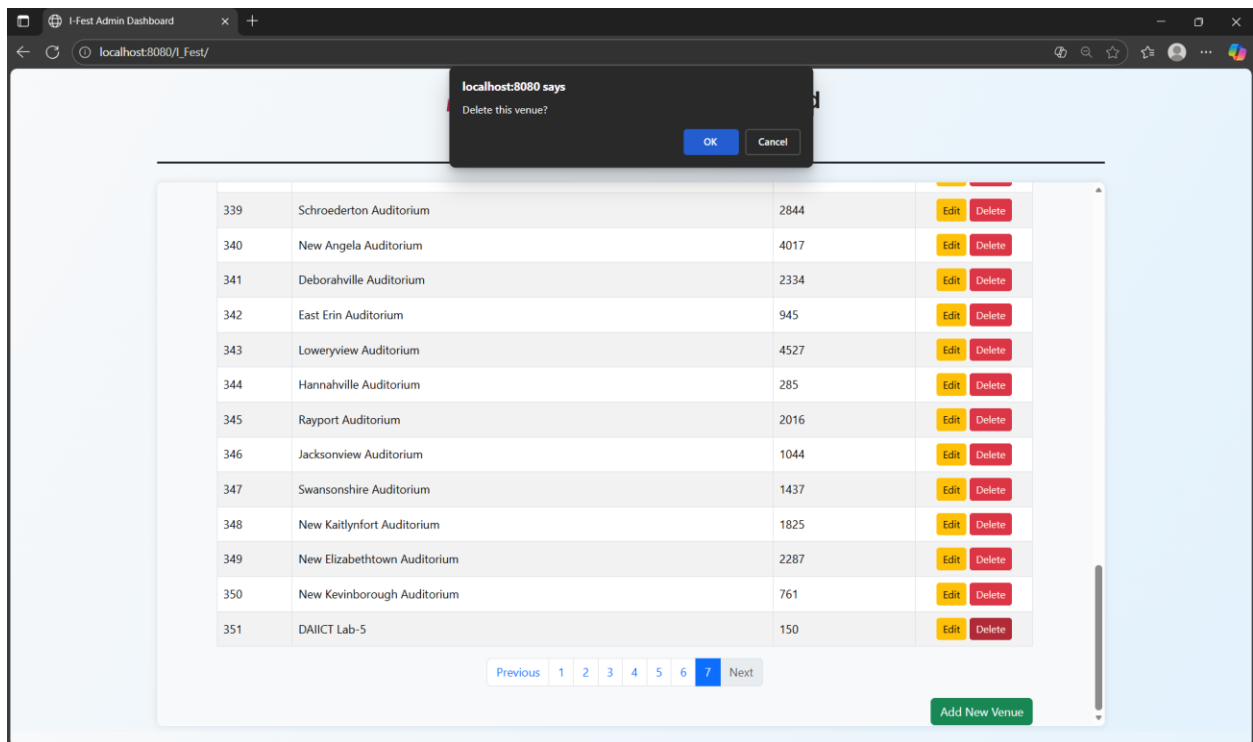
ID	Name	Email	Phone	Gender	Team ID	Actions
1	Misty Patel	mistypatel@gmail.com	9116591761	Female	295	EditDelete
2	Michael Novak	kwalker@example.net	8151446400	Female	145	EditDelete
3	Laura Johnson	dennisjones@example.org	9953899989	Male	41	EditDelete
4	Dana Warner	barrettmichael@example.net	6314560257	Female	284	EditDelete
5	Julie Miller	thomas43@example.net	0861786873	Male	180	EditDelete
6	Lauren Weber	garciachristopher@example.net	1849194403	Other	144	EditDelete
7	Sean West	johnhenry@example.org	5135844157	Other	145	EditDelete
8	Julie Reynolds	jeremiah36@example.com	1150671200	Male	166	EditDelete
9	Chelsea McKinney	wfowler@example.org	0344941172	Other	182	EditDelete
10	Megan Larsen	james62@example.com	7935728931	Other	183	EditDelete
11	Deanna Frey DDS	tsingh@example.org	6037994609	Male	180	EditDelete
12	Emily Glenn	ericmays@example.com	3243316279	Male	129	EditDelete

Delete Records :



The screenshot shows the I-Fest Management Dashboard with the 'Venues' tab selected. A table lists 12 venues with their IDs, names, and registration counts. Each row has 'Edit' and 'Delete' buttons. The 'Delete' button for the first venue (ID 339) is highlighted in red. Below the table is a pagination bar with 'Previous', '1', '2', '3', '4', '5', '6', '7', and 'Next'. A green 'Add New Venue' button is at the bottom right.

ID	Venue Name	Registrations	Edit	Delete
339	Schroederton Auditorium	2844	Edit	Delete
340	New Angela Auditorium	4017	Edit	Delete
341	Deborahville Auditorium	2334	Edit	Delete
342	East Erin Auditorium	945	Edit	Delete
343	Loweryview Auditorium	4527	Edit	Delete
344	Hannahville Auditorium	285	Edit	Delete
345	Rayport Auditorium	2016	Edit	Delete
346	Jacksonview Auditorium	1044	Edit	Delete
347	Swansonshire Auditorium	1437	Edit	Delete
348	New Kaitlynfort Auditorium	1825	Edit	Delete
349	New Elizabethtown Auditorium	2287	Edit	Delete
350	New Kevinborough Auditorium	761	Edit	Delete
351	DAIICT Lab-5	150	Edit	Delete



The screenshot shows the same I-Fest Management Dashboard with the 'Venues' tab selected. A confirmation dialog box is displayed in the center, asking 'Delete this venue?' with 'OK' and 'Cancel' buttons. The dialog box is titled 'localhost:8080 says'. The table and pagination bar are visible in the background.

ID	Venue Name	Registrations	Edit	Delete
339	Schroederton Auditorium	2844	Edit	Delete
340	New Angela Auditorium	4017	Edit	Delete
341	Deborahville Auditorium	2334	Edit	Delete
342	East Erin Auditorium	945	Edit	Delete
343	Loweryview Auditorium	4527	Edit	Delete
344	Hannahville Auditorium	285	Edit	Delete
345	Rayport Auditorium	2016	Edit	Delete
346	Jacksonview Auditorium	1044	Edit	Delete
347	Swansonshire Auditorium	1437	Edit	Delete
348	New Kaitlynfort Auditorium	1825	Edit	Delete
349	New Elizabethtown Auditorium	2287	Edit	Delete
350	New Kevinborough Auditorium	761	Edit	Delete
351	DAIICT Lab-5	150	Edit	Delete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 **I-Fest Management Dashboard**

EventsVenuesParticipantsRegistrations

Venues Management

Venue deleted successfully!

Search by venue name

Search

ID	Venue Name	Capacity	Actions
1	Danaview Auditorium	2105	Edit Delete
2	East Jeremyberg Auditorium	3426	Edit Delete
3	Adamsview Auditorium	1807	Edit Delete
4	Brandytown Auditorium	121	Edit Delete
5	New Jonathanport Auditorium	412	Edit Delete
6	North pine Auditorium	1913	Edit Delete
7	East Peter Auditorium	893	Edit Delete
8	Port Catherine Auditorium	2744	Edit Delete
9	Morgantown Auditorium	2928	Edit Delete
10	Crossfort Auditorium	4937	Edit Delete

I-Fest Admin Dashboard

localhost:8080/I_Fest/

 **I-Fest Management Dashboard**

EventsVenuesParticipantsRegistrations

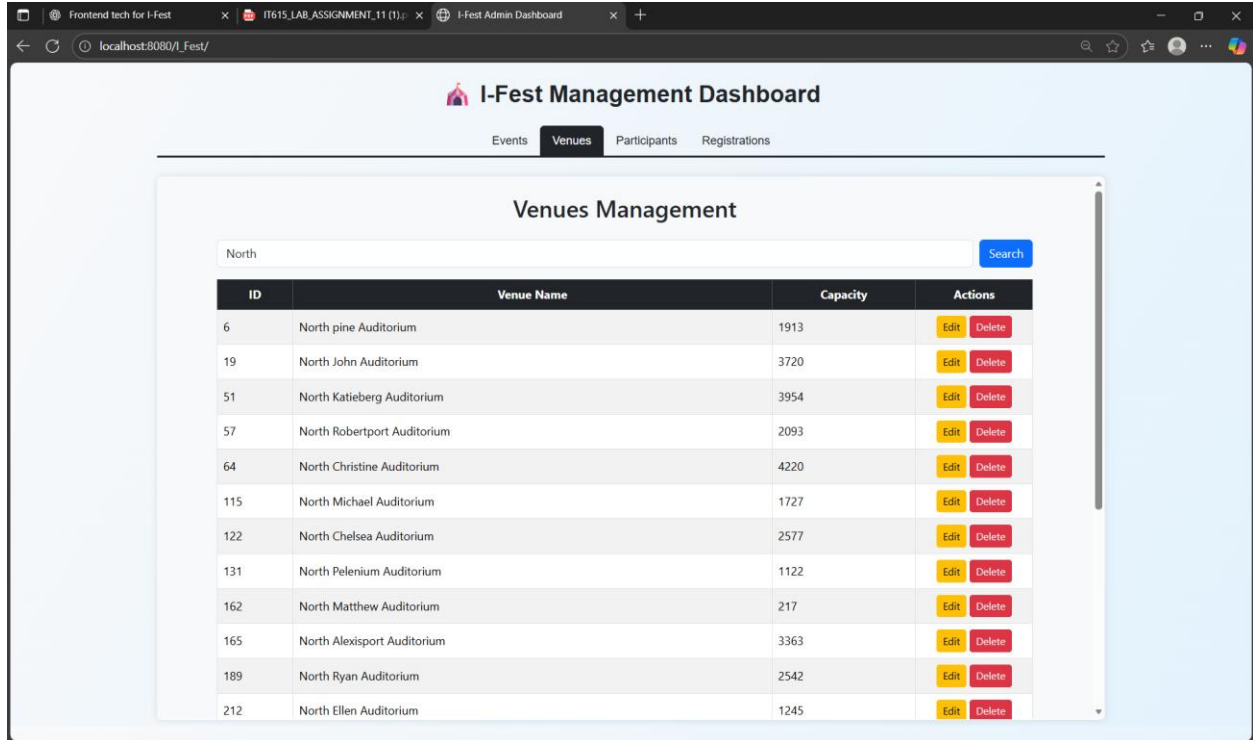
338	North Krista Auditorium	3950	Edit Delete
339	Schroederton Auditorium	2844	Edit Delete
340	New Angela Auditorium	4017	Edit Delete
341	Deborahville Auditorium	2334	Edit Delete
342	East Erin Auditorium	945	Edit Delete
343	Loweryview Auditorium	4527	Edit Delete
344	Hannahville Auditorium	285	Edit Delete
345	Rayport Auditorium	2016	Edit Delete
346	Jacksonview Auditorium	1044	Edit Delete
347	Swansonshire Auditorium	1437	Edit Delete
348	New Kaitlynfort Auditorium	1825	Edit Delete
349	New Elizabethtown Auditorium	2287	Edit Delete
350	New Kevinborough Auditorium	761	Edit Delete

[Previous](#) [1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [Next](#)

Add New Venue

localhost:8080/I_Fest/venues?page=7

Search Functionality :



The screenshot displays the I-Fest Management Dashboard in a web browser. The dashboard has a navigation bar with tabs for Events, Venues, Participants, and Registrations. The Venues tab is active, showing the Venues Management section. At the top of this section is a search bar with the text "North" and a "Search" button. Below the search bar is a table with the following data:

ID	Venue Name	Capacity	Actions
6	North pine Auditorium	1913	Edit Delete
19	North John Auditorium	3720	Edit Delete
51	North Katieberg Auditorium	3954	Edit Delete
57	North Robertport Auditorium	2093	Edit Delete
64	North Christine Auditorium	4220	Edit Delete
115	North Michael Auditorium	1727	Edit Delete
122	North Chelsea Auditorium	2577	Edit Delete
131	North Pelenium Auditorium	1122	Edit Delete
162	North Matthew Auditorium	217	Edit Delete
165	North Alexisport Auditorium	3363	Edit Delete
189	North Ryan Auditorium	2542	Edit Delete
212	North Ellen Auditorium	1245	Edit Delete

13. CONCLUSION

Challenges Faced :

While building the CRUD-based web application, I encountered multiple challenges across connectivity, backend logic, and UI integration. One of the major issues was dealing with database connection errors—especially incorrect JDBC URLs, driver loading issues, and SQLExceptions caused by invalid queries or missing parameters. Handling foreign-key relationships also became tricky, particularly when events needed venue names instead of venue IDs, which required proper JOIN queries. Pagination and search filters introduced additional bugs such as incorrect page numbers or empty results due to parameter mismatches. I also faced UI-related challenges when using JSP modals for Edit/Add operations, because the servlet failed to receive form parameters if the names didn't exactly match. Debugging NULL values in columns like "category" and ensuring UTF-8 encoding for special characters also took time.

Lessons Learnt :

I learned how essential proper DBMS design is when building a full-stack Java application. Setting up tables with correct data types, foreign keys, and constraints helped maintain data integrity, especially when linking Events with Venues or Participants with Registrations. Implementing CRUD operations taught me how to write safe and efficient SQL using prepared statements, JOINS, LIMIT/OFFSET for pagination, and handling NULL values correctly. I also understood how servlets interact with the database through JDBC—how data flows from the UI form → servlet → SQL query → Java objects → JSP—and how even a small mismatch in column names or parameters can break the pipeline. Overall, this project strengthened my understanding of connecting the UI, backend logic, and database safely and efficiently.